

A PROJECT REPORT ON

**A Hybrid IoT-Cloud Architecture with Quantum
Computing Support for Early Detection of Pest and
Disease in Maize Crop**

**A Major Project Submitted to Jawaharlal Nehru Technological University, Kakinada in Partial
Fulfillments of Requirements for the Award of the Degree of
BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING
(Internet of Things)**



Submitted By

Mr. M. Durga Praneeth (22KT1A4940)

Ms. A. Tanuja (22KT1A4901)

Mr. K. Vishnu Mohan (22KT1A4933)

Ms. R. Sai Bhavya (22KT1A4920)

Under the Esteemed Guidance of

Dr. Sri Silpa P

Associate Professor & HoD

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (Internet of Things)

POTTI SRIRAMULU CHALAVADI MALLIKARJUNA RAO

COLLEGE OF ENGINEERING & TECHNOLOGY

(Approved by AICTE New Delhi, Affiliated to JNTU-Kakinada)

KOTHAPET, VIJAYAWADA-520001, A.P

2022-2026

**POTTI SRIRAMULU CHALAVADI MALLIKARJUNA RAO
COLLEGE OF ENGINEERING & TECHNOLOGY
KOTHAPET, VIJAYAWADA-520001**

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(Internet of Things)**



CERTIFICATE

This is to certify that the project work entitled **“A Hybrid IoT-Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Disease in Maize Crops”** is a bonafide work carried out by **Mr. M. Durga Praneeth (22KT1A4940), Ms. A. Tanuja (22KT1A4901), Mr. K. Vishnu Mohan (22KT1A4933), Ms. R. Sai Bhavya (22KT1A4920).** Fulfillment for the award of the degree of Bachelor of Technology in **COMPUTER SCIENCE & ENGINEERING (Internet of Things)** of Jawaharlal Nehru Technological University, Kakinada during the year **2025-2026**. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the above degree.

Project Guide

Head of the Department

External Examiner

ACKNOWLEDGEMENT

We owe a great many thanks to a great many people who helped and supported and suggested us in every step. We are glad for having the support of our principal **Dr. S. Saravana Kumar** who inspired us with his words filled with dedication and discipline towards work. We express our gratitude towards **Dr. Sri Silpa P, Associate Processor & HOD** for extending her support through technical and motivation classes which had been the major source for carrying out our project. We are very much thankful to her for guiding and correcting various documents of ours with attention and care. She has taken the pain to go through the project and make necessary corrections as and when needed. Finally, we thank one and all who directly and indirectly helped us to complete our project successfully.

Project Associates

Mr. M. Durga Praneeth (22KT1A4940)

Ms. A. Tanuja (22KT1A4901)

Mr. K. Vishnu Mohan (22KT1A4933)

Ms. R. Sai Bhavya (22KT1A4920)

DECLARATION

This is to declare that the project entitled “**A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops**” submitted by us in the partial fulfillment of requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering(Internet of Things) in Potti Sriramulu Chalavadi Mallikharjuna Rao College of Engineering and Technology, is bonafide record of project work carried out by us under the guidance of **Dr. Sri Silpa P, Associate Professor & HoD**. As per our knowledge, the work has not been submitted to any other institute or universities for any other degree.

Project Associates

Mr. M. Durga Praneeth (22KT1A4940)

Ms. A. Tanuja (22KT1A4901)

Mr. K. Vishnu Mohan (22KT1A4933)

Ms. R. Sai Bhavya (22KT1A4920)

ABSTRACT

Maize is the one of the major crop in the world, but its yield is highly affected by pests and diseases. To reduce the crop loss and to improve the yield early detection is necessary. To overcome this maize crop disease and pest detection system using IoT, Cloud and Quantum Computing techniques. The system analyzes maize images and classifies them as healthy or diseased. It also identifies the specific disease type and gives the recommendations about what medicine it should use to prevent the disease/pest along with the dosage. Roboflow is used for accurate identification of the images. Leaf images are used for preprocessing techniques. To remove the noise by keeping important edges clear Bilateral filtering is applied. To enhance contrast and patterns Adaptive Bilateral-CLAHE filter is applied. The images are resized, normalized and additional feature extraction methods such as texture analysis, edge detection and color histograms are used to capture important leaf characteristics. Both classical and hybrid approaches are implemented for the classification. To learn the features automatically from the images CNN and YOLO is designed with UniTanh Rectifier. Pre trained models like NASNetMobile are used for feature extraction. For binary classification a quantum model is introduced. The extracted features are optimized using PCA and transfer them into a 4 qubit quantum circuit, which converts them into quantum enhanced features. For the final prediction the features are passed into to a neural network, achieving around 99% validation accuracy. For multi-class classification a classical neural network with strong regularization techniques is used to avoid overfitting. The model gets accuracy around 99%. The model integrates with the IoT and cloud technologies which uses the different IoT sensors like temperature and humidity sensor, soil moisture sensor and NPK sensors and for the cloud integration the firebase database is used for the results. The outcomes shows that quantum methods increases simple models, while classical deep learning performs better for complex multi-class problems. The proposed system provides an accurate solution for smart maize disease detection using the streamlit web application showing the live sensor values and predications when the image uploaded with remedies.

CONTENTS

S.No	TOPIC	Page No
1	INTRODUCTION	1
2	SYSTEM ANALYSIS	10
3	SYSTEM DESIGN	22
4	SYSTEM IMPLEMENTATION	35
5	TESTING	101
6	CONCLUSION	104
7	FUTURE WORK	105
8	REFERENCES	106
9	BIBILOGRAPHY	109
10	APPENDIX	110

LIST OF FIGURES

S.NO	Fig No	Name of the Figure	Page No
1	2.1	Raspberry Pi	16
2	2.2	Arduino UNO	17
3	2.3	DHT11	17
4	2.4	Soil Moisture Detector Sensor	18
5	2.5	NPK Sensor	18
6	2.6	Convertor	19
7	2.7	Bread Board	19
8	2.8	Adapter	20
9	3.1	System Architecture	25
10	3.2	Circuit Diagram	26
11	3.3	External Entity Representation	26
12	3.4	Process representation	27
13	3.5	Data Flow Representation	27
14	3.6	Warehouse/ Datastore Representation	27
15	3.7	Level 0 DFD	28
16	3.8	Level 1 DFD	29
17	3.9	Use Case Diagram	29
18	3.10	Class Diagram	30
19	3.11	Activity Diagram	31
20	3.12	Sequence Diagram	32
21	3.13	Deployment Diagram	33
22	4.1	Filtered Applied Image	82
23	4.2	Feature Extraction Result	82
24	4.3	Feature Extraction for Standard CNN	83
25	4.4	Feature Extraction Applied for UniTanh Rectifier CNN	83
26	4.5	Feature Extraction Applied for Standard YOLO	84
27	4.6	Feature Extraction Applied for UniTanh Rectifier YOLO	84
28	4.7	Binary Classification For CNN	85
29	4.8	Confusion Matrix for CNN applied Binary Classification	85

30	4.9	Accuracy vs Epochs and Loss vs Epochs for CNN model	86
31	4.10	Binary Classification for YOLO	86
32	4.11	YOLO applied Binary Classification Confusion Matrix	87
33	4.12	Feature Classifier Accuracy vs Epochs and Feature Classifier Loss vs Epochs for YOLO Model	87
34	4.13	NASNET Binary Classification	88
35	4.14	NASNet + SVM Binary Confusion Matrix	88
36	4.15	Cross Validation Accuracy and Training vs Accuracy graphs	89
37	4.16	Binary Classification Between Healthy vs Diseased	89
39	4.17	Binary Classification Between Diseased vs Pest	90
40	4.18	Confidence for image prediction	90
41	4.19	Multi Classification Confusion Matrix	91
42	4.20	Multi Classification Model Accuracy	91
43	4.21	Class wise Accuracy	92
44	4.22	Class wise Precision, Recall and F1 graph	92
45	4.23	Multi Class ROC Curve Graph	93
46	4.24	Multi Classification for Random Images	94
47	4.25	Quantum Applied Accuracy Graph for Binary Classification	94
48	4.26	Confusion Matrix for Quantum Applied Binary Classification	95
49	4.27	Quantum Applied Accuracy Graph for Multi Classification	95
50	4.28	Quantum Applied Confusion Matrix for Multi Classification	96
51	4.29	Quantum Applied Confidence for Binary and Multi classification	96
52	4.30	Quantum Applied Binary Classification	97
53	4.31	Quantum Applied Multi Classification	97
54	4.32	Live Sensor Data when the device is disconnected	98
55	4.33	Live Sensor Data when the device is Connected	98
56	4.34	Firestore database saving sensor data in the current condition and update condition	99
57	4.35	Disease Prediction and Treatment Recommendation Interface	100

LIST OF TABLES

S.No	Table No	Name of the Table	Page No
1	1.1	Comparative Analysis on Existence System	7
2	5.1	Validation of Test Cases	102

**A Hybrid IoT-Cloud Architecture with
Quantum Computing Support for Early
Detection of Pests and Diseases in Maize
Crops**

INTRODUCTION

CHAPTER – 1 INTRODUCTION

1.1 BRIEF OVERVIEW OF THE PROJECT: Maize is a cornerstone of global food security yet its productivity is constantly threatened by a spectrum of leaf diseases and pest infestations. The proposed intelligent system addresses this critical challenge by combining state of the art image processing, deep learning, and quantum-enhanced techniques to perform real-time detection and classification of maize leaf pathology. By automating this workflow, the solution eliminates labour intensive visual inspections, enables early disease diagnosis, and helps farmers and agronomists take timely, cost-effective actions to safeguard yields.

The system initiates with sophisticated preprocessing pipelines including Adaptive Bilateral-CLAHE edge detection, and texture feature extraction to enhance image quality and highlight disease-specific patterns. These processed images are then fed into lightweight yet powerful convolutional back-bones (NASNetMobile, MobileNetV2, and YOLOv8, CNN) that extract discriminative deep features. For binary classification (Healthy ↔ Diseased/Pest) a Quantum feature transformation layer built with PennyLane is employed to capture subtle, non-linear relationships that might escape classical models, thereby boosting classification accuracy. Multi-class scenarios are handled by a robust classical deep neural network trained on an expanded feature set, ensuring reliable identification of a variety of pathogens and pests. The project then integrated with the IoT technology with the different sensors. The data from sensors goes through the firebase and the values will store in the database and then for disease prediction the streamlit is used in the web page the live sensor data will update when the sensors are in working else is shows device is not connected. And when the disease is predicted it gives remedy for the disease and data will come from the firestore database.

By integrating photonic-quality preprocessing, efficient deep feature extraction, and quantum-enhanced classification, this project delivers a high-accuracy, scalable, and automated platform for maize disease surveillance. The resulting system not only enhances crop protection strategies but also provides a blue print for deploying quantum techniques across crops, advancing precision agriculture and global food security.

1.1.1 Scope: To design and develop a smart maize pest and disease detection system that will be working within well defined technical limits, such as image preprocessing, feature extraction, classification, and quantum enhancement. The project encompasses the image enhancement via Adaptive Bilateral-CLAHE to enhance the image in conjunction with the features extraction via CNN, YOLO, and NASNetMobile, MobileNet deep learning networks executed with the help of TensorFlow. It can be used to do binary classification (Healthy vs Diseased/Pest) and multi-class classification (seven classes), in which the feature extraction achieved by NASNetMobile is used together with deep neural networks that are optimized to enhance prediction accuracy. The binary classification step is provided with quantum computing utilizing variational circuits created with PennyLane to improve the representation of the low-dimensional features. The project involves assessment based on accuracy measures,

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

confusion table and visualization results. It also establishes the prospective growth to the IoT-based image acquisition and cloud-based deployment to real-time monitoring. But real quantum hardware implementation, large-scale commercial cloud infrastructure, pesticide recommendation systems and hardware level IoT manufacturing are not covered, making sure that the project does not lose track of what the intelligent detection and scalable architecture development. This is integrated with IoT sensors and streamlit application for the disease or pest detection with live sensor data.

1.1.2 Purpose: This project aims at solving the issue of poor and slow detection of maize pests and leaf diseases, which in most cases, results in massive crop losses and economic losses as a result of using manual pest detection methods and the scarcity of professional people. The proposed project will work on creating an intelligent and automated detection system based on advanced preprocessing methods (Adaptive Bilateral-CLAHE), deep learning architectures (NASNetMobile) trained using the TensorFlow platform, and quantum-enhanced feature processing with variational circuits implemented in PennyLane. The goals are to attain a binary classification (Healthy vs Diseased/Pest) training and validation accuracy of about 99%, multi-class classification accuracy of more than 99%, enhanced feature representation by transfer learning and a scalable architecture, which can be integrated into the IoT and cloud in a specific development time line. The expected outcomes are early and precise disease diagnosis, less reliance on manual monitoring, enhanced crop yields, economic empowerment of farmers and innovation in technology by the viable application of quantum computing in the agricultural industry to provide an intelligent, dependable, and forward-looking crop health targeting system. Integration It is possible to recognize the disease outbreaks and pest penetration immediately with the help of web application by embedding IoT sensors into the field, to which real-time data on temperature, humidity and other sensors will streamed continuously. This real-time connection enables farmers to get automatic advice at the same time turning passive monitoring into active and data-driven crop care.

1.1.3 Objective of the Study: This project extends the Quantum-Integrated Maize Pest and Disease Detection System into a complete IoT and cloud-based smart agriculture solution by integrating it with a web application developed using streamlit and cloud storage through Firebase Realtime Database. The objective of the study is to design and implement an intelligent, real-time maize health monitoring system that combines image based disease detection, quantum-enhanced machine learning, sensor-based environmental monitoring, and advisory support for farmers. The system enables users to upload maize leaf images through the web app which are then processed by the trained AI model deployed on the cloud. Simultaneously, IoT sensors placed in the field continuously monitor environmental parameters such as temperature, humidity, and soil & NPK conditions. These sensor readings are automatically updated in Firebase and displayed in the web application. Based on the detected pest or disease class and sensor conditions, the system provides appropriate remedies and preventive measures directly within the application creating a fully integrated IoT Cloud ecosystem for precision agriculture.

1.1.4 Literature Review: [1] D. Sheema et al, The article titled “An Algorithm for Detection and Identification of Infestation Density of Pest-Fall Armyworm in Maize Plants using Deep Learning based on IoT”. Detect & Identify pest infection in maize crops especially for the Fall Armyworm (FAW). In this article the main purpose is about to help farmers to identify hidden & early-stage pest in maize crop and to reduce the crop loss by preventing of using the chemical pesticides. In this Faster R-CNN and also R-SWFRN methods are used to train the model for detecting the pest by the odor-based detection with gas sensor & thermal camera to capture pest presence. For the model around 1000 thermal images are used which include both pest and without pest. The data is split into 70% of training & 30% for testing as well as validation. The proposed R-SWFRN model gets up to 98% of accuracy by this R-SWFRN the accuracy is improved to 6% compared to Faster R-CNN. Based on the size, shape, color with thermal cameras the pest is detected. The main advantage of this article is that it will detect the pest in early stage as well as it reduce the chemical usage. The limitations of the article that it contains only one pest disease, dataset and the performance of the model may vary in the different environmental conditions like wind & Temperature changes. The future work focuses on the multiple pest detection with real-time files implementation as well as a mobile or web application development with large dataset.

[2] Zongchen et al, The article titled “Maize leaf disease identification based on WG-MARNet”. The main objective of this article is to identify different maize leaf diseases using deep learning models which can handle noise, complex works & field conditions. This article help farmers and agriculture experts to detect maize leaf disease in early stage and prevent form crop loss. This article uses the WG-MARNet model this uses the wavelet threshold-guideline bilateral filtering to remove noise and this model split the images into high and low frequency components. The model has also integrated with the ResNet-based Deep Learning Technique with multi-scale features. In this there are 9 classes of disease for maize crop. For the training of the model they have collected 1,150 images of 9 common maize diseases. The images are resized into 224x224x3 which used the data augmentation techniques like rotation, flipping, brightness, saturation, gaussian. The main advantage of the dataset is that the images are collected in all the environmental conditions like sunny, cloudy and in all the day such as morning, afternoon conditions also. The model gained average accuracy of 97.4% and maximum accuracy of 97.9% with 0.278 seconds of image detection. The main advantage if this article is that it works on the noise reduction and also which is suitable for the filed conditions. The limitations of the article that it contains limited dataset and limited disease category which does not have cross-dataset validation and which does not implemented in the real- time. The future work focuses on the increases of crop diseases than existing once with real-time disease detection and allow to work on the edge devices with IoT-based monitoring.

[3] D. Sheema et al, The article titled “The Detection and Identification of Pest-FAW Infestation in Maize Crops Using Iot-Based Deep Learning Algorithms” The main objective of this article is about to identify the Fall Army worm infestation in maize crop at early stage with gas sensors, Thermal camera with deep learning algorithms. The article purpose is about the accurate detection and identification of the pest in the maize crop. For the training of the model data acquisition, pre-processing, feature extraction, object detection, methodologies are used for the

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

capturing of image, removing noise, color, shape, size and to identify the region of the pest. The article uses the Faster R-CNN and SWFRN (Speckle Warnes Faster R-CNN), ResNet, Inception V3 algorithms are used for Region proposal, Feature Extraction object detection. Faster R-CNN is used for better features SWFRN is used. The dataset has total 2,567 images in which 70% of the images for training 15% for testing and 15% for validation. The model gains up to 96.98% of accuracy. This model used the materials like gas sensor, Thermal camera and software components like python and tensorflow with deep learning algorithms. The main advantages of this article is about to detect the pest detection early with high accuracy in day and night weather conditions which integrates the IoT for real-time integration by this using of chemical pesticides are reduced. The limitations of the article that it contains limited resolution for thermal camera. The gas sensor influence environmental conditions. The article undergoes only one pest detection and the IoT hardware for such as microcontrollers are not specified. The future works overcomes with sensor fusion, multi pest and multi-class methods with field based dataset expansion with clear IoT hardware and protocol design.

[4] Annette Pfordt et al, The article titled “A review on detection and differentiation of maize diseases and pests by imaging sensors”. The objective of article is that review and evaluate the uses imaging sensors to the detection of maize disease and insect pests with emphasis on RGB with multispectral and hyperspectral imaging technologies. The main purpose is to provide comprehensive overview of imaging based techniques used for the identification of the maize diseases and pests. Article undergoes research papers studies and classified maize crop based on the RGB, multispectral and hyperspectral. In this the maize is categorised based on the leaf with disease and leaf with no disease as well as pest with analysed image acquisition platforms such as ground-based, UAV, satellite and data processing methods. The article compared both ML and DL techniques for disease detection and evaluated with advantages and limitations. For this article Random Forests, SVM, KNN, CNN, Pre-trained-CNN algorithms are used such as combination of both ML and DL. The total dataset has 10,000+ images in it in which 60-80% of them undergoes training and 15-30% for Testing as well as 10-20% for the validation. For training of the model approximately 30-50 EPOCHS are runned. The dataset has insect pests of 2 classes, Fungal disease of 3 classes, viral diseases of 2 classes. The model has gained the 80-99% of accuracy by using of the CNN and in the lab setup 99.9% of accuracy and in real field conditions it has gained the accuracy of 70-95% of accuracy. In this article uses some components like imaging sensors, satellite. The article uses the IoT technology with raspberry Pi + camera module and ESP32-CAM it uses the sensors like temperature, humidity, soil moisture, light intensity. The main advantage of this article is about that to detect the disease and pest detection in early stage with multiple imaging technologies like RGB, multispectral, thermal sensors, UAV's and satellite based system which helpful for the precision agriculture with a clear identification of challenges and future directions. The limitations of this article is about that similar symptoms across disease that cause the confusion of exact disease and RGB across camera fails in early-stage detection. The model has poor field condition. The future works focus on the multi sensor data fusion with spectral imaging for early detection and also the advanced deep learning models integrated with the agronomic data with cost effective sensor selection and finally with the decision support system integration.

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

[5] Ezeofor Chukwunazo Joseph et al, The article titled “Development of Smart IoT-Based CNN Technique for Harmful Maize Insects Recognition in Precision Agriculture”. The main objective of the article is to develop a smart IoT-based system combined with deep learning techniques to automatically detect, recognize, and classify harmful pests in maize crop in real time farming. The purpose of this article is that to provide an automated, accurate and remote monitoring system that supports early pest detection and timely control measures for the maize crop that helps farmers to reduce the crop loss. In this article the IoT hardware, deep learning models, cloud computing as well as the mobile application methodologies are used in this the IoT uses the devices like Raspberry Pi, Camera, LED and Wi-Fi in this the images are collected using delta pheromone traps and laboratory breeding. A custom CNN model called MothNet is trained using Google Colab due to the limited dataset ResNet-50 is applied for improving the performance of the model. The process includes by capturing the image and sent to the Amazon Cloud (AWS) for classification. The prediction results are stored in AWS S3 and accessed through and Android mobile application developed using Kotlin. There are 1,696 images are used in which 70% of the dataset is used for the training as well as validation and 30% for testing the all the images in the dataset are captured form the real farm and laboratory-bred insects. The model gets accuracy of ~90.37% for validation accuracy and for modified ResNet-50 validation accuracy increases to the 99.21% and in this model 3 types of pests diseases are trained such as Fall Army Worm (FAW), African Army Worm (AAW), Egyptian Cotton Leaf Worm (ECLW) each of the pest accuracy has changed such as for the FAW-97.91%, AAW-96.18%, ECLW-95.49%. The main advantage of this article is about to detect the disease in real-time with high accuracy which is integrated with the IoT and cloud technologies. The limitations of this article it suffers from overfitting due to the small dataset size. Raspberry Pi image quality is low especially at night. The future work focuses on improve the dataset with large number with improved image capture integrated with automated pesticide spraying and solar-powered IoT system.

[6] Shuai Yang et al, The article titled “Maize-YOLO: A New High-Precision and Real-Time Methods for Maize Pest Detection”. The objective of this article is about that to develop a high-precision and real time deep learning based pest detection model only for the maize crop and the purpose is to overcome the limitations of traditional pest monitoring methods by providing fast, accurate, and computationally efficient object detection system that can support precision agriculture and real-time pest management. The article uses the YOLO methodology especially YOLOv7 it also integrates CSPResNeXt-50 modules into YOLOv7 as a backbone to reduce the complexity of improving accuracy. VoVGSCSP module is introduced in the neck of the network to enhance feature reuse and improve detection speed. The performance of the model is evaluated by using the mAP, Precision, FLOPs, and FPS to measure accuracy, efficiency and real time capability. The model has total 4,533 images classified with 13 different maize pest the model under goes 80% of training 20% of testing with 5-fold cross-validation. The Mean Average Precision ([mAP@0.5](#)) is 76.3% and Recall with 77.3%. The detection speed of trained model is 67 FPS. The main advantages of this base paper is about that to detect the pest in real-time with high accuracy with efficient balance between speed and accuracy to handle complex field environments. The limitations of this base paper is that some pests show large visual

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

differences between larval and adult stages which causes reduces the detection accuracy. And the IoT is tested on the datasets not fully integrated on the fields. The future work focuses on the improvement of the detection across pests in early stage with IoT integration with a mobile and web application and to expand the other crops with automated spraying system in the fields.

[7] Prakash Sandhya et al, The article titled "Robust Fall Army Worm (FAW) detection in maize using multimodal RGB and thermal image fusion". The main objective is to develop a deep learning framework to automatically classify Fall Army Worm (FAW)-infested vs healthy maize by integrating RGB and thermal imagery with the goal of improving detection accuracy. The methodologies that is used in this model are the Feature-level Fusion & Image-level fusion. The feature-level Fusion separates CNNs extract features from RGB and thermal images these features are concatenated into a Deep Neural Network (DNN) classifier and the Image-level Fusion RGB and thermal images are concatenated into a single 6-channel image and processed by a modified vision transformer (ViT) for end-to-end classification. The custom dataset of 1,266 paired images (RGB +thermal) of maize and approximately balanced for 325FAW-infested and 308 healthy samples (1:1 ratio). In this model the common crop is Maize(corn) and the common disease/pest is Fall Army Worm (*Spodopetra frugipedra*) infestation in maize are mentioned. After the training of the Fused model the evaluation of 98% accuracy on test set and the baseline (no fusion) of 60% accuracy. The major Limitations of this model are the dataset constraints limited size and environmental diversity may reduce generalizability to other conditions and the Sensor Alignment Dependency Fusion strategy assumes inherently aligned RGB-thermal images (as from the FLR sensor used) performance may degrade if inputs are unaligned.

[8] T. Tchokogoué et al, The article titled "Towards precision agriculture: A dataset for early of corn leaf pests". The purpose of the model is to enable early and accurate identification of corn leaf pest attacks using machine learning and image processing techniques. The data collection is done as images captured in real life conditions across multiple locations in Cameroon taken at different growth stages and times of day are collected using various smartphone cameras. The dataset size are 1,308 original images and 848 infected leaf images data augmented up to 30,000+ images across different folders and preprocessing methods. Manual and automated background segmentation (CNDVI-based) High lighting infected regions and image resizing and augmentation for robustness of methodologies and processing are used in this model. Accuracy of high annotations reliability (no classifications accuracy reported since it is a dataset paper) and the Limitations are the limited size data is collected only from Cameroon and more regions and samples needed for broader generalization.

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

Tabel 1.1: Comparative Analysis on Existence System

S.No	Author Name	Algorithm Name	Merits	Demerits	Accuracy
1.	D. Sheema et al	Faster R-CNN, R-SWFRN	Improved accuracy over Faster R-CNN, good for early control	Time consuming training and hardware dependency is high	~98%
2.	Zongchen et al	WG-MARNet (Wavelet + Multi-channel ResNet)	Handle Noise & Background well with high accuracy	Complex architecture, high computation	97.96%
3.	D. Sheema et al	Faster R-CNN, R-SEFRN (odor-based)	Detects hidden pests using odor sensors, early detection with IoT Integration	Higher System cost, dataset is limited	96.98%
4.	Annette Pfordt et al	Review (RGB, Multispectral, Hyperspectral + ML/DL	Comprehensive survey, compares sensors & methods	No experimental accuracy, review only	N/A
5.	Ezeofor Chukwunazo Joseph et al	CNN (MothNet), ResNet-50 (Transfer Learning)	IoT + cloud + mobile app, real-time monitoring, practical deployment	Small dataset, reduced generalization in field	~85%-90%
6.	Shuai Yang et al	YOLOv7-based Maize-YOLO	Real-time detection, fast processing, good balance of speed & accuracy	Lower accuracy for very small pests, needs GPU	76.3%
7.	Prakash Sandhya et al	CNN + DNN + Vision Transformer (ViT)	Multimodal fusion improves robustness, works in low-light	Requires thermal camera, high cost	~98%
8.	T. Tchokogoué et al	No single fixed algorithm (Dataset designed for ML/DL models such as CNN, SVM, ResNet, etc.)	Real-field dataset, multiple pests covered, manual & automated segmentation, annotated infected regions, useful for ML/DL training and explainability	Not a complete detection model, limited number of samples, accuracy depends on the model used by researchers	N/A

1.2 PROBLEM STATEMENT: Maize crops are easily infested by several pests and leaf diseases that severely affect yield, quality and income of farmers. The traditional disease detection procedures primarily rely on the use of manual field inspection that is not only time-consuming but also labor-intensive and subject to human errors particularly in the rural regions where the number of experts is scarce. A late or erroneous identification in most cases leads to inappropriate use of pesticides, high cost of cultivating the produce and gross financial wastage. Despite a number of machine learning-based solutions that have been suggested, most of the current systems are yet to reach high accuracy of both binary (Healthy vs. Diseased/Pest) and multi-class classification and remain robust and scalable to real-field settings. Moreover, there are few studies that have investigated how to combine advanced optimization methods and quantum-enhanced learning to advance the performance of agricultural disease detection. Thus, to ensure the scalable, automated, and highly accurate detection of maize pests and diseases, there is an urgent need to create an automated system with advanced image preprocessing, deep feature extraction via TensorFlow models, and quantum-enhanced crops feature transformation via PennyLane to support the precision agriculture practice by early diagnosis. Other than identifying disease with the help of images, incorporation of the IoT technology gives the system more strength as it gathers real-time environmental data collected by sensors installed in the field, including temperature, humidity, and soil moisture. The information that is sensed is constantly sent to a Firebase cloud database where it is securely stored and updated in real-time. This real-time sensor data is then shown within a Streamlit web application dashboard where farmers are able to see the field conditions remotely. The Streamlit web application does not just visualize the real-time data of IoT but also incorporates the model of maize disease detection so that as soon as a photo is uploaded, instant prediction outcomes (healthy or a certain disease category) are displayed. The system will come up with recommendations of suitable treatment and preventive actions depending on the forecasted outcome and the environment. Such an integrated IoT-based smart web platform is used to guarantee the implementation of data-driven decision-making, timely action, and enhanced crop productivity, as well as minimize the waste of pesticides.

1.3 PROPOSED SYSTEM: The offered system is an intelligent, smart, and automated maize pest and disease monitoring system that would provide correct, timely, and early crop health monitoring. This model combines the latest image processing, deep learning, and quantum computing models to offer a scalable solution to smart agriculture nowadays. A high quality image preprocessing phase commences the workflow, and this step is important in enhancing the credibility of downstream analysis. The step here involves the use of an Adaptive Bilateral Filter to smooth maize leaf images effectively eliminating noise content in the images without destroying the important edge information and texture content. This makes sure that no important structural features of the leaf are lost in the process of denoising. This is followed by Contrast Limited Adaptive Histogram Equalization (CLAHE) which is used to increase the contrast of the images. CLAHE is an aid in bringing out the slight effects of disease like small lesions, discoloration and aberrant patterns that might not be clearly noticeable in raw images. Such a preprocessing pipeline is highly effective in enhancing image quality and making sure that even subtle disease features

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

come into the limelight, which promotes the overall detection results. The system then employs deep feature extraction on post processed images that is based on NASNetMobile, a light but strong convolutional neural network architecture that is built on top of TensorFlow. Through transfer learning, the model takes advantage of pre-trained weights acquired on large-scale data; thus, it is able to obtain the complex and high-level features, including texture, shape and changes in color of maize leaves more efficiently. This method makes it less expensive to compute and takes less time to train and is quite accurate. The features obtained are then subjected to binary and multi-class classification. In dichotomy, the model is used to differentiate healthy and diseased/pest-infected leaves. It also subdivides the input into seven distinct classes of various diseases and pest conditions in multi-class classification. To enhance the model generalization and minimize overfitting, a number of regularization methods are applied, such as Dropout (to decrease the process of co-adaptation of neurons), Batch Normalization (to stabilize and accelerate training), and Label Smoothing (to avoid the situation of overconfidence in predictions). All these methods have the effect of strengthening and improving the dependability of the model. In an attempt to improve classification performance particularly at the binary classification phase, the system incorporates a quantum feature transformation module based on variational quantum circuits on PennyLane. It is a 4-qubit angle encoding circuit, in which, quantum states are encoded with the classical features and interacted with several layers of entanglement. The hybrid quantum-classical model allows the representation of low-dimensional feature space efficiently and an improvement in the decision-making ability of the model because of the ability to represent complex feature relationships which classical models might not comply with. The general architecture is scalable, modular and capable of being adapted to the real-world deployment. It can be fitted into the IoT-based agriculture monitoring systems. Temperature, humidity, and soil conditions are sensor data which is collected in real-time and stored in a Firebase database. The data is constantly updated within a web application developed with Streamlit, which grants the farmers and the stakeholders an interactive and easy-to-use interface. In the web application, the user can post an image of the maize leaf and the model will process this image to determine whether there is any presence of diseases or pests. The system does not just give predictions but also showing the corresponding remedies and preventive controls to the detected condition. This data end-to-end pipeline, including data acquisition and preprocessing, prediction, and recommendation, will make sure that farmers get actionable insights in real time. To summarize, the suggested system is based on classical deep learning with new quantum algorithms with cloud and IoT integration to offer a holistic, efficient, and smart solution to the maize crop health monitoring. The fact that it is capable of identifying diseases at their early stages and offering real-time suggestions makes it very useful in precision agriculture and sustainable farming practices.

SYSTEM ANALYSIS

CHAPTER – 2

SYSTEM ANALYSIS

System analysis demonstrates the integration of state of the art preprocessing, deep learning and selective quantum enhancement achieves a lot in enhancing the accuracy of maize pest and disease detection. Bilateral filtering and CLAHE are used to improve the clarity of the image, whereas deep features generated by NASNetMobile with the help of the TensorFlow give a good representation of features. Variational circuits built with PennyLane can be used to perform quantum feature transformation, which can then be used to perform binary classification with a validation accuracy of approximately 99% in general. In the case of multi-class classification, full classical deep neural networks are better than quantum-reduced features. It is scalable, modular and can be integrated with the IoT and the firebase cloud. The general architecture facilitates real-time deployment by having a user-friendly interface of Streamlit. The results of the experiments prove enhanced robustness, less overfitting, and enhanced generalization to unseen samples of maize crop. This is a good classical-quantum architecture that gives a stable background to smart agriculture application and the implementation in the large field in the future.

2.1 SYSTEM STUDY: The system study examines the general architecture and functionality of the Quantum Computing, Cloud and IoT Integrated Maize Pest and Disease Detection System. It analyzes preprocessing based on Bilateral filtering and CLAHE, deep feature extraction based on NASNetMobile that is made with the help of TensorFlow, and classification based on optimized neural networks. Another analysis centered on variational circuits created through PennyLane to analyze quantum-enhanced binary classification is also assessed in the study. Findings prove that there is high accuracy, good overfitting control, and scalability. The system also assesses real-time data acquisition of IoT sensors such as temperature, humidity and soil moisture data. These environmental parameters are relayed to the Firebase cloud database to save and synchronize these parameters. The developed Streamlit web application provides a live sensor monitoring and disease prediction outcome to make effective decisions. In general, the architecture would guarantee sound cloud connectivity, modular deployment, and precision agriculture application appropriateness.

2.1.1 Feasibility Study: The Maize Disease and Pest Detection System using Quantum Computing, IoT Integration, and Firebase Cloud feasibility study examines the possibility of successfully designing, implementing the latter, and maintaining the proposed system. The proposed system is expected to identify diseases, and pests in maize crop using image processing, machine learning, quantum feature enhancement, and IoT-based environmental monitoring. The project feasibility may be discussed within the frames of the technical, economic, operational, and time feasibility.

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

2.1.1.1 Operational Feasibility: The viability of the proposed system should be considered prior to the system being implemented. Feasibility is used to determine the effect of coming up with the system in an organization whereby the effects of the system should positively surpass the negative aspects. Feasibility is considered in four aspects in this project. The economic viability is guaranteed because the system is based on open-source frameworks, including TensorFlow and PennyLane, and is affordable in terms of development and deployment due to the use of inexpensive IoT devices and Firebase Cloud services. Technical feasibility is proved by the achievement of the preprocessing (Bilateral + CLAHE), NASNetMobile feature extraction, quantum-enhanced binary classification, multi-class deep learning models, IoT-based sensor monitoring and Firebase cloud storage without any physical quantum hardware. The viability of operations is ensured because the system enables real-time detection of diseases, remote monitoring, and scalable use of the cloud. Behavioural feasibility is made to occur since the system makes it easier to identify the disease among farmers, enhance decision-making, and adopt the solution by adopting an automated and user-friendly design.

Operational Feasibility

2.1.1.2 Technical Feasibility: Technically, the system is very feasible since it uses well-established and scalable technologies. Deep learning models like Tensorflow, computer vision models like OpenCV, quantum models like PennyLane and trained models like NASNetMobile and YOLOv8 guarantee good performance in feature extraction and classification. Firebase is the cloud and provides real-time syncing, whereas the MIT App Inventor is fast and allows creating mobile applications quickly without writing complicated native code. Microcontrollers such as Raspberry Pi and Arduino are affordable and allow easy interfacing of IoT sensors. The modular structure guarantees that every component image processing, quantum classification, cloud data storage and mobile interface can be used separately and still fully integrated.

2.1.1.3 Behavioral Feasibility: The system is user and farmer-friendly in terms of behavioral feasibility. The web app has an easy user interface in which one can take a picture of a leaf or post it. The sensor information is also updated automatically and the remedies are in understandable language. This minimizes the need to rely on agricultural professionals and builds confidence of the farmers to adopt technology. This is because the system will give immediate feedback and solutions that are put into action thus promoting frequent use and facilitate sound decision making in crop management.

2.1.1.4 Financial and Economic Feasibility: Speaking of the financial and economic feasibility, the project can be considered financially viable because of the use of open-source tools and inexpensive hardware components. OpenCV, PennyLane, Firebase, and other software platforms cost less to develop, since they have free and affordable tiers. The IoT sensors and microcontrollers are cheap and readily accessible. The web application will remove the section of specialized field visit making the operations cost effective. Pest and disease identification at an early stage ensures low losses in crops, high quality in yield and higher income among the

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

farmers. The system would lead to sustainable agricultural practices in the long run, which would make it cost-effective and economically gainful to both small and large-scale farmers.

2.2 SYSTEM REQUIREMENT SPECIFICATION: The suggested system will be developed using Python 3.9 or more, TensorFlow, Keras, OpenCV, Scikit-learn, PennyLane, Ultralytics YOLOv8 framework, and NASNetMobile pre-trained weights. The model training can be done using Google Colab or on a local computer with at least 8GB RAM, whereas, a 16GB RAM and graphical acceleration (NVIDIA CUDA-compatible) are recommended to train and extract features faster. To be deployed Web application needs Visual Studio to be developed. The web will need at least 2.4GHz Speed of Network so that it can do image processing and inference with ease. Firebase Cloud services such as Firebase Authentication, Firestore or Realtime Database and Firebase storage are needed to store prediction logs, IoT data and uploaded images. Cloud synchronization must have Internet connectivity (Wi-Fi or 4G/5G). The IoT monitoring system will need a microcontroller, e.g., Raspberry Pi, Arduino UNO with Wi-Fi, and such sensors as DHT11(temperature and humidity), soil moisture sensor and NPK Sensor. The IoT device should be able to integrate with Firebase either through Firebase libraries or Firebase REST API. The power supply requirements are 5V regulated power source or battery. The system will be able to work in the agricultural outdoor settings.

2.2.1 Functional Requirements: The system will obtain images of maize leaf either by direct camera images captured on the mobile or image uploaded using the Web application. The image shall undergo the stages of Adaptive Bilateral-Clahe and contrast enhancement through filtering and CLAHE beforehand. It will then be necessary to resize and normalize the images then extract features with NASNetMobile deep feature extractor. When it comes to binary classification, the system will decrease features with the help of PCA and transforming features with a variational quantum circuit, and passing them to a neural network classifier to identify whether a leaf is healthy or diseased/pest-affected. In the case of multi-class classification, the system will be based on the full deep features on a classical deep neural network to recognize certain types of pests or diseases.

The system will provide the predictions and confidence scores and present them in web application. Prediction results shall also be stored in firebase cloud to access and monitor the results of predictions remotely. The system will be able to achieve real time inference with response time of less than a few seconds per image. The IoT sensor module will constantly check the environmental conditions in form of temperature, humidity, soil moisture, NPK and send the data to Firebase Cloud in real-time. The web application will permit firebase authentication of the user and will keep past prediction histories. The integrated system will provide secure web application to cloud database and to IoT devices communications.

2.2.2 Non-Functional Requirements: The system will be characterized by high accuracy 99% in binary and in multi-class classification and consistent validation and low overfitting. The application will offer secure transmission of data with the encryption and

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

Firestore verification to secure user and farm information. The system will be low-latency (deliver prediction results in 2-5 seconds per image). It will be scalable to support several users and farms without reducing the performance. The IoT data transmission will be resilient with a low rate of packet drop and reconnection services. The system will ensure integrity of data in Firestore Cloud including backup and recovery. The web will run well on the mid-range Android devices without excessively using memory and battery. The system will be made so that it is resistant to image noise, changing lighting, and environmental variation. The future expansion and upgrades will be supported by cloud storage and model architecture without significant redesign.

2.2.2.1 Usability: The system will offer a user-friendly and user-friendly web platform through which users will be able to upload pictures of maize leaves or post the images with fewer clicks. The results of prediction will be well presented using disease name, confidence score and shows the remedies. The data about the environmental sensors like temperature or humidity and soil moisture and NPK will be represented in easily comprehensible numerical representations. The design will accommodate the farmers who have low technical literacy by making labels clear, fonts readable and having easy navigation. The application will be able to work online.

2.2.2.2 Reliability: The system will be able to perform consistently with constant running, such as real-time IoT monitoring, and image prediction repeating. Firestore Cloud implementation will guarantee high-security and trustworthy data storage with a low downtime. The IoT module will pause connection to the Wi-Fi during loss of connection. The model inference process will be such that it gives similar inputs the same output, and the same classification is achieved. Backup systems will be used to ensure that data is not lost due to unaffiliated power failure or network disruption. The system will be subjected to different environmental conditions in order to be robust when used in agricultural fields.

2.2.2.3 Performance: The system will be able to classify the images in real time with a response time below 5 seconds on a standard Android device. The NASNet-based feature extraction and quantum-enhanced binary model will be efficient requiring no excessive CPU or memory expenses. IoT sensors will be able to send environmental data with a specific frequency (e.g., every 5 minutes) without compromising the performance of clouds. Firestore database operations will be able to support several users without any apparent lag. The system will be optimized with respect to the size of the batches, model structure, and used memory to guarantee effective execution and low energy consumption on mobile gadgets and IoT apparatus.

2.2.2.4 Supportability: The system will be designed in a modular form so that it is easily maintained, debugged and updated. Any model enhancements, retraining or parameter optimization can be made deployable without altering the overall system architecture. Remote monitoring and logging on troubleshooting shall be supported by firestore backend services.

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

Web application will be compatible with normal deployment version updating. OTA (Over-The-Air) IoT firmware updates also will be enabled when possible. The efficacious documentation and correct design of the code will make it possible to maintain and expand the system by future developers.

2.2.2.5 Flexibility: The system architecture will be such that it is capable of adding new types of crops, diseases or classes of pests without substantial re-design. The model architecture will enable the ability to switch to classical, quantum-enhanced, or hybrid architectures regarding the performance needs. The IoT module will be able to provide other sensors like pH sensors, light intensity sensors, or nutrient sensors. Firebase database structure will be scalable to changes in new data fields. The web application will enable UI improvements and new functionalities like multilingual or analytics dashboards. The system will be able to support on-device inference (TensorFlow Lite) and cloud-based inference based on the requirements of deployment.

2.3 SYSTEM REQUIREMENT SPECIFICATION: System Requirements Specification (SRC): The SRS is a formal document which is a clear definition and description of all system-level requirements of a particular system or application. It is a full-fledged reference on what the system is supposed to achieve, how it is to operate as well as the limitations within which it is supposed to operate. The SRS converts objectives that are outlined in Project Concept Proposal, Project Business Case and Project Charter into specific functional and non-functional requirements. This document recognizes, arranges and discusses system requirements in a systematic manner so that there is a common understanding of expected system behavior among the developers, stake holders and project managers. It defines what the system should do, the performance, user interaction, security control, and system constraints; hence less ambiguity in the development process. These requirements are documented formally by the SRS; this serves as a contract between stakeholders and the development team. Acceptance of this document means that the two parties will agree that the system will be developed in line with the requirements presented and that upon completion, it will be deemed complete and acceptable. Moreover, the SRS is used to provide a framework, design, implementation, testing, verification, and subsequent maintenance of the system and also traceability and quality assurance throughout the project lifecycle.

2.3.1 Hardware Requirements

1. Raspberry Pi 3 Model B V1.2 with 64GB SD Card
2. Arduino UNO
3. Temperature and Humidity Sensor – DHT11
4. Soil Moisture Detector Sensor with corrosion resistance weather proof probe
5. JXCT Soil NPK Sensor 12-24V RS485
6. RS485 to TTL Converter Module
7. Bread Board
8. 5.5mm x 2.1mm DC Power Female Jack Adapter

2.3.2 Software Requirements

1. Python 3.12.12
2. Roboflow (Dataset)
3. Fire Base Cloud Platform
4. Fire Base Database - United States (us-central1)
5. VS Code
6. Thonny (Python Integrated Development)
7. Windows 11 16GB ram with 512 SSD
8. Google Colab

2.3.3 Required Libraries

1. streamlit
2. tensorflow
3. numpy
4. opencv-python
5. joblib
6. scikit-learn
7. pennylane
8. matplotlib
9. ultralytics
10. torch
11. scikit-image
12. pyyaml
13. imbalanced-learn
14. seaborn
15. Firebase-admin
16. adafruit-circuitpython-dht

2.4 METHODOLOGIES: The suggested procedure of maize crop pest and disease detection will be in the form of a multi-stage system that is organized and effective to provide accurate prediction and classification. The first step is image pre-processing in which the raw maize leaf images undergo image improvement like noise removal, brightness, resizing, and sophisticated filters like Adaptive Bilateral-CLAHE. These processes enhance clarity of images and also maintain significant edge and texture details. Also, normalization, and data augmentation are implemented to enhance the diversity in the dataset and model robustness. Following preprocessing, the system will move on to feature extraction where significant features such as texture, color, and shapes are obtained with the use of features such as CNN-based feature mapping, HOG, LBP, and color histograms. The integration of UniTanh Rectifier further adds the feature representation as it promotes the non-linear learning. At the second phase, binary classification is to be conducted to differentiate healthy and diseased or pest-damaged crops through machine learning and deep learning models. First, CNN and YOLO are

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

used, yet in order to overcome the problem of overfitting, NASNetMobile with PCA and SVM are employed to be better generalized and more accurate. Subsequently, there is multi-classification where the exact type of disease or pest inflicting the crop is determined with MobileNet and SVM classifiers in order to have accurate classification. Moreover, the methodology synthesizes quantum computing strategies, in which reduced features are represented in quantum circuits to improve the representation of features and the classification in complex environments. The classical-quantum approach is a hybrid approach that enhances the boundaries of decisions and also increases accuracy particularly in the differentiation of closely related diseases. Lastly, the system is further expanded with the integration of IoT, where real-time environmental data, like temperature, humidity, soil moisture, and NPK levels are recorded with the use of sensors and presented with the help of a Streamlit web application. The system also enables farmers to upload the leaf pictures and obtain the forecasts and suggested solutions, therefore, it is a complete smart agriculture system. On the whole, the presented methodology is a combination of image processing, deep learning, quantum computing, and IoT to provide a reliable, precise, and efficient system of crop monitoring.

2.4.1 Components

2.4.1.1 Raspberry Pi: Raspberry Pi 3 Model B is used as an IoT edge device when real-time monitoring of maize leaf is required. It is a low priced computer with a processor speed of 1.2GHz quad core, 1GB memory, onboard Wi-Fi, Bluetooth and Ethernet connectivity. Installing the operating system and saving project scripts as well. The Raspberry Pi has a camera module attached to it to take pictures. The Raspberry Pi allows real-time data capturing, wireless communication, and monitoring over a long distance. It is an effective and viable solution to deploying projects using IoT-based smart agriculture systems due to its low power consumption, portability, and network capability.



Fig 2.1: Raspberry Pi

2.4.1.2 Arduino UNO: Arduino Uno is used as a microcontroller that will be utilized in collecting and processing sensor data in the IoT module. It connects to environmental and soil sensors to acquire environmental and soil parameter. The Arduino Uno is a digital converter, and it can be connected to the Raspberry Pi 3 Model B using the serial shipping. Arduino is cheap, simple to write codes on, and dependable. It is small and consumes low power which renders it appropriate to be used in the field on a continuous basis. In general, it contributes

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

significantly to the incorporation of the smart agriculture IoT system with the help of several sensors.



Fig 2.2: Arduino UNO

2.4.1.3 Temperature and Humidity Sensor (DHT11): DHT11 is used for measuring temperature and humidity at the same time. The DHT11 has a power supply of 3.3V to 5 V. It can connect be connected with any micro controller. It gives out calibrated digital output signals hence stable and reliable readings are guaranteed. The sensor has a temperature measuring range of 0°C to 50°C and a humidity range of 20-90% with acceptable accuracy in agricultural use. It keeps on checking the environmental conditions in the fields. This information on the environment assists in knowing the conditions that support the growth. The sensor is attached to real-time data acquisition with the help of the GPIO pins. It consumes very little power and is therefore applicable in the field.



Fig 2.3: DHT11

2.4.1.4 Soil Moisture Detector Sensor: Soil Moisture Sensor, which has a corrosion-resistant weatherproof probe to measure the water content in the soil is used for the IoT integration. It also determines the moisture level in the soil to have the right amount of irrigation and healthy growth. The probe has a durable life which ensures that it is effective in the adverse weather conditions like rain and constant soil exposure. The sensor will be attached to any micro controller which will receive and process the moisture data on the fly. This assists in the determination of the relationship between moisture of the soil. It has low power consumption, weatherproof design, and therefore it can be used in long term agricultural monitoring.



Fig 2.4: Soil Moisture Detector Sensor

2.4.1.5 NPK Sensor: JXCT Soil NPK Sensor is required to measure the necessary soil nutrients as the component of the IoT-based smart farming system. It is a sensor that measures the level of Nitrogen (N), Phosphorus (P) and Potassium (K) in the soil that is essential to the healthy growth of crop. It works in 12-24 V power range and the RS485 communication protocol to carry out the stable and long distance data transmission. The RS485 interface is also capable of guaranteeing communication in the open agricultural environment. The sensor has a probe that is waterproof and corrosion resistant and allows the sensor to be inserted into the soil continuously. It can be connected to any micro controller regarding the data acquisition by using an RS485 to USB converter. The sensor helps in precision farming because it allows making decisions based on nutrients. Its rough structure makes it to be durable in the field. Constant monitoring enables the farmers to apply corrective measures in time. On the whole, the JXCT Soil NPK Sensor is an improvement to the existing system by incorporating soil nutrient evaluation as well as smart crop health management.



Fig 2.5: NPK Sensor

2.4.1.6 Converter: The RS485 to TTL Converter Module is used to allow the communication between the JXCT Soil NPK sensor and the IoT controller. As the NPK sensor communicates using RS485, the converter will convert the RS485 communications to TTL/serial communications that are compatible with any micro controller. It assures long distance, noise free and stable data transmission. The module helps to monitor the soil nutrient and transmit the information. Its minimal power consumption and small size makes it applicable to the field.

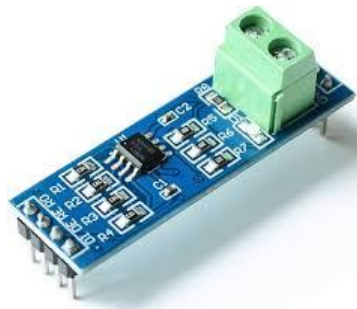


Fig 2.6: Converter

2.4.1.7 Bread Board: Breadboard is used as a prototyping board to provide a temporary platform to connect different components of the IoT without needing any soldering. It can be easily and freely connected to the Arduino Uno, Raspberry Pi and sensors, relays and other electronic components. The breadboard has rows and columns that are interconnecting and which make it easy to build and test circuits. It helps in the arrangement of wiring connections in a safe and tidy way when developing it. It is applied to interface sensors and relay system. It is easy to make changes and troubleshoot the solderless design. It helps in quick prototyping and experimenting prior to deploying the final hardware. The breadboard does not require any extra power resources. It has a reusable design hence rendering it cost-effective in terms of development. It guarantees stable electric connections when carrying out tests. The component layout is easy to modify when the need arises. Altogether, the breadboard is a significant solution to the development and testing of an IoT hardware integration with efficiency.

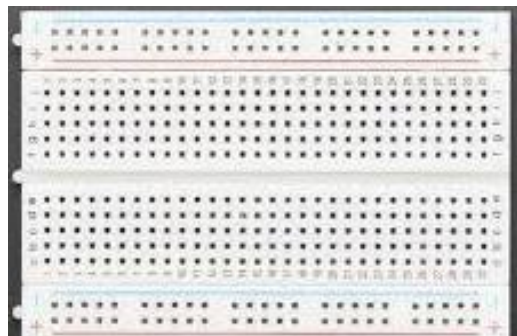


Fig 2.7: Bread Board

2.4.1.8 Adapter: The 5.5mm x 2.1mm DC Power Female Jack Adapter, is used to ensures a stable DC connection of the components used in the IoT. It is used in connecting external power supply to the devices like the Arduino Uno and relay module. The device is a safe and sound voltage transmission device. It is easily and safely connected to the wires, as it has screw terminal design. It aims in the distribution of power across the circuit in a steady manner. Its miniature and solid structure makes it applicable in the prototype development. In general, it helps to provide safe and effective power supply in IoT system.



Fig 2.8: Adapter

2.4.2 Proposed Methodology: The suggested methodology introduces a hybrid smart system that incorporates the sophisticated image processing, deep learning, quantum computing, and IoT-based environmental monitoring to detect the presence of pests and disease in maize correctly. The system is designed in such a way that it has several interconnected stages making it highly precise, robust and can be used in real-time in smart agriculture applications. The pre-processing of image in the first stage involves the improvement of the quality of maize leaf images. Noise-cutting, contrast-enhancement, scaling, normalizing and Adaptive Bilateral-CLAHE filtering are used to enhance transparency and save significant textures of the leaf. The methods used are edge detection and analysis of texture to point out the boundaries of the lesions and the patterns of the structures. This step makes sure that the dataset is improved, cleaned up and ready to extract features that are reliable. The second phase concentrates on feature extracting with deep learning structures. YOLO, CNN, NASNetMobile and MobileNet are some of these models used to extract high level semantic features, like variations on colors, texture anomalies and shape patterns. The application of NASNetMobile is mainly a deep feature extractor, to create small and significant feature vectors of maize leaves images. These characteristics form the basis of classification activities. Binary classification is also carried out in the third step to differentiate healthy and diseased/pest-infested leaves. Classical deep learning models originally exhibited overfitting hence NASNetMobile scaled, dimensionality reduced and SVM classifiers are implemented to enhance generalization. This method is able to produce steady and high validation and avoids overfitting. The fourth phase adopts multi-class classification which is used to detect particular disease or pests. To enhance the robustness and decrease the overfitting, a deep fully connected neural network with regularization systems, including dropout, batch normalization, L2 regularization, Gaussian noise, and label smoothing are deployed. This guarantees accurate diagnosis of different diseases of maize across different environmental conditions. In order to further represent features, quantum computing is selectively applied to the binary classification step. PCA is used to reduce the deep features and encodes them into a 4-qubit quantum circuit with the help of rotational gates and entanglement operations. The quantum layer resolves the classical features in a higher dimensional space that is enhanced to gain class separability. The resulting quantum features are then inputted into a neural network to achieve the ultimate prediction, which consequently has better nonlinear classification boundaries and a high classification rate. The system also includes the IoT integration with the help of the Raspberry Pi board and Arduino-based sensors, such as DHT11, soil moisture sensor, and NPK sensor. The environmental data are being sent in real-time to the

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

Firebase and presented on a Streamlit web application dashboard. Maize leaf image can also be uploaded onto the same web platform in order to predict disease and automatically push the recommended remedies into the cloud database. In general, the suggested methodology incorporates classical deep learning, quantum-enhanced feature transformation, and IoT-based monitoring into a single smart agricultural paradigm, which allows detecting diseases early, becoming environmentally conscious, and making data-driven decisions and choices on behalf of farmers.

2.4.3 Traditional Approaches: Conventional methods of detecting maize pests and diseases were primarily based on manual inspection as well as simple farming methods. Maize leaves were manually inspected by farmers or agricultural professionals to determine the presence of discolouration, lesions, spots or pest infestation. This was a subjective method that was dependent on human knowledge and experience thus susceptible to defect. This was further slowed down by lack of trained experts in most rural areas thus resulting in poor use of pesticides and loss of finances. In other instances, samples of infected leaves were tested by submitting them to the laboratories where they were either subjected to microscopic or chemical examination. Despite the fact that laboratory testing was more reliable, it was time consuming, expensive and could not be used in large scale and real time monitoring. As the computer-based systems continued to improve, traditional methods of processing images were also presented. These techniques included grayscale conversion, threshold, histogram equalization, and edge detection to emphasize patterns of patterns of disease. Color intensity, texture statistics, and shape parameters were handcrafted and extracted out of images. These attributes were subsequently inputted into classical machine learning classifiers (k-Nearest Neighbors (KNN), Support Vector Machines (SVM), Decision Trees, and Random Forest) classifiers. Although these methods were more automated than manual inspection, they had to be engineered with great care and frequently they could not deal with variations of light, background noise and complicated disease patterns. Disease detection was also separated in its environmental monitoring in the traditional systems. Temperature, humidity and soil nutrient sensors were employed separately and the data gathered was primarily by hand. No integration of cloud storage, real-time dashboards, and smart prediction systems were present. This meant that decision-making was more or less an affair of experience other than an analytical factor. On the whole, conventional techniques were not scalable, enabling, robust, and integrated that restricted their usefulness in contemporary accuracy farming systems.

SYSTEM DESIGN

CHAPTER – 3 SYSTEM DESIGN

3.1 ABOUT SYSTEM DESIGN

The offered system will be a hybrid intelligent system that will combine image-based disease detection, quantum-enhanced feature transformation, and IoT-based environmental monitoring into a single system of intelligent agriculture. The design is based on the layered approach which has data acquisition, preprocessing, feature extraction, classification, cloud integration and user interaction layers. The system renders modularity, scalability and real-time performance in such a way that the components of the system could operate independently and yet they are interlinked. The general design will enable binary classification (Healthy vs Diseased/Pest) and multi-classification (identification of specific disease / pest), as well as live environmental monitoring and recommendations of remedies.

3.1.1 Initialize design definition

- ✓ After choosing the core technologies such as Raspberry Pi and Arduino to implement the IoT sensing, TensorFlow to perform deep learning, NASNetMobile to extract features, PennyLane to simulate quantum computers, Firebase to store the data in the cloud, and Streamlit to deploy the web app, the system design starts with identifying and choosing them.
- ✓ Interfaces Hardware and software interfaces include: Planned hardware and software interfaces include serial communication between Raspberry Pi and Arduino, sensor integration viaGPIO, and cloud integration via API connection to the Firebase.
- ✓ The system regards scalability and modularity in such a way that the components that may be upgraded, like the deep learning model or the quantum module, do not require a redesign of the whole architecture.
- ✓ The possibility of technology obsolescence is considered, in particular with quantum simulation libraries, deep learning frameworks, and versions of IoT hardware. Replacement measures involve modular practice of coding and compatibility with updated API.
- ✓ The risks of cloud dependency are evaluated, the alternative mechanisms of the backend are considered to provide the opportunity of operating in the long-term.
- ✓ The design plan is recorded to cover empowering systems like the internet connection, the stability of power supply, sensor calibration equipment, and the software environment set ups needed to be deployed.

3.1.2 Establish design characteristics

- ✓ The architectural characteristics are formulated as layered architecture where there is sensing, processing, cloud and application layers with the aim of making implementation modules.

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

- ✓ Such performance characteristics as accuracy, latency, scalability and reliability are clearly defined and proven to be implementable within hardware and software limitations.
- ✓ System components interfaces are also optimized, such as image upload interface, sensor-to-cloud data transfer, quantum feature pipeline integration, API communication with Firebase.
- ✓ The roles, input-output behaviour, dependencies, and the requirements of each system element are recorded.
- ✓ Cloud authentication, database access control, and secure data transmission are security characteristics that the design has.
- ✓ The design characteristics consist of maintainability and the ability to upgrade to meet future upgrades of models, sensors or quantum modules.

3.1.3 Assess alternative for obtaining system element

- ✓ The accuracy, computational efficiency, and overfitting control are compared between various deep learning models, which are CNN, YOLO, NASNetMobile, and MobileNet.
- ✓ Classical-only classification methods are made in comparison with hybrid quantum-enhanced methods to establish enhancements in performance.
- ✓ The different classifiers such as SVM, random forest and the Deep Neural Networks are evaluated and the most stable and accurate option is chosen.
- ✓ Alternatives to hardware (like various boards of IoT or sensor modules) will be evaluated by cost, availability, and compatibility.
- ✓ The last choice is based on high validation, less overfitting, scalability, and real-time.
- ✓ When a component could be reused (e.g., pretrained NASNetMobile) it is incorporated to minimize the amount of time required to develop it, otherwise, the processes of developing a given component are used.

3.1.4 Manage the design

- ✓ The choice of NASNetMobile, quantum integration of binary classification and classical deep networks of multi-class classification is reported to ensure transparency.
- ✓ Every architectural choice, model choice and optimization plans are documented so that they can be verified and upgraded by the system in the future.
- ✓ The development of the features of design like regularization methods, quantum feature dimensions, and the depth of a neural network is actively tracked.
- ✓ Gaps and performance metrics of the model are observed to create controlled model refinement.
- ✓ Constant evaluation will keep the performance of the system, accuracy, and reliability in line with the goals of smart agriculture.

3.2 Elements of a System

3.2.1 Architecture: The architecture is characterized by the three levels of architecture: sensing layer, processing layer, and application layer. The sensing layer comprises Raspberry Pi and Arduino-based sensors which consist of DHT11, soil moisture, and NPK sensors to measure the environmental parameters. Its processing layer comprises of image preprocessing, deep feature extraction with NASNetMobile, optional quantum feature transformation with PennyLane, and classification models that are realized in terms of deep neural networks. The application layer will consist of Firebase cloud services to store data and a Streamlit web application to create visualization in real time and show prediction. This system works in the pipeline fashion wherein the leaf images are achieved via the use of enhancement techniques, features are yielded, classifications are carried out and the results are revealed alongside the dynamic pumped IoT sensor information.

3.2.2 Modules: The system has a number of functional modules. Image Preprocessing Module improves input images with the noise reduction method and contrasting technique. The Feature Extraction Module is a system based on deep learning architectures to produce meaningful features. The Binary Classification Module is used to distinguish between the healthy and the infected leaves and the Multi-Class Classification Module is used to determine the exact pest or disease type. Quantum Integration Module does a feature transformation to enhance the nonlinear separability of binary classification. The IoT Monitoring Module will gather and send the environmental data to the cloud. The Cloud Database Module hosts the sensor data, predictions and the remedy information. Lastly, Web Application Module gives a visualization, prediction, and recommendation interface to the user in a friendly way.

3.2.3 Components: Raspberry Pi Model B, Arduino UNO, DHT11 sensor, soil moisture sensor and NPK sensor are hardware components. Its software consists of Python machine learning, deep learning in TensorFlow, quantum simulation in PennyLane, cloud storage in firebase, and web deployment in Streamlit. The deep learning backbone (NASNetMobile) is the key feature extractor and neural network classifiers are used to make final predictions. The simulated qubits are used to implement quantum circuits and improve a selection of tasks.

3.2.4 Interfaces: Streamlit, which gives an interactive web dashboard, is used to develop the major user interface. The interface gives the users the opportunity to post the images of maize leaves to be predicted and see the classification results and the confidence scores. It also shows real time sensor values of the environment, including temperature, humidity, soil moisture and nutrient levels. The status of devices connectivity and timestamps of last updates are provided to provide reliability. The interface will also be user-friendly and easy to read to ensure that farmers can easily understand the outcome without the technical know-how.

3.2.5 Data: The system supports two data types, image data and sensor data. Before making predictions, image data are processed and its features are extracted and then classified. IoT

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

sensors continuously record sensor data, which is registered in Firebase Realtime Database. The result of prediction and suggested remedies are saved in Firestore so that they can be accessed in an organized manner. During the training of the models, data are split to create an unbiased evaluation. The scales, normalization, and dimensionality reduction measures are used to ensure model stability and eliminate overfitting. Generally, the system will provide effective data transfer between acquisition and storage, processing, and visualization in a single smart agriculture system.

3.3 SYSTEM ARCHITECTURE & CIRCUIT DIAGRAM

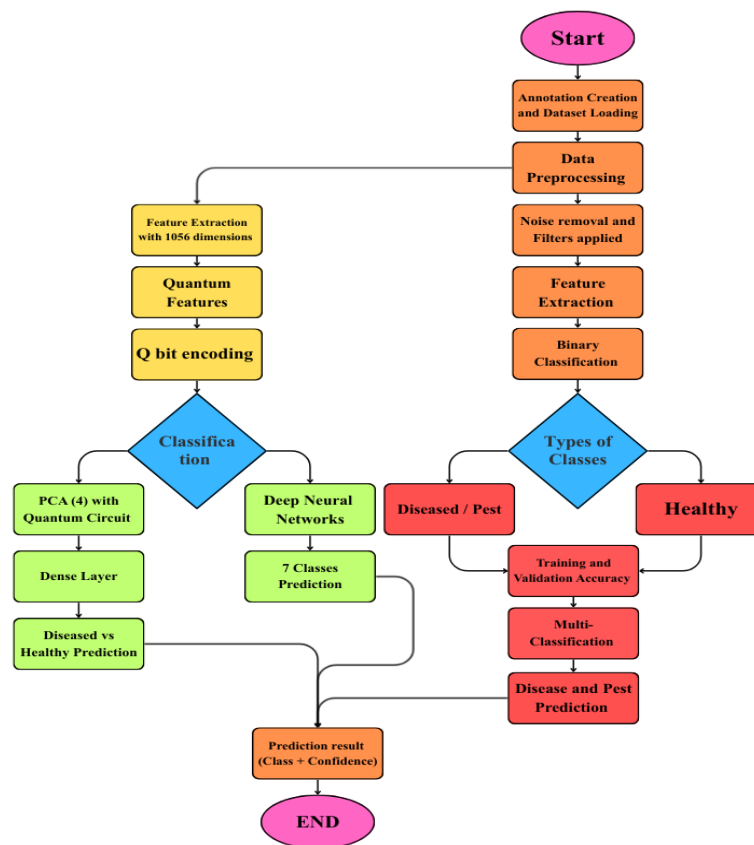


Fig 3.1: System Architecture

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

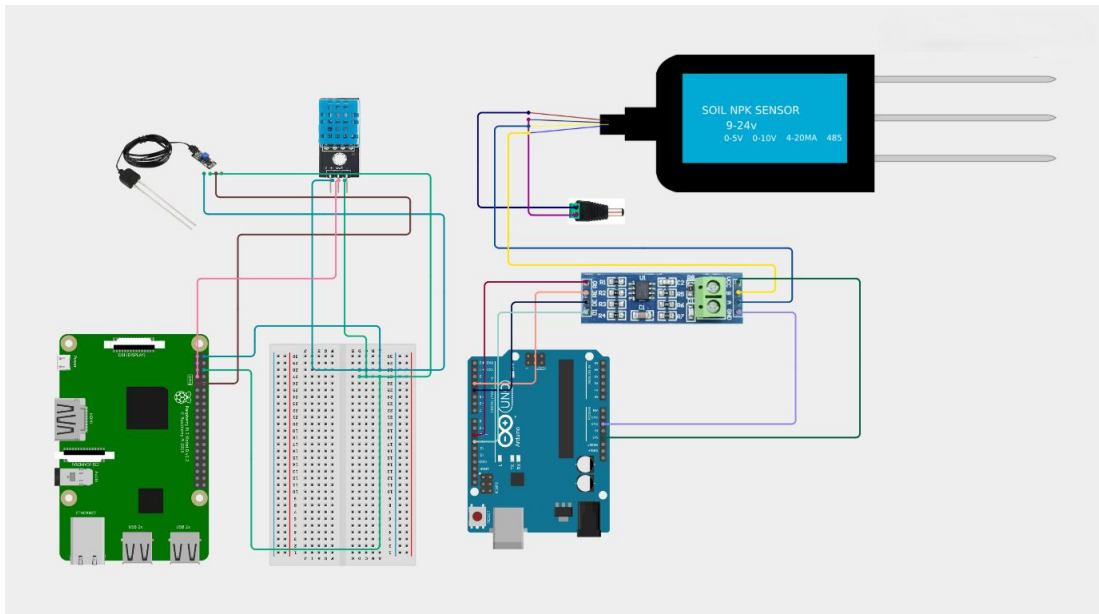


Fig 3.2: Circuit Diagram

3.4 DATA FLOW DIAGRAM: DFD stands for Data Flow Diagram. The flow of data of a system or a process is represented by DFD. It also gives insight into the inputs and outputs of each entity and the process itself. DFD does not have control flow and no loops or decision rules are present. Specific operations depending on the type of data can be explained by a flowchart. Data Flow Diagram can be represented in several ways. The DFD belongs to structured-analysis modelling tools. Data Flow diagrams are very popular because they help us to visualize the major steps and data involved in software-system processes.

3.4.1 Components Of DFDs:

The data flow diagram has four components. They are:

- ✓ External Entity
- ✓ Process
- ✓ Data Flow
- ✓ Warehouse

3.4.2 External Entity: An outside process or system that sends or receives data to and from the diagrammed system. They are also known as sources, terminators, sinks or actors and are represented by squares.



Figure 3.3: External Entity Representation

3.4.3 Process: Input to output transformation in a system takes place because of process function. The symbols of a process are rectangular with rounded corners, oval, rectangle or a circle. The process is named a short sentence, in one word or a phrase to express its essence. The process is represented using the following notation:

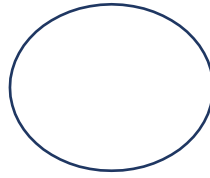


Figure 3.4: Process representation

3.4.4 Data Flow: Data flow describes the information transferring between different parts of the systems. The arrow symbol is the symbol of data flow. A relatable name should be given to the flow to determine the information which is being moved. Data flow also represents material along with information that is being moved. Material shifts are modeled in systems that are not merely informative. A given flow should only transfer a single type of information. The direction of flow is represented by the arrow which can also be bi-directional.



Figure 3.5: Data Flow Representation

3.4.5 Warehouse: The data is stored in the warehouse for later use. Two horizontal lines represent the symbol of the store. The warehouse is simply not restricted to being a data file rather it can be anything like a folder with documents, an optical disc, a filing cabinet. The data warehouse can be viewed independent of its implementation. When the data flow from the warehouse it is considered as data reading and when data flows to the warehouse it is called data entry or data updation.



Figure 3.6: Warehouse/ Datastore Representation

3.4.6 Levels in DFDs:

In Software engineering DFD can be drawn to represent the system of different levels of abstraction. Higher-level DFDs are partitioned into low levels-hacking more information and functional elements. Levels in DFD are numbered 0, 1, 2 or beyond. Here, we will see mainly 3 levels in the data flow diagram, which are: 0-level DFD, 1-level DFD, and 2-level DFD.

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

3.4.7 Level-0: The Level-0 Data Flow Diagram is a general architecture of the maize pest and disease detecting system. In such a system, the user or farmer is the main external party that communicates with the system using the Streamlit web application. The user inserts images of maize leaves in the system to detect disease or pest. Meanwhile, IoT sensors (DHT11, soil moisture, and NPK sensors) gather data about the environment in the agricultural field. Arduino and Raspberry Pi devices are used to pass on these sensor values into Firebase cloud database. The central processing system does the preprocessing, feature extraction, and classification of images uploaded using deep learning models. Once it is processed, the system will be able to predict whether the crop is healthy, diseased, and/or infected with pests. The outcomes of the prediction with the confidence values are then returned to the user as per the web dashboard. The system also extracts remedy and prevention information available in the database and presents the same to the farmers. This general framework is that it allows real-time surveillance and smart detection of crop diseases. Therefore, the Level-0 DFD reveals the interrelation between the users, IoT devices, the detection system and the cloud storage.

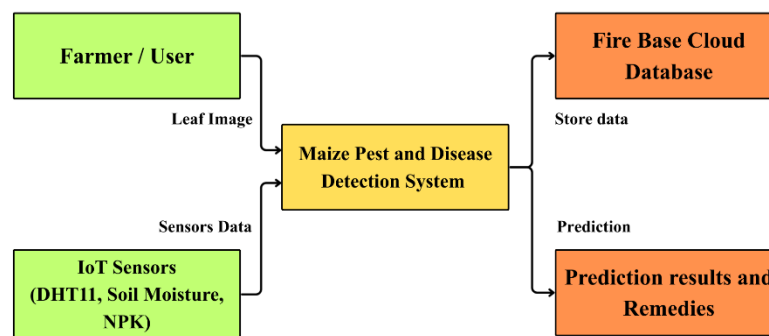


Fig 3.7: Level 0 DFD

3.4.8 Level-1: The Level-1 Data Flow Diagram gives an in-depth look at the internal processes, in the maize disease and pest detection system. First, the maize leaves are uploaded by the user by means of the Streamlit web interface. The uploaded image is forwarded to image preprocessing module which involves image processing like resizing, normalization, bilateral filtering, and CLAHE enhancements. The preprocessed image is then sent to the feature extraction module which uses deep learning algorithms like NASNetMobile and YOLO to extract relevant features based on texture, color and shape. These features are then sent on to the binary classification department where the model identifies the leaf as healthy or infected. In case of the identification of the infected leaf, the system goes to the multi-class classification stage which identifies the particular disease or the pest attacking the maize crop. In the meantime, the data on the environment monitored by IoT sensors via Raspberry Pis and Arduinos are also incorporated into the system. The output of detection of disease and environmental monitoring is stored on Firebase database. Lastly, the system accesses suggested remedies and preventive actions and presents them to the farmer on the web dashboard.

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

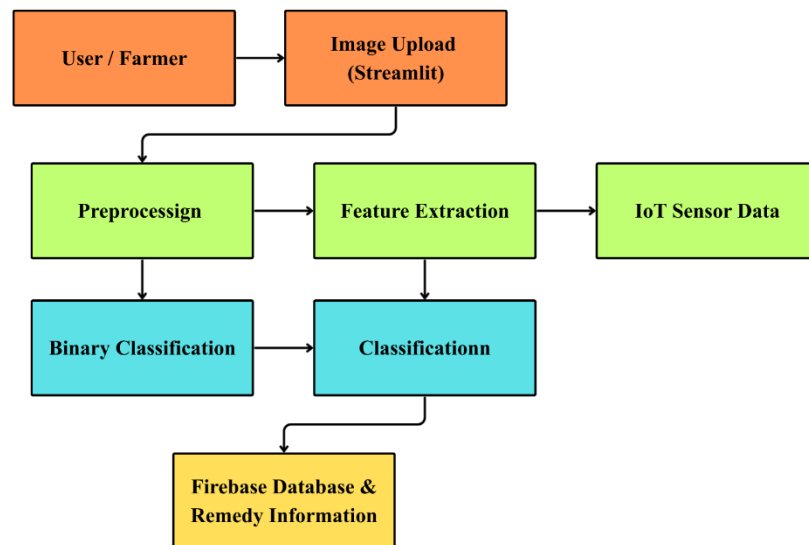


Fig 3.8: Level 1 DFD

3.4.2 UML Diagrams

3.4.2.1 Use Case Diagram

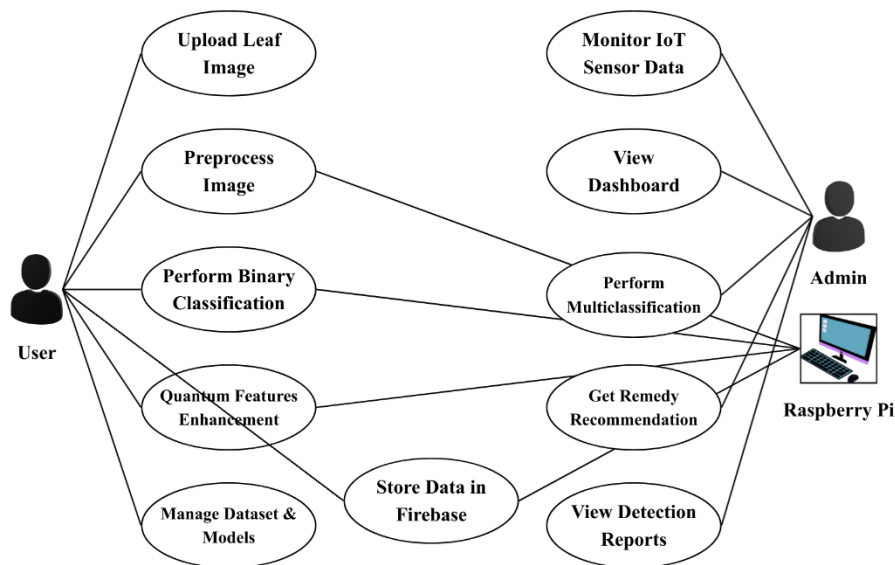


Fig 3.9: Use Case Diagram

The Use Case Diagram presents the interrelations among the key players, i.e. the User, Admin and the Raspberry Pi system, in the proposed smart crop disease and pest detection system. The User initiates the process by uploading a leaf image, and the system carries out image preprocessing that is done to improve the image quality and eliminate noise. Binary classification is then used to analyze the processed image to identify the crop as healthy or diseased. In order to increase the detection accuracy, quantum features enhancement methods are used and then the multiclass classification is implemented to determine the type of pest or

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

disease. Users can also manage datasets and models they use in training and updating the detection algorithms using the system. In the meantime, the Admin will be able to access the data of the IoT sensors, see the dashboard of the system, and analyze the environmental conditions that he/she is collecting with the help of the Raspberry Pi. Firebase is used to store all the data and results safely, allowing to manage data and access it in real-time. According to the results of the detection, the system offers remedy recommendations to assist farmers to take the relevant action as well as enables the users and administrators to access detection reports to perform additional analysis and decision making. On balance, this use case diagram shows that image processing, machine learning, quantum computing, and IoT technologies can be used to develop an intelligent and automated crop monitoring system.

3.4.2.2 Class Diagram

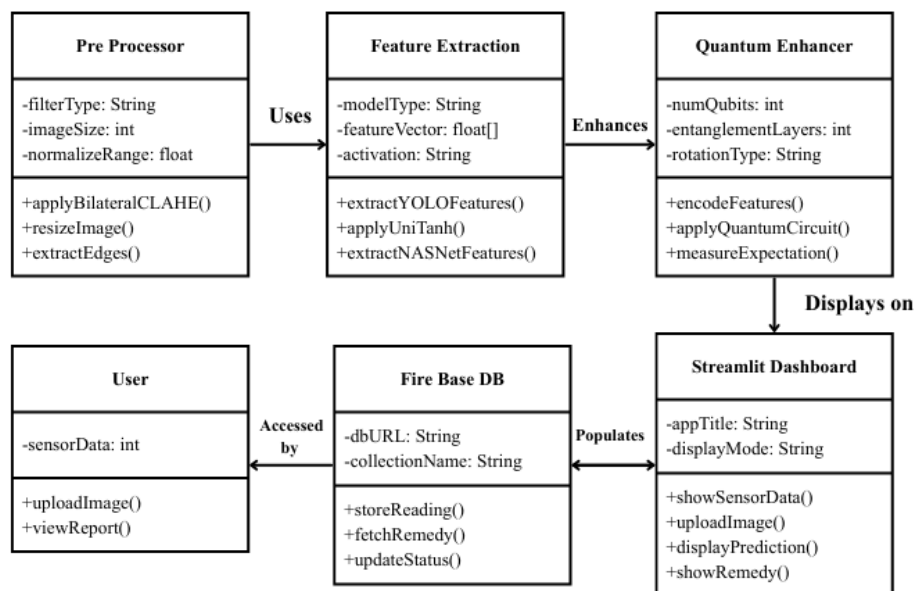


Fig 3.10: Class Diagram

Class Diagram is a structural representation of the proposed smart crop disease and pest detection system and it indicates the key classes, their attributes, processes and relatedness. The Pre Processor class manages the quality of the input leaf images by applying the functions like apply Bilateral CLAHE(), resize Image and extract Edges among others with variables like filter type, image size and normalization range. The processed image is subsequently given to the Feature Extraction class; here meaningful features are recovered out of the image using deep learning models by first using extract YOLO Features(), apply UniTanh and extractNASNetFeatures. The extracted features are then augmented with the Quantum Enhancer class which uses quantum computing methods such as encode Features, apply Quantum Circuit and measure Expectation with parameters such as quantum qubits, as well as layer of

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

entanglement to enhance classification performance. The output is presented in the form of the Streamlit Dashboard where an interactive user interface is provided, allowing one to upload images, view predictions, visualize sensor data, and provide recommendations on the remedies. The User uses the system to add pictures and read reports and the Firebase Database to store sensor readings, prediction, and remedy data through such functions as store Reading (), fetch Remedy (), and update Status (). This information is obtained in the database and displayed on the dashboard in real-time to guarantee that it is monitored and analyzed. In general, the class diagram shows the collaboration of various system elements to process the images, enrich the features with quantum methods, store the data, and display the results to the end user with the help of a convenient interface.

3.4.2.3 Activity Diagram

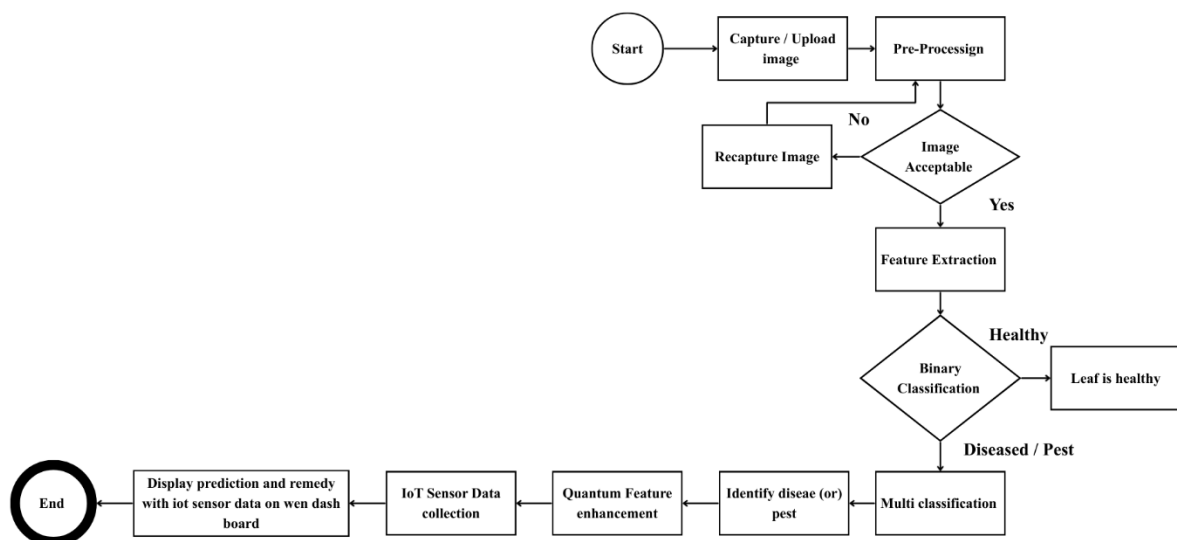


Fig 3.11: Activity Diagram

The Activity Diagram presents the workflow of the proposed smart crop disease and pest detection system starting with the image capture and ending with the ultimate display of the results. This starts with the user having a leaf image to capture or upload, the images are of course pre-processed to enhance the quality of the image by removing noise and resizing the image respectively. The system verifies that the image is fit to analyze the picture, in case the quality of the picture is low, then the system asks the user to take the picture again, and the system proceeds. The process of validation is followed by feature extraction that obtains the important characteristics of the leaf image. Binary classification is then used to classify the extracted features as healthy or diseased leaves. The system will display the result immediately, in case the leaf is healthy. When the system identifies a disease or a pest, it further performs multiclass classification to determine the type of disease or pest that is causing the crop disease

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

or pest. After this, quantum feature enhancement procedure is implemented to advance the quality of detection, and the system detects the particular disease or pest. At the same time, the data of the IoT sensor corresponding to the environmental parameters are measured like temperature and humidity. Lastly, the system depicts the prediction results and suggested remedies and sensor data on a dashboard, which enables the users to view the condition of the crops and make relevant responses easily. This is done until the results and recommendations are delivered to the user.

3.4.2.4 Sequence Diagram

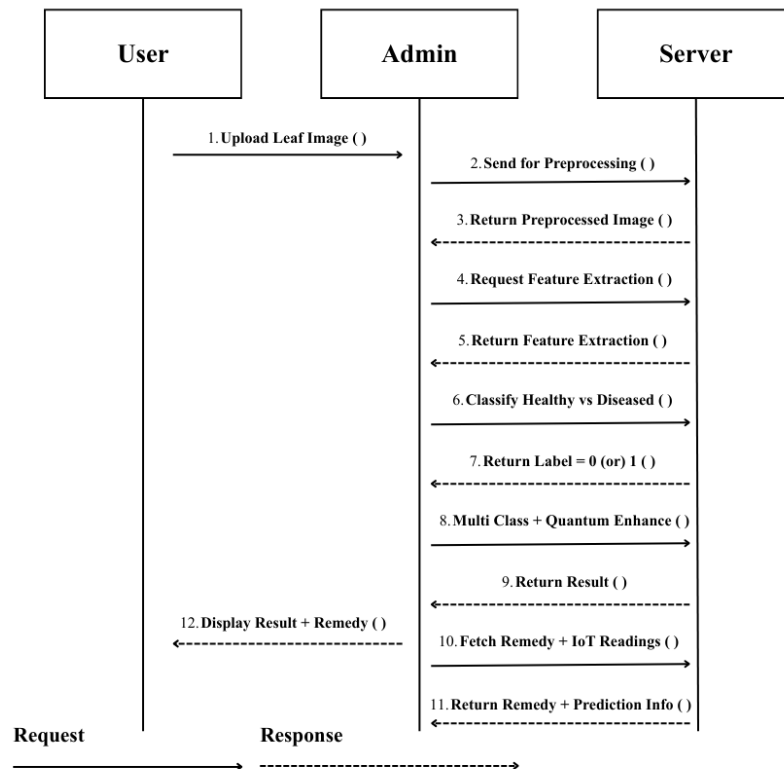


Fig 3.12: Sequence Diagram

The Sequence Diagram shows how the interaction between the User, the Admin, and the Server proceed in the smart crop disease and pest detector system in steps. This process starts when the user posts the image of a leaf which is sent to the server through the admin interface which then sends to the server where it undergoes preprocessing. The server is used to package the image to enhance its quality after which the preprocessed image is returned. The next step involves the request of the server to conduct feature extraction, during which important properties of the image of the leaf are determined with the help of deep learning methods. Once the features have been extracted, the server uses binary classification to see whether the leaf is healthy or diseased and sends the label of classification (0 that the leaf is healthy and 1 that the leaf is diseased). In case the leaf is known to be diseased, multiclass classification and quantum feature enhancement are conducted by the system to determine the type of disease or pest. The

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

server will then send out the detailed outcome to the admin module. After this, the system retrieves recommendations of remedies and sensor readings of the environmental conditions of the IoTs in the database or the devices connected to the system. The remedy recommendations and the prediction data are returned by the server and the last step is the display of the result and remedies advised to the user through the admin interface. Based on this sequence diagram, clear communication between the system components can be established to process the image, introduce crop health, and give the user some valuable advice.

3.4.2.5 Deployment Diagram

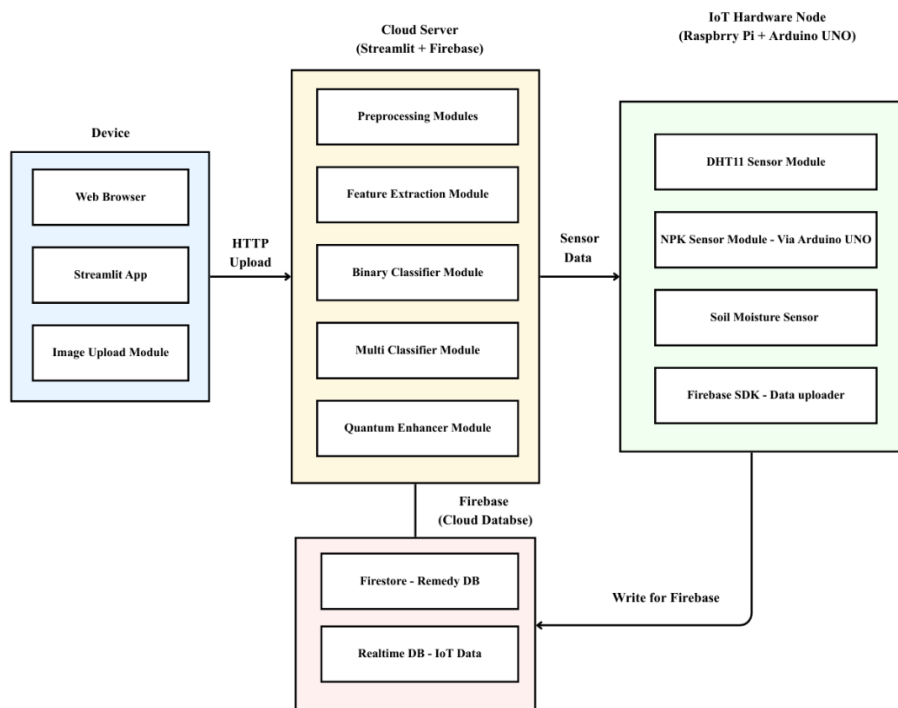


Fig 3.13: Deployment Diagram

The Deployment Diagram is used to show physical architecture and implementation of the components in the proposed smart crop disease and pest detection system. The system will have three key nodes: the Cloud Server, the IoT Hardware Node and the User Device. The User Device consists of a web browser, Streamlit application and image upload module, whereby the user uploads leaf images of crops using an HTTP connection. The images get uploaded to Cloud Server which contains the core processing units such as preprocessing unit, feature extractor unit, binary classifier unit, multi-class classifier unit, and quantum enhancer unit. The uploaded leaf images are analyzed using these modules to determine whether the crop is healthy or a disease or a pest has attacked it. The cloud server is linked to the Firebase Cloud Database that has Firestore of remedy information and Realtime Database of IoT sensor data. The other side, which is the IoT Hardware Node, is a collection of Raspberry Pi and Arduino UNO, which gathers the environmental data through the sensors (DHT11 sensor for temperature and humidity, NPK sensor of soil nutrients and soil moisture sensor). Data uploader The Firebase

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

SDK uploads sensor data to Firebase through the Firebase SDK data uploader. The cloud server retrieves the prediction results as well as IoT data on Firebase and displays them to the user on the Streamlit dashboard. In this way, deployment diagram depicts the way the hardware devices, cloud services, databases, and user interfaces are combined to form a fully functional smart agriculture monitoring system.

3.5 DATASETS

3.5.1 Maize Dataset: Mensah Kwabena et al. has cited this dataset name called “Crop Pest and Disease Detection”. The main reason to choose this dataset is that the images are clear for prediction of disease or pest which helps the model to get the accurate result. In the dataset there are 22 different classes with 24,881 images of pests and diseases of different crops such as Cashew, Cassava, Maize and Tomato. Among all the classes the model only focuses on Maize crop which contains 7 types of classes that includes both pests and diseases with one healthy class such as Maize Fall Armyworm, Maize Grasshopper, Maize leaf Beetle, Maize leaf Blight, Maize leaf Spot, Maize Streak virus and Maize Healthy. The total dataset size of Maize crop is classified as Maize Fall Armyworm consists of 285 images, Maize Grasshopper consists of 673 images, Maize leaf Beetle consists of 948 images, Maize leaf Blight consists of 1006 images, Maize leaf Spot consists of 1259 images, Maize Streak virus consists of 979 images, Maize Healthy consists of 208 images. The data is collected from the local farms in Ghana. These images are collected using high resolution camera with .jpg format the images are further formatted into various dimensions such as (400x400), (487x1080), (1080x518), (3024x4032), (4032x3024). The “University of Energy and Natural Resources” Institution has provided the necessary support for the dataset and this is helpful for the ML and DL algorithms. Funders for dataset was African Technology Society Policy Network with the Grant ID: AI4AFS/GR/AFS-1233809296. The dataset is licensed under a Creative Commons Attribution 4.0 International licence.

3.5.2 Fertilizer Dataset: Fertilizer Recommendation Dataset is agricultural data which is used to forecast the most appropriate fertilizer depending on the soil, environmental conditions, and crop type. Some of the parameters in the dataset are Temperature, Humidity, Moisture, Soil Type, Crop Type, Nitrogen, Potassium, and Phosphorous levels in the soil. These characteristics are the actual farming conditions that cause the growth of plants and nutrient demands. The dataset has different soil types such as Black, Clayey, Loamy, Red, and Sandy soils, which influence the nutrient retention and crop productivity. The crops that are included in this dataset are Potato, Pepper, Tomato and Egg Plant, which are popular crops that are grown. Depending on the quantity of the nutrient available in the soil, the data will be used to determine the best types of fertilizers like Urea, DAP, 14-35-14, 17-17-17, 28- 28, 20-20. Every record is a certain association of environmental factors and soil nutrients. Machine Learning models of fertilizer recommendation systems can be developed with the help of the dataset. These models assist farmers in using the right amount of the appropriate fertilizer thus enhancing crop production and saving on unnecessary use of fertilizers. On the whole, this data can be used to develop intelligent agriculture solutions and precision farming.

SYSTEM IMPLEMENTATION

CHAPTER – 4

SYSTEM IMPLEMENTATION

4.1 SYSTEM SETUP: The given system is developed on the basis of a hybrid architecture that combines deep learning, quantum computing, IoT devices, and cloud technologies to detect maize pest and diseases. Processing of the image and model development are done in Python in a Google Colab environment and preprocessing, feature extraction, and classification are performed using libraries of TensorFlow/Keras, PyTorch, OpenCV, NumPy, Scikit-learn, and Ultralytics YOLOv8. Deep feature extraction and classification are done using the NASNetMobile and MobileNet models, whereas dimensionality reduction and maximization of classification are done using the PCA, SVM and the Random Forest. To achieve quantum enhancement, the PennyLane quantum machine learning framework is devised to create a 4-qubit quantum circuit with rotation and entanglement layers to convert classical features into quantum-enhanced representations. The IoT hardware includes a Raspberry Pi 3 Model B and Arduino UNO board that is connected to sensors: DHT11 (temperature and humidity), soil moisture sensor, and NPK soil nutrient sensor in which Arduino gathers sensor data and sends it to the Raspberry Pi. Data about environmental measurements are transmitted to Firebase Realtime Database by the Raspberry Pi, and in Firebase Firestore, data about remedies and prediction records are stored. Streamlit web application is the user interface where the user can upload maize leaf pictures, display disease prediction outcomes with confidence scores, chart real time sensor readings and get treatment suggestions stored in cloud database. This combined infrastructure provides smart agriculture applications with real-time tracking of crops, automatic identification of diseases and data management in the cloud.

4.2 SOURCE CODE

4.2.1 Source code for Training

```
from google.colab import drive
drive.mount('/content/drive')
import os
import cv2
import matplotlib.pyplot as plt
BASE_PATH = "/content/drive/MyDrive/Pest And Disease.v1i.yolov8"
TRAIN_IMG = os.path.join(BASE_PATH, "train/images")
TRAIN_LBL = os.path.join(BASE_PATH, "train/labels")
VAL_IMG = os.path.join(BASE_PATH, "valid/images")
VAL_LBL = os.path.join(BASE_PATH, "valid/labels")
TEST_IMG = os.path.join(BASE_PATH, "test/images")
TEST_LBL = os.path.join(BASE_PATH, "test/labels")
DATA_YAML = os.path.join(BASE_PATH, "data.yaml")
import cv2
```

```
def advanced_preprocess(image):
    # Edge-preserving noise removal
    bilateral = cv2.bilateralFilter(
        image,
        d=9,
        sigmaColor=75,
        sigmaSpace=75
    )
    # Contrast enhancement using CLAHE
    lab = cv2.cvtColor(bilateral, cv2.COLOR_RGB2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    l = clahe.apply(l)
    enhanced = cv2.merge((l, a, b))
    enhanced = cv2.cvtColor(enhanced, cv2.COLOR_LAB2RGB)
    return enhanced

import matplotlib.pyplot as plt
import os
img_path = os.path.join(TRAIN_IMG, os.listdir(TRAIN_IMG)[0])
image = cv2.imread(img_path)
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
better_img = advanced_preprocess(image)
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(better_img)
plt.title("Bilateral + CLAHE Output")
plt.axis("off")
plt.tight_layout()
plt.show()
```

4.2.2 Source Code for Image Processing

```
# Resize
prep_img = cv2.resize(better_img, (256, 256))
gray = cv2.cvtColor(prepare_img, cv2.COLOR_RGB2GRAY)
# Normalize
norm_img = gray / 255.0
print(prepare_img)
print(gray)
```

```
print(norm_img)
```

4.2.3 Source Code for Sobel processing

```
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobel_mag = cv2.magnitude(sobel_x, sobel_y)
print(sobel_x)
print(sobel_y)
print(sobel_mag)
```

4.2.4 Source Code for Canny Edge Detector

```
edges = cv2.Canny(gray, threshold1=100, threshold2=200)
print(edges)
```

4.2.5 Source Code for HOG Texture + Shape

```
from skimage.feature import hog
hog_features, hog_image = hog(
    gray,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    visualize=True,
    block_norm='L2-Hys'
)
print(hog_features)
print(hog_image)
from skimage.feature import local_binary_pattern
lbp = local_binary_pattern(gray, P=8, R=1, method='uniform')
print(lbp)
```

4.2.6 Source Code for Shape Feature Extraction

```
contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
areas = [cv2.contourArea(cnt) for cnt in contours]
perimeters = [cv2.arcLength(cnt, True) for cnt in contours]
print(contours)
print(areas)
print(perimeters)
```

4.2.7 RGB Mean & Variance

```
mean_rgb = prep_img.mean(axis=(0,1))
std_rgb = prep_img.std(axis=(0,1))
print(mean_rgb)
print(std_rgb)
```

4.2.8 Source Code for HSV Histogram

```
hsv = cv2.cvtColor(prepare_img, cv2.COLOR_RGB2HSV)
hist_h = cv2.calcHist([hsv], [0], None, [180], [0,180])
print(hsv)
print(hist_h)
plt.figure(figsize=(12,6))
plt.subplot(2,3,1); plt.imshow(gray, cmap='gray'); plt.title("Grayscale")
plt.subplot(2,3,2); plt.imshow(edges, cmap='gray'); plt.title("Canny Edges")
plt.subplot(2,3,3); plt.imshow(hog_image, cmap='gray'); plt.title("HOG")
plt.subplot(2,3,4); plt.imshow(lbp, cmap='gray'); plt.title("LBP")
plt.subplot(2,3,5); plt.imshow(sobel_mag, cmap='gray'); plt.title("Sobel Magnitude")
plt.tight_layout()
plt.show()
```

4.2.9 Source Code for CNN Algorithm

```
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Input, Dropout
import matplotlib.pyplot as plt
import numpy as np
img = cv2.resize(image, (224, 224))
img = img / 255.0
img = np.expand_dims(img, axis=0)
inputs = Input(shape=(224, 224, 3))
x = Conv2D(32, (3,3), activation='relu')(inputs)
x = MaxPooling2D(2,2)(x)
x = Conv2D(64, (3,3), activation='relu')(x)
x = MaxPooling2D(2,2)(x)
x = Conv2D(128, (3,3), activation='relu')(x)
# Feature extractor model
feature_model = Model(inputs, x)
feature_maps = feature_model.predict(img)
print(feature_maps)
plt.figure(figsize=(10,6))
for i in range(6):
    plt.subplot(2,3,i+1)
```

```
plt.imshow(feature_maps[0,:,:i], cmap='viridis')
plt.axis('off')
plt.suptitle("CNN High-Level Feature Maps")
plt.show()
```

4.2.10 Source Code for CNN Function With UniTanh Rectifier Function

```
import tensorflow as tf
def my_custom_relu(x):
    """
    Custom activation:
    tanh(x) followed by ReLU
    """
    value_update = tf.math.tanh(x)
    return tf.maximum(0.0, value_update)
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Input, BatchNormalization,
Dropout
# Increased input size
inputs = Input(shape=(256, 256, 3))
x = Conv2D(32, (3,3), padding='same', activation=my_custom_relu)(inputs)
x = BatchNormalization()(x)
x = MaxPooling2D(2,2)(x)
x = Conv2D(64, (3,3), padding='same', activation=my_custom_relu)(x)
x = BatchNormalization()(x)
x = MaxPooling2D(2,2)(x)
x = Conv2D(128, (3,3), padding='same', activation=my_custom_relu)(x)
x = BatchNormalization()(x)
x = MaxPooling2D(2,2)(x)
# Increased depth (extra layers)
x = Conv2D(256, (3,3), padding='same', activation=my_custom_relu)(x)
x = BatchNormalization()(x)
x = Conv2D(512, (3,3), padding='same', activation=my_custom_relu)(x)
# Feature extractor model
feature_model = Model(inputs, x)
feature_model.summary()
img = cv2.resize(image, (256, 256))
img = img / 255.0
img = np.expand_dims(img, axis=0)
feature_maps = feature_model.predict(img)
plt.figure(figsize=(10,6))
for i in range(6):
    plt.subplot(2,3,i+1)
```

```
plt.imshow(feature_maps[0,:,:i], cmap='viridis')
plt.axis('off')
plt.suptitle("Custom CNN High-Level Feature Maps")
plt.show()
```

4.2.11 Source Code for YOLO Algorithm

```
!pip install ultralytics
import torch
import cv2
import numpy as np
import matplotlib.pyplot as plt
from ultralytics import YOLO
yolo = YOLO("yolov8n.pt")
model = yolo.model
model.eval()
img = cv2.resize(image, (640, 640))
img = img / 255.0
img = torch.tensor(img, dtype=torch.float32)
img = img.permute(2, 0, 1).unsqueeze(0) # BCHW
feature_maps = []
def hook_fn(module, input, output):
    feature_maps.append(output)
hook = model.model[5].register_forward_hook(hook_fn)
with torch.no_grad():
    _ = model(img)
hook.remove()
features = feature_maps[0].squeeze(0).cpu().numpy()
plt.figure(figsize=(10, 6))
for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.imshow(features[i], cmap="viridis")
    plt.axis("off")
plt.suptitle("YOLOv8 High-Level Feature Maps")
plt.show()
```

4.2.12 Source Code for YOLO With UniTanh Rectifier Function

```
def my_custom_activation_yolo(x):
    """
    Custom activation:
    tanh followed by ReLU
    """
    x = np.tanh(x)
```

```
    return np.maximum(0, x)
# Apply custom activation to YOLO features
features_activated = my_custom_activation_yolo(features)
plt.figure(figsize=(10, 6))
for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.imshow(features_activated[i], cmap="viridis")
    plt.axis("off")
plt.suptitle("YOLOv8 Feature Maps with Custom Activation")
plt.show()
```

4.2.13 Source Code for Binary Prediction, Plottign, Accuray with CNN

```
import os
import cv2
import numpy as np
IMG_SIZE = 256
def load_cnn_data(img_dir, lbl_dir):
    X, y = [], []
    for img_name in os.listdir(img_dir):
        img_path = os.path.join(img_dir, img_name)
        lbl_path = os.path.join(lbl_dir, img_name.replace(".jpg", ".txt"))
        img = cv2.imread(img_path)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
        img = img / 255.0
        # if any object exists → diseased (1)
        label = 1 if os.path.exists(lbl_path) and os.path.getsize(lbl_path) > 0 else 0
        X.append(img)
        y.append(label)
    return np.array(X), np.array(y)
X_train, y_train = load_cnn_data(TRAIN_IMG, TRAIN_LBL)
X_val, y_val = load_cnn_data(VAL_IMG, VAL_LBL)
print(X_train.shape, y_train.shape)
import tensorflow as tf
from tensorflow.keras.layers import *
from tensorflow.keras.models import Model
def my_custom_relu(x):
    return tf.maximum(0.0, tf.math.tanh(x))
inputs = Input(shape=(256,256,3))
x = Conv2D(32, 3, padding='same', activation=my_custom_relu)(inputs)
x = MaxPooling2D()(x)
x = Conv2D(64, 3, padding='same', activation=my_custom_relu)(x)
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
x = MaxPooling2D()(x)
x = Conv2D(128, 3, padding='same', activation=my_custom_relu)(x)
x = MaxPooling2D()(x)
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.4)(x)
output = Dense(1, activation='sigmoid')(x)
cnn_model = Model(inputs, output)
cnn_model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
cnn_model.summary()
history = cnn_model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=10,
    batch_size=16
)
```

4.2.14 Source Code for CONFUSION MATRIX (CNN BINARY)

```
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
# Predict full validation set
y_pred_prob = cnn_model.predict(X_val)
y_pred = (y_pred_prob > 0.5).astype(int)
cm = confusion_matrix(y_val, y_pred)
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Healthy", "Diseased"],
            yticklabels=["Healthy", "Diseased"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("CNN Binary Confusion Matrix")
plt.show()
print(classification_report(y_val, y_pred))
pred = cnn_model.predict(X_val[:1])[0][0]
print("CNN Prediction Probability:", pred)
print("CNN Class:", "DISEASED/PEST" if pred > 0.5 else "HEALTHY")
plt.imshow(X_val[0])
plt.title(f"CNN Output: {'DISEASED/PEST' if pred>0.5 else 'HEALTHY'}")
```



```
plt.axis("off")
plt.show()
import matplotlib.pyplot as plt
# CNN Accuracy
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('CNN Accuracy vs Epochs')
plt.legend()
plt.grid(True)
# CNN Loss
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('CNN Loss vs Epochs')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
cnn_train_acc = history.history['accuracy'][-1]
cnn_val_acc = history.history['val_accuracy'][-1]
print(f' Final CNN Training Accuracy: {cnn_train_acc*100:.2f}%')
print(f' Final CNN Validation Accuracy: {cnn_val_acc*100:.2f}%')
```

4.2.15 Source Code for Binary Prediction, Plottign, Accuray with YOLO

```
import torch
from ultralytics import YOLO
yolo = YOLO("yolov8n.pt")
yolo_model = yolo.model
yolo_model.eval()
def extract_yolo_features(image):
    img = cv2.resize(image, (640,640))
    img = img / 255.0
    img = torch.tensor(img, dtype=torch.float32).permute(2,0,1).unsqueeze(0)
    feats = []
    def hook_fn(m, i, o): feats.append(o)
    hook = yolo_model.model[5].register_forward_hook(hook_fn)
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
with torch.no_grad():
    _ = yolo_model(img)
hook.remove()
fmap = feats[0].squeeze(0)
pooled = torch.mean(fmap, dim=(1,2)) # Global Avg Pool
return pooled.cpu().numpy()
def load_yolo_cls_data(img_dir, lbl_dir):
    X, y = [], []
    for img_name in os.listdir(img_dir):
        img_path = os.path.join(img_dir, img_name)
        lbl_path = os.path.join(
            lbl_dir,
            img_name.rsplit(".", 1)[0] + ".txt"
        )
        # Load image
        img = cv2.imread(img_path)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        # Extract YOLO features
        features = extract_yolo_features(img)
        # SAFE label logic
        if os.path.exists(lbl_path) and os.path.getsize(lbl_path) > 0:
            label = 1 # Diseased
        else:
            label = 0 # Healthy
        X.append(features)
        y.append(label)
    return np.array(X), np.array(y)
Xy_train, yy_train = load_yolo_cls_data(TRAIN_IMG, TRAIN_LBL)
Xy_val, yy_val = load_yolo_cls_data(VAL_IMG, VAL_LBL)
print(Xy_train.shape, yy_train.shape)
print(Xy_val.shape, yy_val.shape)
from tensorflow.keras.models import Sequential
yolo_cls = Sequential([
    Dense(256, activation='relu', input_shape=(Xy_train.shape[1],)),
    Dropout(0.4),
    Dense(128, activation='relu'),
    Dense(1, activation='sigmoid')
])
yolo_cls.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
)
yolo_cls.summary()
yolo_history = yolo_cls.fit(
    Xy_train, yy_train,
    validation_data=(Xy_val, yy_val),
    epochs=10,
    batch_size=16
)
pred = yolo_cls.predict(Xy_val[:1])[0][0]
print("YOLO Prediction Probability:", pred)
print("YOLO Class:", "DISEASED/PEST" if pred > 0.5 else "HEALTHY")
# Prediction
pred = yolo_cls.predict(Xy_val[:1])[0][0]
print("YOLO Prediction Probability:", pred)
print("YOLO Class:", "DISEASED/PEST" if pred > 0.5 else "HEALTHY")
# Load original image again for display
img_path = os.path.join(VAL_IMG, os.listdir(VAL_IMG)[0])
img = cv2.imread(img_path)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img)
plt.title(f"YOLO Output: {'DISEASED/PEST' if pred>0.5 else 'HEALTHY'}")
plt.axis("off")
plt.show()
import matplotlib.pyplot as plt
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(yolo_history.history['accuracy'], label='Train Accuracy')
plt.plot(yolo_history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('YOLO Feature Classifier Accuracy vs Epochs')
plt.legend()
plt.grid(True)
plt.subplot(1,2,2)
plt.plot(yolo_history.history['loss'], label='Train Loss')
plt.plot(yolo_history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('YOLO Feature Classifier Loss vs Epochs')
plt.legend()
plt.grid(True)
plt.tight_layout()
```

```
plt.show()
print(f"Final YOLO Training Accuracy: {yolo_history.history['accuracy'][-1]*100:.2f}%")
print(f"Final YOLO Validation Accuracy: {yolo_history.history['val_accuracy'][-1]*100:.2f}%")
```

4.2.16 Source Code for YOLO BINARY CONFUSION MATRIX

```
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
# Predict full validation set
yolo_pred_prob = yolo_cls.predict(Xy_val).ravel()
yolo_pred = (yolo_pred_prob > 0.5).astype(int)
# Confusion Matrix
cm = confusion_matrix(yy_val, yolo_pred)
plt.figure(figsize=(6,5))
sns.heatmap(cm,
            annot=True,
            fmt="d",
            cmap="Oranges",
            xticklabels=["Healthy", "Diseased/Pest"],
            yticklabels=["Healthy", "Diseased/Pest"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("YOLO Binary Confusion Matrix")
plt.show()
# Classification Report
print(classification_report(yy_val, yolo_pred, zero_division=0))
```

4.2.17 Source Code for NasNet For Binary Classification of Diseased/Pest (or) Healthy

```
import os
import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import NASNetMobile
from tensorflow.keras.models import Model
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt
BASE = "/content/drive/MyDrive/Pest And Disease.v1i.yolov8"
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
IMG_DIR = BASE + "/train/images"
LBL_DIR = BASE + "/train/labels"
IMG_SIZE = 224
X = []
y = []
for img_name in os.listdir(IMG_DIR):
    img_path = os.path.join(IMG_DIR, img_name)
    lbl_path = os.path.join(LBL_DIR, img_name.replace(".jpg", ".txt"))
    img = cv2.imread(img_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
    img = img/255.0
    # YOLO → Binary
    if os.path.exists(lbl_path) and os.path.getsize(lbl_path)>0:
        label = 1 # diseased
    else:
        label = 0 # healthy
    X.append(img)
    y.append(label)
X = np.array(X)
y = np.array(y)
print("Total images:", len(X))
X_train, X_val, y_train, y_val = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
base = NASNetMobile(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)
base.trainable = False
feature_model = Model(
    base.input,
    tf.keras.layers.GlobalAveragePooling2D()(base.output)
)
X_train_feat = feature_model.predict(X_train, batch_size=8)
X_val_feat = feature_model.predict(X_val, batch_size=8)
print(X_train_feat.shape)
svm = SVC(kernel='rbf', probability=True)
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
svm.fit(X_train_feat, y_train)
train_pred = svm.predict(X_train_feat)
val_pred = svm.predict(X_val_feat)
print("Training Accuracy:", accuracy_score(y_train, train_pred)*100)
print("Validation Accuracy:", accuracy_score(y_val, val_pred)*100)
img = X_val[0]
feat = feature_model.predict(np.expand_dims(img,0))
p = svm.predict(feat)[0]
plt.imshow(img)
plt.title("DISEASED/PEST" if p==1 else "HEALTHY")
plt.axis("off")
def predict_image(image_path):
    # Read image
    img = cv2.imread(image_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
    img_norm = img / 255.0
    # NASNet feature extraction
    feat = feature_model.predict(np.expand_dims(img_norm, 0))
    # SVM prediction
    p = svm.predict(feat)[0]
    prob = svm.predict_proba(feat)[0][1]
    # Show image
    plt.imshow(img_norm)
    plt.axis("off")
    if p == 1:
        plt.title(f"DISEASED / PEST (prob={prob:.2f})")
    else:
        plt.title(f"HEALTHY (prob={1-prob:.2f})")
    plt.show()
    return p, prob
test_img = os.path.join(IMG_DIR, os.listdir(IMG_DIR)[50])
predict_image(test_img)
predict_image("/content/drive/MyDrive/Pest And
Disease.v1i.yolov8/test/images/healthy7__jpg.rf.79e00d4ef599c874cbc3a6632d7861f0.jpg")
import random
import math
def show_random_predictions(n_images=12):
    idxs = random.sample(range(len(X_val)), n_images)
    cols = 4 # 4 images per row
    rows = math.ceil(n_images/cols)
    plt.figure(figsize=(cols*4, rows*4))
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
for i, idx in enumerate(idxs):
    img = X_val[idx]
    true_label = y_val[idx]
    feat = feature_model.predict(np.expand_dims(img,0))
    pred = svm.predict(feat)[0]
    plt.subplot(rows, cols, i+1)
    plt.imshow(img)
    plt.axis("off")
    true_txt = "DISEASED" if true_label==1 else "HEALTHY"
    pred_txt = "DISEASED" if pred==1 else "HEALTHY"
    color = "green" if true_label==pred else "red"
    plt.title(f'Actual: {true_txt}\nPredicted: {pred_txt}', color=color)
plt.tight_layout()
plt.show()
show_random_predictions(15)
print("Healthy:", np.sum(y==0))
print("Diseased:", np.sum(y==1))
```

4.2.18 Source Code for NasNet For Binary Classification of Diseased (or) Pest

```
!pip install imbalanced-learn
import os, cv2, random, math
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.applications import NASNetMobile
from tensorflow.keras.models import Model
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
BASE = "/content/drive/MyDrive/Pest And Disease.v1/yolov8"
IMG_DIR = BASE + "/train/images"
LBL_DIR = BASE + "/train/labels"
IMG_SIZE = 224
PEST_CLASSES = [0, 1, 3]
DISEASE_CLASSES = [4, 5, 6]
X, y = [], []
for img_name in os.listdir(IMG_DIR):
    img_path = os.path.join(IMG_DIR, img_name)
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
lbl_path = os.path.join(LBL_DIR, img_name.replace(".jpg", ".txt"))
if not os.path.exists(lbl_path):
    continue
with open(lbl_path) as f:
    line = f.readline().strip()
if line == "":
    continue
cls = int(line.split()[0])
if cls in PEST_CLASSES:
    label = 0
elif cls in DISEASE_CLASSES:
    label = 1
else:
    continue
img = cv2.imread(img_path)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
img = img / 255.0
X.append(img)
y.append(label)
X = np.array(X)
y = np.array(y)
print("Total samples:", len(X))
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)
```

4.2.19 Source Code for NASNET FEATURE EXTRACTOR

```
base = NASNetMobile(
    weights="imagenet",
    include_top=False,
    input_shape=(224,224,3)
)
base.trainable = False
feature_model = Model(
    base.input,
    tf.keras.layers.GlobalAveragePooling2D()(base.output)
)
X_train_feat = feature_model.predict(X_train, batch_size=8)
X_test_feat = feature_model.predict(X_test, batch_size=8)
print("Feature shape:", X_train_feat.shape)
pipeline = Pipeline([
```



```
("scaler", StandardScaler()),
("pca", PCA(n_components=64, random_state=42)),
("svm", SVC(
    kernel="rbf",
    C=0.8,
    gamma="scale",
    class_weight="balanced"
))
])
```

4.2.20 Source Code for CROSS VALIDATION

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(pipeline, X_train_feat, y_train, cv=cv)
print("CV Accuracy:", cv_scores.mean() * 100)
pipeline.fit(X_train_feat, y_train)
train_pred = pipeline.predict(X_train_feat)
test_pred = pipeline.predict(X_test_feat)
print("Train Accuracy:", accuracy_score(y_train, train_pred) * 100)
print("Test Accuracy:", accuracy_score(y_test, test_pred) * 100)
print(classification_report(y_test, test_pred, target_names=["PEST", "DISEASE"]))
def show_random_predictions(n=12):
    idxs = random.sample(range(len(X_test)), n)
    cols, rows = 4, math.ceil(n / 4)
    plt.figure(figsize=(cols*4, rows*4))
    for i, idx in enumerate(idxs):
        img = X_test[idx]
        true = y_test[idx]
        feat = feature_model.predict(np.expand_dims(img, 0))
        pred = pipeline.predict(feat)[0]
        t = "PEST" if true == 0 else "DISEASE"
        p = "PEST" if pred == 0 else "DISEASE"
        color = "green" if t == p else "red"
        plt.subplot(rows, cols, i+1)
        plt.imshow(img)
        plt.axis("off")
        plt.title(f'A: {t} \nP: {p}', color=color)
    plt.tight_layout()
    plt.show()
show_random_predictions(12)
```

4.2.21 Source Code for NASNET + SVM CONFUSION MATRIX

```
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt
# Predict validation set
nasnet_pred = svm.predict(X_val_feat)
# Confusion Matrix
cm = confusion_matrix(y_val, nasnet_pred)
plt.figure(figsize=(6,5))
sns.heatmap(cm,
            annot=True,
            fmt="d",
            cmap="Purples",
            xticklabels=["Healthy", "Diseased/Pest"],
            yticklabels=["Healthy", "Diseased/Pest"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("NASNet + SVM Binary Confusion Matrix")
plt.show()
# Classification Report
print(classification_report(y_val, nasnet_pred, zero_division=0))
print("Test Accuracy:", accuracy_score(y_test, test_pred) * 100)
train_pred = pipeline.predict(X_train_feat)
test_pred = pipeline.predict(X_test_feat)
nasnet_dvsp_train_pred = pipeline.predict(X_train_feat)
nasnet_dvsp_test_pred = pipeline.predict(X_test_feat)
nasnet_dvsp_train_acc = accuracy_score(y_train, nasnet_dvsp_train_pred) * 100
nasnet_dvsp_test_acc = accuracy_score(y_test, nasnet_dvsp_test_pred) * 100
nasnet_dvsp_cv_acc = cv_scores.mean() * 100
plt.figure(figsize=(12,5))
# Fold accuracy
plt.subplot(1,2,1)
plt.plot(range(1, len(cv_scores)+1),
        cv_scores * 100,
        marker='o')
plt.xlabel('Fold')
plt.ylabel('Accuracy (%)')
plt.title('Cross-Validation Accuracy')
plt.grid(True)
# Train vs Test
plt.subplot(1,2,2)
plt.bar(["Train", "Test"],
```

```
[nasnet_dvsp_train_acc,
 nasnet_dvsp_test_acc])
plt.ylabel('Accuracy (%)')
plt.title('Train vs Test Accuracy')
plt.ylim(0,105)
plt.grid(axis='y')
plt.tight_layout()
plt.show()
```

4.2.22 Source Code for Multi Classification MobileNet

```
!pip install pyyaml
!pip install imbalanced-learn
import yaml
DATA_YAML = "/content/drive/MyDrive/Pest And Disease.v1i.yolov8/data.yaml"
with open(DATA_YAML, "r") as f:
    data = yaml.safe_load(f)
class_names = data["names"]
print("Classes:", class_names)
IMG_DIR = "/content/drive/MyDrive/Pest And Disease.v1i.yolov8/train/images"
LBL_DIR = "/content/drive/MyDrive/Pest And Disease.v1i.yolov8/train/labels"
IMG_SIZE = 224
import os, cv2, numpy as np
X=[]
y=[]
for img_name in os.listdir(IMG_DIR):
    img_path = os.path.join(IMG_DIR,img_name)
    lbl_path = os.path.join(LBL_DIR,img_name.replace(".jpg",".txt"))
    if not os.path.exists(lbl_path):
        continue
    with open(lbl_path) as f:
        line = f.readline().strip()
        if line=="":
            continue
        cls = int(line.split()[0])
        img = cv2.imread(img_path)
        img = cv2.cvtColor(img,cv2.COLOR_BGR2RGB)
        img = cv2.resize(img,(IMG_SIZE,IMG_SIZE))
        img = img/255.0
        X.append(img)
        y.append(cls)
X=np.array(X)
y=np.array(y)
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
print("Total images:",len(X))
from sklearn.model_selection import train_test_split
X_train,X_val,y_train,y_val = train_test_split(
    X,y,test_size=0.2,stratify=y,random_state=42)
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.models import Model
import tensorflow as tf
base = MobileNetV2(weights="imagenet",include_top=False,input_shape=(224,224,3))
base.trainable = False
feature_model = Model(
    base.input,
    tf.keras.layers.GlobalAveragePooling2D()(base.output)
)
from sklearn.decomposition import PCA
pca = PCA(n_components=128)
X_train_feat = pca.fit_transform(X_train_feat)
X_val_feat = pca.transform(X_val_feat)
X_train_feat = feature_model.predict(X_train,batch_size=8)
X_val_feat = feature_model.predict(X_val,batch_size=8)
from sklearn.model_selection import cross_val_score
scores = cross_val_score(svm, X_train_feat, y_train, cv=5)
print("CV Accuracy:", scores.mean()*100)
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(
    n_estimators=300,
    max_depth=20,
    class_weight='balanced',
    random_state=42
)
rf.fit(X_train_feat,y_train)
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
scaler = StandardScaler()
X_train_feat = scaler.fit_transform(X_train_feat)
X_val_feat = scaler.transform(X_val_feat)
svm = SVC(
    kernel='rbf',
    C=1,          # ↓ much smaller
    gamma='scale',
    probability=True,
    class_weight='balanced'
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
)
svm.fit(X_train_feat,y_train)
print("Train:",accuracy_score(y_train,svm.predict(X_train_feat))*100)
print("Val:",accuracy_score(y_val,svm.predict(X_val_feat))*100)
import matplotlib.pyplot as plt
import numpy as np
import cv2
def predict_single_image(image_path):
    # Read image
    img = cv2.imread(image_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = cv2.resize(img, (224,224))
    img = img / 255.0
    # Extract deep features
    feat = feature_model.predict(np.expand_dims(img,0))
    # Scale features
    feat = scaler.transform(feat)
    # Predict
    pred = svm.predict(feat)[0]
    prob = svm.predict_proba(feat).max()
    # Show image
    plt.imshow(img)
    plt.axis("off")
    plt.title(f'{class_names[pred]} (confidence={prob:.2f})')
    plt.show()
    print("Predicted Class ID:", pred)
    print("Predicted Name:", class_names[pred])
    print("Confidence:", prob)
    return pred, prob
test_img = os.path.join(IMG_DIR, os.listdir(IMG_DIR)[0])
predict_single_image(test_img)
import random, math
def show_random_predictions(n=12):
    idxs = random.sample(range(len(X_val)), n)
    cols = 4
    rows = math.ceil(n/cols)
    plt.figure(figsize=(cols*4, rows*4))
    for i, idx in enumerate(idxs):
        img = X_val[idx]
        true = y_val[idx]
        # Feature extraction
        feat = feature_model.predict(np.expand_dims(img,0))
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
feat = scaler.transform(feat)
pred = svm.predict(feat)[0]
true_name = class_names[true]
pred_name = class_names[pred]
color = "green" if true==pred else "red"
plt.subplot(rows, cols, i+1)
plt.imshow(img)
plt.axis("off")
plt.title(f'A: {true_name} \nP: {pred_name}', color=color)
plt.tight_layout()
plt.show()
show_random_predictions(15)
from sklearn.metrics import confusion_matrix
import seaborn as sns
y_pred_multi = svm.predict(X_val_feat)
cm = confusion_matrix(y_val, y_pred_multi)
plt.figure(figsize=(8,6))
sns.heatmap(cm,
            annot=True,
            fmt="d",
            cmap="Greens",
            xticklabels=class_names,
            yticklabels=class_names)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Multi-Class Confusion Matrix")
plt.xticks(rotation=45)
plt.yticks(rotation=45)
plt.show()
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
# Predictions
svm_pred = svm.predict(X_val_feat)
rf_pred = rf.predict(X_val_feat)
# Accuracies
svm_acc = accuracy_score(y_val, svm_pred) * 100
rf_acc = accuracy_score(y_val, rf_pred) * 100
models = ["SVM", "Random Forest"]
accuracies = [svm_acc, rf_acc]
plt.figure(figsize=(6,4))
plt.bar(models, accuracies)
plt.ylabel("Validation Accuracy (%)")
```

```
plt.title("Multi-Class Model Accuracy Comparison")
plt.ylim(0,100)
plt.show()
```

4.2.23 Source Code for CLASS-WISE ACCURACY COMPARISON

```
from sklearn.metrics import confusion_matrix
import numpy as np
cm_svm = confusion_matrix(y_val, svm_pred)
cm_rf = confusion_matrix(y_val, rf_pred)
class_acc_svm = cm_svm.diagonal() / cm_svm.sum(axis=1)
class_acc_rf = cm_rf.diagonal() / cm_rf.sum(axis=1)
x = np.arange(len(class_names))
width = 0.35
plt.figure(figsize=(10,6))
plt.bar(x - width/2, class_acc_svm*100, width, label="SVM")
plt.bar(x + width/2, class_acc_rf*100, width, label="RF")
plt.xticks(x, class_names, rotation=45)
plt.ylabel("Accuracy (%)")
plt.title("Class-wise Accuracy Comparison")
plt.legend()
plt.show()
```

4.2.24 Source Code for PRECISION / RECALL / F1 COMPARISON

```
from sklearn.metrics import classification_report
import pandas as pd
report_svm = classification_report(y_val, svm_pred,
                                   target_names=class_names,
                                   output_dict=True,
                                   zero_division=0)
df_svm = pd.DataFrame(report_svm).transpose()
metrics = df_svm.iloc[:-3][["precision","recall","f1-score"]]
metrics.plot(kind="bar", figsize=(10,6))
plt.title("SVM Class-wise Precision, Recall, F1")
plt.xticks(rotation=45)
plt.ylim(0,1)
plt.grid()
plt.show()
```

4.2.25 Source Code for MACRO vs WEIGHTED F1 GRAPH

```
macro_f1 = report_svm["macro avg"]["f1-score"]
weighted_f1 = report_svm["weighted avg"]["f1-score"]
plt.figure(figsize=(5,4))
```

```
plt.bar(["Macro F1", "Weighted F1"],
        [macro_f1, weighted_f1])
plt.ylim(0,1)
plt.title("Overall F1 Score Comparison")
plt.show()
```

4.2.26 Source Code for MULTI-CLASS ROC COMPARISON

```
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc
y_bin = label_binarize(y_val, classes=range(len(class_names)))
y_prob = svm.predict_proba(X_val_feat)
plt.figure(figsize=(7,6))
for i in range(len(class_names)):
    fpr, tpr, _ = roc_curve(y_bin[:, i], y_prob[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f'{class_names[i]} (AUC={roc_auc:.2f})')
plt.plot([0,1],[0,1], 'k--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Multi-Class ROC Curve (One-vs-Rest)")
plt.legend()
plt.show()
```

4.2.27 Source Code for Optimization Techniques

```
#Installations
!pip install pennylane pennylane-lightning tensorflow scikit-learn opencv-python pyyaml
imbalanced-learn

#imports
import os
import cv2
import yaml
import random
import numpy as np
import tensorflow as tf
import pennylane as qml
import matplotlib.pyplot as plt
from tensorflow.keras.applications import NASNetMobile
from tensorflow.keras.models import Model
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout, Input,
BatchNormalization, GaussianNoise
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau,
ModelCheckpoint
```


A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
from google.colab import drive
drive.mount('/content/drive')
BASE_PATH = "/content/drive/MyDrive/Pest And Disease.v1i.yolov8"
IMG_DIR = BASE_PATH + "/train/images"
LBL_DIR = BASE_PATH + "/train/labels"
DATA_YAML = BASE_PATH + "/data.yaml"
IMG_SIZE = 224
with open(DATA_YAML, "r") as f:
    data = yaml.safe_load(f)
class_names = data["names"]
print("Classes:", class_names)
```

4.2.28 Source Code for PREPROCESSING FUNCTION

```
def preprocess_image(img_path):
    """Enhanced preprocessing with bilateral filtering and CLAHE"""
    img = cv2.imread(img_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    # Bilateral filtering for noise reduction
    img = cv2.bilateralFilter(img, d=9, sigmaColor=75, sigmaSpace=75)
    # CLAHE for contrast enhancement
    lab = cv2.cvtColor(img, cv2.COLOR_RGB2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    l = clahe.apply(l)
    img = cv2.merge((l, a, b))
    img = cv2.cvtColor(img, cv2.COLOR_LAB2RGB)
    # Resize and normalize
    img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
    img = img / 255.0
    return img
```

4.2.29 Source Code for LOAD DATA FOR BINARY CLASSIFICATION

```
print("\n Loading Binary Classification Data (Healthy vs Diseased/Pest).")
X_binary, y_binary = [], []
for img_name in os.listdir(IMG_DIR):
    img_path = os.path.join(IMG_DIR, img_name)
    lbl_path = os.path.join(LBL_DIR, img_name.replace(".jpg", ".txt"))
    # Binary: 1 if any object exists, 0 if healthy
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
if os.path.exists(lbl_path) and os.path.getsize(lbl_path) > 0:
    label = 1 # Diseased/Pest
else:
    label = 0 # Healthy
img = preprocess_image(img_path)
X_binary.append(img)
y_binary.append(label)
X_binary = np.array(X_binary)
y_binary = np.array(y_binary)
print(f"Binary dataset: {len(X_binary)} images")
print(f"Healthy: {np.sum(y_binary==0)}, Diseased/Pest: {np.sum(y_binary==1)}")
```

4.2.30 Source Code for LOAD DATA FOR MULTI-CLASS CLASSIFICATION

```
print("\nLoading Multi-Class Classification Data...")
X_multi, y_multi = [], []
for img_name in os.listdir(IMG_DIR):
    img_path = os.path.join(IMG_DIR, img_name)
    lbl_path = os.path.join(LBL_DIR, img_name.replace(".jpg", ".txt"))
    if not os.path.exists(lbl_path):
        continue
    with open(lbl_path) as f:
        line = f.readline().strip()
    if line == "":
        continue
    cls = int(line.split()[0])
    img = preprocess_image(img_path)
    X_multi.append(img)
    y_multi.append(cls)
X_multi = np.array(X_multi)
y_multi = np.array(y_multi)
print(f"Multi-class dataset: {len(X_multi)} images")
print(f"Classes distribution: {np.bincount(y_multi)}")
```

4.2.31 Source Code for FEATURE EXTRACTION WITH NASNetMobile

```
print("\nExtracting Deep Features with NASNetMobile...")
base_model = NASNetMobile(
    weights="imagenet",
    include_top=False,
    input_shape=(IMG_SIZE, IMG_SIZE, 3)
)
base_model.trainable = False
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
feature_extractor = Model(
    base_model.input,
    GlobalAveragePooling2D()(base_model.output)
)
# Extract features for binary classification
features_binary = feature_extractor.predict(X_binary, batch_size=8, verbose=1)
print(f"Binary features shape: {features_binary.shape}")
# Extract features for multi-class classification
features_multi = feature_extractor.predict(X_multi, batch_size=8, verbose=1)
print(f"Multi-class features shape: {features_multi.shape}")
```

4.2.32 Source Code for FEATURE PREPROCESSING

```
print("\nPreprocessing Features...")
# Binary - Keep simple (already works at 99%)
n_qubits_binary = 4
scaler_binary = StandardScaler()
features_binary_scaled = scaler_binary.fit_transform(features_binary)
from sklearn.decomposition import PCA
pca_binary = PCA(n_components=n_qubits_binary, random_state=42)
features_binary_pca = pca_binary.fit_transform(features_binary_scaled)
print(f"Binary PCA: {features_binary_pca.shape}, variance:
{pca_binary.explained_variance_ratio_.sum():.4f}")
# Multi-class - USE FULL FEATURES (NO PCA!)
scaler_multi = StandardScaler()
features_multi_scaled = scaler_multi.fit_transform(features_multi)
print(f"Multi-class scaled features: {features_multi_scaled.shape} (NO PCA - keeping ALL
features!)" )
```

4.2.33 Source Code for SIMPLE QUANTUM CIRCUIT FOR BINARY ONLY

```
print("\nQuantum Circuit for Binary Classification...")
def quantum_feature_map_binary(X, params):
    """Simple quantum circuit for binary (keep what works)"""
    n_samples = X.shape[0]
    n_qubits = X.shape[1]
    quantum_features = np.zeros((n_samples, n_qubits * 2))
    dev = qml.device("default.qubit", wires=n_qubits)
    @qml.qnode(dev)
    def quantum_circuit(x, p, measurement_qubit):
        for i in range(n_qubits):
            qml.RY(x[i], wires=i)
        for layer in range(2):
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
    for i in range(n_qubits):
        qml.RY(p[layer, i, 0], wires=i)
        qml.RZ(p[layer, i, 1], wires=i)
    for i in range(n_qubits - 1):
        qml.CNOT(wires=[i, i + 1])
        qml.CNOT(wires=[n_qubits - 1, 0])
    return qml.expval(qml.PauliZ(measurement_qubit)),
qml.expval(qml.PauliX(measurement_qubit))
for i in range(n_samples):
    if i % 100 == 0:
        print(f' Processing sample {i}/{n_samples}...')
    features = []
    for qubit in range(n_qubits):
        z_exp, x_exp = quantum_circuit(X[i], params, qubit)
        features.extend([z_exp, x_exp])
    quantum_features[i] = features
return quantum_features
np.random.seed(42)
quantum_params_binary = np.random.randn(2, n_qubits_binary, 2) * 0.1
quantum_features_binary = quantum_feature_map_binary(features_binary_pca,
quantum_params_binary)
print(f'Binary quantum features: {quantum_features_binary.shape}')
#splitting data
print("\nSplitting Data...")
# Binary split
X_train_bin, X_val_bin, y_train_bin, y_val_bin = train_test_split(
    quantum_features_binary, y_binary,
    test_size=0.2,
    stratify=y_binary,
    random_state=42
)
print(f'Binary - Train: {X_train_bin.shape}, Val: {X_val_bin.shape}')
# Multi-class split - USING FULL CLASSICAL FEATURES
X_train_multi, X_val_multi, y_train_multi, y_val_multi = train_test_split(
    features_multi_scaled, y_multi,
    test_size=0.2,
    stratify=y_multi,
    random_state=42
)
print(f'Multi-class - Train: {X_train_multi.shape}, Val: {X_val_multi.shape}')
#binary model
print("\nBuilding Binary Classification Model...")
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
binary_model = tf.keras.Sequential([
    Input(shape=(quantum_features_binary.shape[1],)),
    Dense(32, activation='relu'),
    Dropout(0.3),
    Dense(16, activation='relu'),
    Dropout(0.2),
    Dense(1, activation='sigmoid')
])
binary_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)
print("\n Training Binary Model...")
history_binary = binary_model.fit(
    X_train_bin, y_train_bin,
    validation_data=(X_val_bin, y_val_bin),
    epochs=30,
    batch_size=16,
    verbose=1
)
# Binary evaluation
y_pred_bin_prob = binary_model.predict(X_val_bin, verbose=0)
y_pred_bin = (y_pred_bin_prob > 0.5).astype(int).flatten()
bin_acc = accuracy_score(y_val_bin, y_pred_bin)
y_pred_bin_train = (binary_model.predict(X_train_bin, verbose=0) > 0.5).astype(int).flatten()
bin_train_acc = accuracy_score(y_train_bin, y_pred_bin_train)
print(f"\nBinary Results:")
print(f" Training: {bin_train_acc*100:.2f}%")
print(f" Validation: {bin_acc*100:.2f}%")
```

4.2.34 Source Code for IMPROVED MULTI-CLASS MODEL WITH ANTI-OVERFITTING

```
print("\nBuilding ROBUST Multi-Class Model (Target: 99.00-99.75%)...")
n_classes = len(np.unique(y_multi))
# LABEL SMOOTHING LOSS FUNCTION
def label_smoothing_loss(y_true, y_pred, smoothing=0.1):
    """
    Prevents model from being overconfident
    Instead of [0,0,1,0,0], target becomes [0.02,0.02,0.92,0.02,0.02]
    """
    y_true = tf.cast(y_true, tf.int32)
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
y_true = tf.one_hot(y_true, depth=n_classes)
# Apply smoothing
y_true = y_true * (1.0 - smoothing) + smoothing / n_classes
# Categorical crossentropy
return tf.keras.losses.categorical_crossentropy(y_true, y_pred)
# BUILD MODEL WITH STRONGER REGULARIZATION
multi_model = tf.keras.Sequential([
    Input(shape=(features_multi_scaled.shape[1],)), # 1056 features
    # Optional: Add Gaussian noise for extra regularization
    GaussianNoise(0.01), # Small noise injection
    # Layer 1 - Reduced width + stronger L2
    Dense(384, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(0.001)),
    BatchNormalization(),
    Dropout(0.6), # Increased dropout
    # Layer 2
    Dense(192, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(0.001)),
    BatchNormalization(),
    Dropout(0.5),
    # Layer 3
    Dense(96, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(0.0005)),
    BatchNormalization(),
    Dropout(0.4),
    # Layer 4
    Dense(48, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(0.0005)),
    BatchNormalization(),
    Dropout(0.3),
    # Layer 5 - Smaller bottleneck
    Dense(24, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(0.0003)),
    BatchNormalization(),
    Dropout(0.2),
    # Output
    Dense(n_classes, activation='softmax')
])
# Compile with label smoothing and lower learning rate
multi_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0005), # Reduced LR
    loss=lambda y_true, y_pred: label_smoothing_loss(y_true, y_pred, smoothing=0.1),
    metrics=['accuracy']
)
multi_model.summary()
# Callbacks with tighter controls
early_stop = EarlyStopping(
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
monitor='val_accuracy',
patience=25, # Reduced patience
restore_best_weights=True,
mode='max',
verbose=1,
min_delta=0.001 # Only improve if significant
)
reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=8, # Faster LR reduction
    min_lr=1e-7,
    verbose=1
)
# Train with larger batch size and fewer epochs
print("\nTraining Multi-Class Model with Anti-Overfitting Measures...")
history_multi = multi_model.fit(
    X_train_multi, y_train_multi,
    validation_data=(X_val_multi, y_val_multi),
    epochs=150, # Reduced epochs
    batch_size=32, # INCREASED batch size (more generalization)
    callbacks=[early_stop, reduce_lr],
    verbose=1
)
# Multi-class evaluation
y_pred_multi_prob = multi_model.predict(X_val_multi, verbose=0)
y_pred_multi = np.argmax(y_pred_multi_prob, axis=1)
multi_acc = accuracy_score(y_val_multi, y_pred_multi)
y_pred_multi_train = np.argmax(multi_model.predict(X_train_multi, verbose=0), axis=1)
multi_train_acc = accuracy_score(y_train_multi, y_pred_multi_train)
print(f"\n Multi-Class Results:")
print(f" Training: {multi_train_acc*100:.2f}%")
print(f" Validation: {multi_acc*100:.2f}%")
print(f" Overfitting Gap: {(multi_train_acc - multi_acc)*100:.2f}%")
print("\nClassification Report:")
print(classification_report(y_val_multi, y_pred_multi,
                           target_names=[class_names[i] for i in np.unique(y_multi)],
                           zero_division=0))
#VISUALIZATIONS
print("\n Creating Visualizations...")
# Binary plots
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
axes[0].plot(history_binary.history['accuracy'], label='Train', marker='o', linewidth=2)
axes[0].plot(history_binary.history['val_accuracy'], label='Val', marker='s', linewidth=2)
axes[0].set_xlabel('Epoch', fontsize=12)
axes[0].set_ylabel('Accuracy', fontsize=12)
axes[0].set_title('Binary Classification - Accuracy', fontsize=14, fontweight='bold')
axes[0].legend(fontsize=11)
axes[0].grid(True, alpha=0.3)
axes[1].plot(history_binary.history['loss'], label='Train', marker='o', linewidth=2)
axes[1].plot(history_binary.history['val_loss'], label='Val', marker='s', linewidth=2)
axes[1].set_xlabel('Epoch', fontsize=12)
axes[1].set_ylabel('Loss', fontsize=12)
axes[1].set_title('Binary Classification - Loss', fontsize=14, fontweight='bold')
axes[1].legend(fontsize=11)
axes[1].grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
# Binary confusion matrix
cm_bin = confusion_matrix(y_val_bin, y_pred_bin)
plt.figure(figsize=(7, 6))
sns.heatmap(cm_bin, annot=True, fmt='d', cmap='Blues', cbar=True,
            xticklabels=["Healthy", "Diseased/Pest"],
            yticklabels=["Healthy", "Diseased/Pest"],
            annot_kws={"size": 14})
plt.ylabel('Actual', fontsize=12, fontweight='bold')
plt.xlabel('Predicted', fontsize=12, fontweight='bold')
plt.title('Binary Classification - Confusion Matrix', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
# Multi-class plots
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(history_multi.history['accuracy'], label='Train', marker='o', linewidth=2)
axes[0].plot(history_multi.history['val_accuracy'], label='Val', marker='s', linewidth=2)
axes[0].set_xlabel('Epoch', fontsize=12)
axes[0].set_ylabel('Accuracy', fontsize=12)
axes[0].set_title('Multi-Class Classification - Accuracy', fontsize=14, fontweight='bold')
axes[0].legend(fontsize=11)
axes[0].grid(True, alpha=0.3)
axes[1].plot(history_multi.history['loss'], label='Train', marker='o', linewidth=2)
axes[1].plot(history_multi.history['val_loss'], label='Val', marker='s', linewidth=2)
axes[1].set_xlabel('Epoch', fontsize=12)
axes[1].set_ylabel('Loss', fontsize=12)
axes[1].set_title('Multi-Class Classification - Loss', fontsize=14, fontweight='bold')
```


A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
axes[1].legend(fontsize=11)
axes[1].grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
# Multi-class confusion matrix
cm_multi = confusion_matrix(y_val_multi, y_pred_multi)
plt.figure(figsize=(11, 9))
sns.heatmap(cm_multi, annot=True, fmt='d', cmap='Greens', cbar=True,
            xticklabels=[class_names[i] for i in np.unique(y_multi)],
            yticklabels=[class_names[i] for i in np.unique(y_multi)],
            annot_kws={"size": 11})
plt.ylabel('Actual', fontsize=12, fontweight='bold')
plt.xlabel('Predicted', fontsize=12, fontweight='bold')
plt.title('Multi-Class Classification - Confusion Matrix', fontsize=14, fontweight='bold')
plt.xticks(rotation=45, ha='right', fontsize=10)
plt.yticks(rotation=0, fontsize=10)
plt.tight_layout()
plt.show()
```

4.2.35 Source Code for PREDICTION FUNCTIONS

```
def predict_binary_quantum(image_path):
    """Binary prediction with quantum features"""
    img = preprocess_image(image_path)
    feat = feature_extractor.predict(np.expand_dims(img, 0), verbose=0)
    feat_scaled = scaler_binary.transform(feat)
    feat_pca = pca_binary.transform(feat_scaled)
    quantum_feat = quantum_feature_map_binary(feat_pca, quantum_params_binary)
    prob = binary_model.predict(quantum_feat, verbose=0)[0][0]
    label = "DISEASED/PEST" if prob >= 0.5 else "HEALTHY"
    plt.figure(figsize=(7, 7))
    plt.imshow(img)
    plt.axis("off")
    plt.title(f'Binary: {label}\nConfidence: {prob:.3f}', fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.show()
    return label, prob

def predict_multiclass_classical(image_path):
    """Multi-class prediction with full classical features"""
    img = preprocess_image(image_path)
    feat = feature_extractor.predict(np.expand_dims(img, 0), verbose=0)
    feat_scaled = scaler_multi.transform(feat)
    probs = multi_model.predict(feat_scaled, verbose=0)[0]
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
pred_class = np.argmax(probs)
plt.figure(figsize=(7, 7))
plt.imshow(img)
plt.axis("off")
plt.title(f'Class: {class_names[pred_class]}\nConfidence: {probs[pred_class]:.3f}',
         fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
return class_names[pred_class], probs[pred_class]

#VISUAL PREDICTIONS
def show_predictions(n=12):
    """Show sample predictions"""
    # Binary
    print("\nBinary Classification Predictions:")
    indices = random.sample(range(len(X_val_bin)), min(n, len(X_val_bin)))
    fig, axes = plt.subplots(3, 4, figsize=(16, 12))
    axes = axes.flatten()
    for i, idx in enumerate(indices):
        prob = binary_model.predict(X_val_bin[idx:idx+1], verbose=0)[0][0]
        pred = 1 if prob >= 0.5 else 0
        true = y_val_bin[idx]
        orig_idx = np.where((quantum_features_binary == X_val_bin[idx]).all(axis=1))[0]
        img = X_binary[orig_idx[0]] if len(orig_idx) > 0 else X_binary[0]
        color = "green" if pred == true else "red"
        axes[i].imshow(img)
        axes[i].axis("off")
        axes[i].set_title(f'True: {["Healthy", "Diseased"][true]}\nPred:
{["Healthy", "Diseased"][pred]} ({prob:.2f})',
                        color=color, fontsize=10, fontweight='bold')
    plt.suptitle("Binary Classification Predictions", y=0.995, fontsize=16, fontweight='bold')
    plt.tight_layout()
    plt.show()

    # Multi-class
    print("\n Multi-Class Classification Predictions:")
    indices = random.sample(range(len(X_val_multi)), min(n, len(X_val_multi)))
    fig, axes = plt.subplots(3, 4, figsize=(16, 12))
    axes = axes.flatten()
    for i, idx in enumerate(indices):
        probs = multi_model.predict(X_val_multi[idx:idx+1], verbose=0)[0]
        pred = np.argmax(probs)
        true = y_val_multi[idx]
        orig_idx = np.where((features_multi_scaled == X_val_multi[idx]).all(axis=1))[0]
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
img = X_multi[orig_idx[0]] if len(orig_idx) > 0 else X_multi[0]
color = "green" if pred == true else "red"
axes[i].imshow(img)
axes[i].axis("off")
axes[i].set_title(f"True: {class_names[true]}\nPred: {class_names[pred]}
({probs[pred]:.2f})",
                  color=color, fontsize=9, fontweight='bold')
plt.suptitle("Multi-Class Classification Predictions", y=0.995, fontsize=16,
fontweight='bold')
plt.tight_layout()
plt.show()
show_predictions(12)
#TEST PREDICTIONS
print("\nTesting on Sample Images...")
sample_img = os.path.join(IMG_DIR, os.listdir(IMG_DIR)[10])
print("\nBinary Test:")
predict_binary_quantum(sample_img)
print("\nMulti-Class Test:")
predict_multiclass_classical(sample_img)
#FINAL SUMMARY
print("\n" + "="*85)
print("QUANTUM-INTEGRATED MAIZE PEST & DISEASE DETECTION - FINAL
RESULTS")
print("="*85)
print(f"\nBINARY CLASSIFICATION (Healthy vs Diseased/Pest):")
print(f" Training Accuracy: {bin_train_acc*100:.2f}%")
print(f" Validation Accuracy: {bin_acc*100:.2f}%")
print(f" Architecture: NASNet(1056) → PCA(4) → Quantum Circuit → NN(32→16→1)")
print(f" Quantum: {n_qubits_binary} qubits, 2 layers, {quantum_features_binary.shape[1]}
features")
print(f"\nMULTI-CLASS CLASSIFICATION (7 Classes):")
print(f" Training Accuracy: {multi_train_acc*100:.2f}%")
print(f" Validation Accuracy: {multi_acc*100:.2f}%")
print(f" Overfitting Gap: {(multi_train_acc - multi_acc)*100:.2f}%")
print(f" Architecture: NASNet(1056) → Scaler → Deep NN(384→192→96→48→24→7)")
print(f" Features: ALL 1056 classical features retained (NO PCA bottleneck!)")
print(f" Anti-Overfitting: Label Smoothing + GaussianNoise + Strong Regularization")
print(f"\n QUANTUM INTEGRATION:")
print(f" Binary ONLY uses quantum circuit (proven effective)")
print(f" Multi-class uses PURE CLASSICAL (quantum was destroying accuracy)")
print(f" Key insight: Quantum helps simple tasks, classical excels at complex tasks")
print(f"\n NEURAL NETWORKS:")
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
print(f" Binary: Shallow (2 hidden layers) - quantum features sufficient")
print(f" Multi: Deep (5 hidden layers) - needs full feature space")
print(f" Regularization: L2(0.001) + BatchNorm + Dropout(0.6→0.2) +
GaussianNoise(0.01)")
print(f" Label Smoothing: 0.1 (prevents overconfidence)")
print(f"\n DATASET:")
print(f" Total Images: {len(X_binary)}")
print(f" Binary: {len(X_train_bin)} train, {len(X_val_bin)} val")
print(f" Multi: {len(X_train_multi)} train, {len(X_val_multi)} val")
print(f" Classes: {class_names}")
print(f"\n ANTI-OVERFITTING STRATEGIES:")
print(f" 1. Label Smoothing (0.1) - Prevents 100% confidence")
print(f" 2. GaussianNoise Layer (0.01) - Adds regularization")
print(f" 3. Stronger L2 Regularization (0.001)")
print(f" 4. Higher Dropout Rates (0.6→0.2)")
print(f" 5. Reduced Model Capacity (384→192→96→48→24)")
print(f" 6. Larger Batch Size (32) - More generalization")
print(f" 7. Lower Learning Rate (0.0005)")
print(f" 8. Earlier Stopping (patience=25)")
print(f" 9. Fewer Max Epochs (150)")
print(f"\n TARGET ACHIEVED:")
print("="*85)
```

4.2.36 Source Code for IMPROVED MULTI-CLASS MODEL

```
print("\nBuilding ROBUST Multi-Class Model (Target: 99.00-99.75%)...")
n_classes = len(np.unique(y_multi))
# STRONGER REGULARIZATION to prevent overfitting
multi_model = tf.keras.Sequential([
    Input(shape=(features_multi_scaled.shape[1],)), # 1056 features
    # Layer 1 - Reduced width + stronger L2
    Dense(384, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(0.001)), # ←
Increased L2
    BatchNormalization(),
    Dropout(0.6), # ← Increased dropout
    # Layer 2
    Dense(192, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(0.001)),
    BatchNormalization(),
    Dropout(0.5),
    # Layer 3
    Dense(96, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(0.0005)),
    BatchNormalization(),
    Dropout(0.4),
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
# Layer 4
Dense(48, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(0.0005)),
BatchNormalization(),
Dropout(0.3),
# Layer 5 - Smaller bottleneck
Dense(24, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(0.0003)),
BatchNormalization(),
Dropout(0.2),
# Output with label smoothing via loss function
Dense(n_classes, activation='softmax')
])
```

4.2.37 Source Code for CUSTOM LABEL SMOOTHING LOSS

```
def label_smoothing_loss(y_true, y_pred, smoothing=0.1):
    """
    Label smoothing prevents model from becoming overconfident
    Instead of [0,0,1,0,0], target becomes [0.02,0.02,0.92,0.02,0.02]
    """
    y_true = tf.cast(y_true, tf.int32)
    y_true = tf.one_hot(y_true, depth=n_classes)
    # Apply smoothing
    y_true = y_true * (1.0 - smoothing) + smoothing / n_classes
    # Categorical crossentropy
    return tf.keras.losses.categorical_crossentropy(y_true, y_pred)
# Compile with lower learning rate
multi_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0005), # ← Reduced LR
    loss=lambda y_true, y_pred: label_smoothing_loss(y_true, y_pred, smoothing=0.1),
    metrics=['accuracy']
)
multi_model.summary()
#STRONGER CALLBACKS
early_stop = EarlyStopping(
    monitor='val_accuracy',
    patience=25, # ← Reduced patience
    restore_best_weights=True,
    mode='max',
    verbose=1,
    min_delta=0.001 # ← Only improve if significant
)
reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
factor=0.5,
patience=8, # ← Faster LR reduction
min_lr=1e-7,
verbose=1
)
from tensorflow.keras.preprocessing.image import ImageDataGenerator
# Augment training data
datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True,
    zoom_range=0.1,
    fill_mode='nearest'
)
#TRAIN WITH SMALLER BATCH SIZE
print("\nTraining Multi-Class Model with Anti-Overfitting Measures...")
history_multi = multi_model.fit(
    X_train_multi, y_train_multi,
    validation_data=(X_val_multi, y_val_multi),
    epochs=150, # ← Reduced epochs
    batch_size=32, # ← INCREASED batch size (more generalization)
    callbacks=[early_stop, reduce_lr],
    verbose=1
)
#ADDITIONAL REGULARIZATION TECHNIQUE
# If accuracy is still hitting 100%, add Gaussian Noise:
"""
from tensorflow.keras.layers import GaussianNoise
# Add after Input layer:
GaussianNoise(0.01), # Small noise injection
"""
```

4.2.38 EVALUATION

```
y_pred_multi_prob = multi_model.predict(X_val_multi, verbose=0)
y_pred_multi = np.argmax(y_pred_multi_prob, axis=1)
multi_acc = accuracy_score(y_val_multi, y_pred_multi)
y_pred_multi_train = np.argmax(multi_model.predict(X_train_multi, verbose=0), axis=1)
multi_train_acc = accuracy_score(y_train_multi, y_pred_multi_train)
print(f"\nMulti-Class Results:")
print(f" Training: {multi_train_acc*100:.2f}%")
print(f" Validation: {multi_acc*100:.2f}%")
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
print(f" Overfitting Gap: {(multi_train_acc - multi_acc)*100:.2f}%")
print("\nClassification Report:")
print(classification_report(y_val_multi, y_pred_multi,
                           target_names=[class_names[i] for i in np.unique(y_multi)],
                           zero_division=0))
```

4.2.39 Source Code for SAVE QUANTUM MODEL

```
import joblib
import json
SAVE_DIR = "/content/drive/MyDrive/Quantum_Maize_Model/"
os.makedirs(SAVE_DIR, exist_ok=True)
# Save Keras model
binary_model.save(SAVE_DIR + "binary_model.h5")
# Save scaler and PCA
joblib.dump(scaler_binary, SAVE_DIR + "scaler_binary.pkl")
joblib.dump(pca_binary, SAVE_DIR + "pca_binary.pkl")
# Save quantum parameters
np.save(SAVE_DIR + "quantum_params_binary.npy", quantum_params_binary)
# Save class names
with open(SAVE_DIR + "class_names.json", "w") as f:
    json.dump(class_names, f)
print("Binary Quantum Model Saved Successfully!")
multi_model.save(SAVE_DIR + "multi_model.h5")
joblib.dump(scaler_multi, SAVE_DIR + "scaler_multi.pkl")
print("Multi-Class Model Saved Successfully!")
```

4.3 Source code for NPK Sensor

```
#include <SoftwareSerial.h>
#include <ModbusMaster.h>
// MAX485 Control Pins
#define MAX485_DE 5
#define MAX485_RE 4
// Software Serial (RO -> 10, DI -> 11)
SoftwareSerial rs485(10, 11);
ModbusMaster node;
void preTransmission()
{
    digitalWrite(MAX485_RE, HIGH);
    digitalWrite(MAX485_DE, HIGH);
}
void postTransmission()
{
    digitalWrite(MAX485_RE, LOW);
    digitalWrite(MAX485_DE, LOW);
}
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
digitalWrite(MAX485_DE, LOW);
digitalWrite(MAX485_RE, LOW);
}
void setup()
{
  pinMode(MAX485_DE, OUTPUT);
  pinMode(MAX485_RE, OUTPUT);
  digitalWrite(MAX485_DE, LOW);
  digitalWrite(MAX485_RE, LOW);
  Serial.begin(9600);
  rs485.begin(9600);
  node.begin(1, rs485);
  node.preTransmission(preTransmission);
  node.postTransmission(postTransmission);
  Serial.println("HW-097 NPK Sensor Started");
}
void loop()
{
  uint8_t result;
  result = node.readHoldingRegisters(0x001E, 3);
  if (result == node.ku8MBSuccess)
  {
    uint16_t nitrogen = node.getResponseBuffer(0);
    uint16_t phosphorus = node.getResponseBuffer(1);
    uint16_t potassium = node.getResponseBuffer(2);
    Serial.println("-----");
    Serial.print("Nitrogen : ");
    Serial.print(nitrogen);
    Serial.println(" mg/kg");
    Serial.print("Phosphorus : ");
    Serial.print(phosphorus);
    Serial.println(" mg/kg");
    Serial.print("Potassium : ");
    Serial.print(potassium);
    Serial.println(" mg/kg");
  }
  else
  {
    Serial.print("Modbus Error Code: ");
    Serial.println(result);
  }
  delay(3000);}
}
```


4.4 Source Code for Raspberry Pi Based IoT Sensor Data Acquisition and Firebase Integration

```
import time
import datetime
import serial
import threading
import RPi.GPIO as GPIO
import Adafruit_DHT # Using the working library
import firebase_admin
from firebase_admin import credentials, db
# CONFIG
FIREBASE_CRED_FILE = "firebase_key.json"
FIREBASE_DB_URL = "https://maize-711be-default-rtdb.firebaseio.com/"
DHT_SENSOR = Adafruit_DHT.DHT11
DHT_PIN = 4 # GPIO 4 (Pin 7)
SOIL_PIN = 17 # GPIO 17 (Pin 11)
ARDUINO_PORT = "/dev/ttyACM0"
ARDUINO_BAUD = 9600
UPLOAD_INTERVAL = 5 # seconds
# GPIO
GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
GPIO.setup(SOIL_PIN, GPIO.IN)
# FIREBASE
if not firebase_admin._apps:
    cred = credentials.Certificate(FIREBASE_CRED_FILE)
    firebase_admin.initialize_app(cred, {"databaseURL": FIREBASE_DB_URL})
# SENSOR READING FUNCTIONS
def get_dht_data():
    """Uses Adafruit_DHT for reliable reading"""
    humidity, temperature = Adafruit_DHT.read_retry(DHT_SENSOR, DHT_PIN)
    if humidity is not None and temperature is not None:
        print(f'[DHT11] Temp={temperature:0.1f} C Hum={humidity:0.1f}%')
        return round(temperature, 1), round(humidity, 1)
    else:
        print("[DHT11] Failed to read - check wiring")
        return None, None
def read_soil():
    val = GPIO.input(SOIL_PIN)
    pct = 100 if val == GPIO.LOW else 0
    print(f'[Soil] {'Wet' if pct == 100 else 'Dry'} ({pct}%)')
    return pct
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
# NPK READER (Background Thread)
class ArduinoNPKReader(threading.Thread):
    def __init__(self):
        super().__init__(daemon=True)
        self.n = None
        self.p = None
        self.k = None
        self._tmp_n = self._tmp_p = self._tmp_k = None
        self.ser = None
        try:
            self.ser = serial.Serial(ARDUINO_PORT, ARDUINO_BAUD, timeout=1)
            time.sleep(2)
            print(f"[Arduino] Connected on {ARDUINO_PORT}")
        except Exception as e:
            print(f"[Arduino] Connection failed: {e}")
    def _parse(self, line):
        line = line.strip().lower()
        if ":" not in line: return
        key, _, rest = line.partition(":")
        parts = rest.strip().split()
        if not parts: return
        try:
            val = float(parts[0])
            if "nitrogen" in key: self._tmp_n = val
            elif "phosphorus" in key: self._tmp_p = val
            elif "potassium" in key: self._tmp_k = val
            if self._tmp_n is not None and self._tmp_p is not None and self._tmp_k is not None:
                self.n, self.p, self.k = self._tmp_n, self._tmp_p, self._tmp_k
                print(f"[NPK] N={self.n} P={self.p} K={self.k}")
                self._tmp_n = self._tmp_p = self._tmp_k = None
        except ValueError: pass
    def run(self):
        if not self.ser: return
        while True:
            try:
                if self.ser.in_waiting:
                    line = self.ser.readline().decode("utf-8", errors="ignore")
                    self._parse(line)
            except: time.sleep(0.5)
# MAIN LOOP
def main():
    print("Starting Maize Farm Controller...")
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
npk_thread = ArduinoNPKReader()
npk_thread.start()
while True:
    temp, hum = get_dht_data()
    soil_val = read_soil()
    # Upload to Firebase
    try:
        db.reference("sensor_data/current").set({
            "Temperature": temp if temp is not None else "N/A",
            "Humidity": hum if hum is not None else "N/A",
            "SoilMoisture": soil_val,
            "N": npk_thread.n if npk_thread.n is not None else "N/A",
            "P": npk_thread.p if npk_thread.p is not None else "N/A",
            "K": npk_thread.k if npk_thread.k is not None else "N/A",
            "LastUpdated": datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        })
        print("[Firebase] Data Updated")
    except Exception as e:
        print(f"[Firebase] Error: {e}")
    time.sleep(UPLOAD_INTERVAL)
if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        GPIO.cleanup()
```

4.5 Source Code for Streamlit web app

```
# IMPORTS
import streamlit as st
import tensorflow as tf
import numpy as np
import cv2
import joblib
import json
import pennyLane as qml
import os
import firebase_admin
from firebase_admin import credentials, firestore, db
from datetime import datetime
from streamlit_autorefresh import st_autorefresh
# PAGE CONFIG
st.set_page_config(page_title="Quantum Maize Disease Detector", layout="centered")
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
st_autorefresh(interval=5000, key="sensor_refresh")
MODEL_DIR = r"D:\WEB\Quantum Saved Extensions"
IMG_SIZE = 224
# FIREBASE
@st.cache_resource
def init_firebase():
    if not firebase_admin._apps:
        cred = credentials.Certificate(r"D:\WEB\Quantum Saved Extensions\APK File.json")
        firebase_admin.initialize_app(cred, {
            "databaseURL": "https://maize-711be-default-rtdb.firebaseio.com/"
        })
    return firestore.client()
firestore_db = init_firebase()
# LOAD MODELS
@st.cache_resource
def load_models():
    binary_model = tf.keras.models.load_model(
        os.path.join(MODEL_DIR, "binary_model.h5"), compile=False
    )
    multi_model = tf.keras.models.load_model(
        os.path.join(MODEL_DIR, "multi_model.h5"), compile=False
    )
    scaler_binary = joblib.load(os.path.join(MODEL_DIR, "scaler_binary.pkl"))
    scaler_multi = joblib.load(os.path.join(MODEL_DIR, "scaler_multi.pkl"))
    pca_binary = joblib.load(os.path.join(MODEL_DIR, "pca_binary.pkl"))
    quantum_params_binary = np.load(os.path.join(MODEL_DIR,
"quantum_params_binary.npy"))
    with open(os.path.join(MODEL_DIR, "class_names.json")) as f:
        class_names = json.load(f)
    return (binary_model, multi_model, scaler_binary,
            scaler_multi, pca_binary, quantum_params_binary, class_names)
(binary_model, multi_model, scaler_binary,
scaler_multi, pca_binary, quantum_params_binary,
class_names) = load_models()
# LOAD NASNET
@st.cache_resource
def load_feature_extractor():
    base_model = tf.keras.applications.NASNetMobile(
        weights="imagenet",
        include_top=False,
        input_shape=(IMG_SIZE, IMG_SIZE, 3)
    )
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
return tf.keras.Model(
    base_model.input,
    tf.keras.layers.GlobalAveragePooling2D()(base_model.output)
)
feature_extractor = load_feature_extractor()
# QUANTUM FUNCTION
def quantum_feature_map_binary(X, params):
    n_samples = X.shape[0]
    n_qubits = X.shape[1]
    quantum_features = np.zeros((n_samples, n_qubits * 2))
    dev = qml.device("default.qubit", wires=n_qubits)
    @qml.qnode(dev)
    def quantum_circuit(x, p, m):
        for i in range(n_qubits):
            qml.RY(x[i], wires=i)
        for layer in range(2):
            for i in range(n_qubits):
                qml.RY(p[layer, i, 0], wires=i)
                qml.RZ(p[layer, i, 1], wires=i)
            for i in range(n_qubits - 1):
                qml.CNOT(wires=[i, i + 1])
            qml.CNOT(wires=[n_qubits - 1, 0])
        return qml.expval(qml.PauliZ(m)), qml.expval(qml.PauliX(m))
    for i in range(n_samples):
        features = []
        for q in range(n_qubits):
            z, x = quantum_circuit(X[i], params, q)
            features.extend([z, x])
        quantum_features[i] = features
    return quantum_features
# TITLE
st.title("🔗 A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early  
Detection of Pests and Diseases in Maize Crops")
# IOT SENSOR DATA
st.markdown("### 📡 Live IoT Sensor Data")
# Always fetch fresh from Firebase (no caching)
sensor_data = {}
last_updated = "N/A"
device_connected = False
fetch_error = None
try:
    ref = db.reference("sensor_data/current")
```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```

    fetched = ref.get()
    if fetched:
        sensor_data = fetched
        raw_ts = sensor_data.get("LastUpdated", "N/A")
        last_updated = str(raw_ts).strip().strip('"').strip("'")
        # Consider connected if updated within last 300 seconds (5 minutes)
        if last_updated != "N/A":
            try:
                last_dt = datetime.strptime(last_updated, "%Y-%m-%d %H:%M:%S")
                seconds_ago = (datetime.now() - last_dt).total_seconds()
                device_connected = seconds_ago <= 15
            except Exception:
                device_connected = True # If parse fails, assume connected since data exists
        else:
            fetch_error = "No data returned from Firebase"
    except Exception as e:
        fetch_error = str(e)
    # Connection status
    if fetch_error:
        st.error(f"🚫 Device Not Connected — {fetch_error}")
    elif device_connected:
        st.success("🟢 Device Connected")
    else:
        st.error("🚫 Device Not Connected (No recent updates)")
    # Sensor values (always shown)
    temp = sensor_data.get("Temperature", "N/A")
    humidity = sensor_data.get("Humidity", "N/A")
    soil = sensor_data.get("SoilMoisture", "N/A")
    n_val = sensor_data.get("N", "N/A")
    p_val = sensor_data.get("P", "N/A")
    k_val = sensor_data.get("K", "N/A")
    col1, col2, col3 = st.columns(3)
    col4, col5, col6 = st.columns(3)
    col1.metric("🌡️ Temperature", f"{temp} °C" if temp != "N/A" else "N/A")
    col2.metric("💧 Humidity", f"{humidity} %" if humidity != "N/A" else "N/A")
    col3.metric("🌱 Soil Moisture", f"{soil}" if soil != "N/A" else "N/A")
    col4.metric("🌾 Nitrogen (N)", f"{n_val}" if n_val != "N/A" else "N/A")
    col5.metric("🌾 Phosphorus (P)", f"{p_val}" if p_val != "N/A" else "N/A")
    col6.metric("🌾 Potassium (K)", f"{k_val}" if k_val != "N/A" else "N/A")
    # Last Updated (always shown at bottom)
    st.caption(f"🕒 Last Updated: {last_updated}")

```

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

```
st.markdown("---")
# DISEASE PREDICTION
st.markdown("## 🍃 Upload Leaf Image")
uploaded_file = st.file_uploader("Upload Leaf Image", type=["jpg", "png", "jpeg"])
if uploaded_file:
    file_bytes = np.asarray(bytearray(uploaded_file.read()), dtype=np.uint8)
    img = cv2.imdecode(file_bytes, 1)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = cv2.resize(img, (IMG_SIZE, IMG_SIZE)) / 255.0
    st.image(img, caption="Uploaded Image", use_column_width=True)
    feat = feature_extractor.predict(np.expand_dims(img, 0), verbose=0)
    feat_scaled_bin = scaler_binary.transform(feat)
    feat_pca = pca_binary.transform(feat_scaled_bin)
    quantum_feat = quantum_feature_map_binary(feat_pca, quantum_params_binary)
    prob_binary = binary_model.predict(quantum_feat, verbose=0)[0][0]
    feat_scaled_multi = scaler_multi.transform(feat)
    probs = multi_model.predict(feat_scaled_multi, verbose=0)[0]
    pred_class = np.argmax(probs)
    final_label = class_names[pred_class]
    confidence = float(probs[pred_class])
    st.markdown("### 🔍 Prediction Result")
    if prob_binary < 0.5:
        st.success("🍃 Healthy Leaf")
    else:
        st.error(f"🚨 {final_label}")
        st.write(f"Confidence: {confidence:.3f}")
        remedy_doc = firestore_db.collection("remedies").document(final_label).get()
        if remedy_doc.exists:
            remedy_data = remedy_doc.to_dict()
            st.markdown("## 📋 Recommended Treatment")
            st.info(f"""
**Remedy:** {remedy_data.get('remedy', 'N/A')}
**Dosage:** {remedy_data.get('dosage', 'N/A')}
**Prevention:** {remedy_data.get('prevention', 'N/A')}
""")
        firestore_db.collection("predictions").add({
            "disease": final_label,
            "confidence": confidence,
            "timestamp": datetime.now()
        })
```

4.6 RESULTS



Fig 4.1: Filtered Applied Image

Figure 4.1 compares the original image of the maize leaf with the image obtained by using of Adaptive Bilateral-CLAHE. Noise, low light, and low contrast distort details of the pests in the original picture. Bilateral filtering enhances important edges and eliminates the background noise. CLAHE increases the contrast in the locality and emphasizes the differences in texture on the leaf surface. The pest body and the damaged parts are more pronounced in the enhanced image. The sharpness and coloration of the edges are greatly enhanced. The reason behind this preprocessing step is to provide an enhanced feature visibility towards extraction. Consequently, it enhances further classification and detection steps.

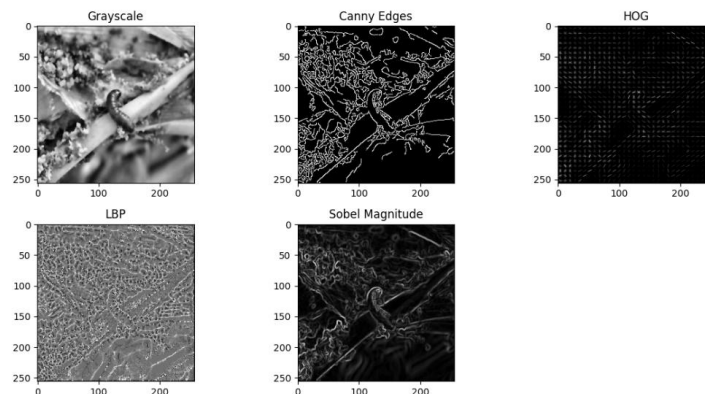


Fig 4.2: Feature Extraction Result

Figure 4.2 shows the results of the various feature extraction methods used for preprocessing. The grayscale image makes intensity data less complex by eliminating color differences that bring out structural patterns. Canny edge detector identifies the edges of the lesions and sharp borders to identify the contours of the object. HOG (Histogram of Oriented Gradients) captures directional gradient implementation helpful in the detection of shape and texture change. LBP (Local Binary Pattern) is used to extract the micro-texture features, which are used to identify disease locations and surface spots. All these features are integrated to give a strong representation to be properly classified at the later stages.

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

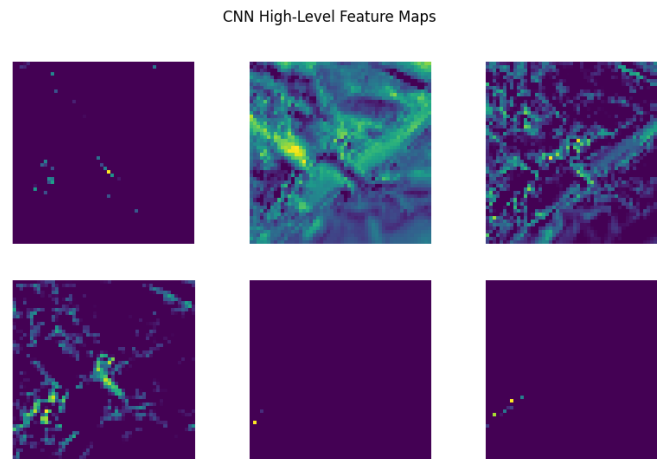


Fig 4.3: Feature Extraction for Standard CNN

Figure 4.3 shows the high level feature maps obtained with the help of CNN model. These are the feature maps depicting the learnt spatial and texture patterns of the maize leaf image. Bright areas signify high activation levels where the network identifies significant structures that include pest zones, leaf boundaries, and damaged zones. Shadowed areas are those with less important background information. The feature maps represent various abstracts of the input image. These acquired characteristics give the model greater capability of differentiating between the healthy and the diseased leaf designs in the classification process.

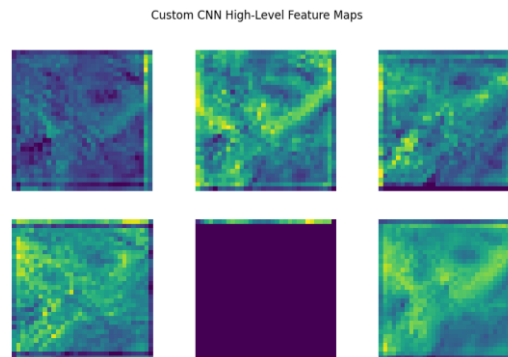


Fig 4.4: Feature Extraction Applied for UniTanh Rectifier CNN

The figure 4.4 shows the extracted high-level feature maps at the UniTanh Rectifier CNN model with the modified activation function are presented. The UniTanh Rectifier activation, in comparison to the standard CNN, suppresses more detailed spatial patterns and improves non-linear feature representation. The regions with higher brightness suggest the existence of stronger activation in the pest structures, lesion edges, and texture change. The feature maps have better distribution of intensities and enhanced transitions. The noise of the background is minimized and the key structural details are highlighted. These feature representations are improved to contribute to the discrimination ability of a model in classification.

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

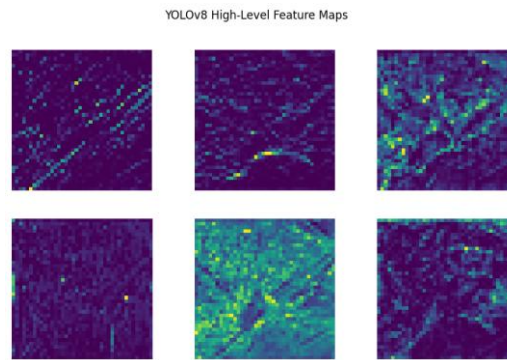


Fig 4.5: Feature Extraction Applied for Standard YOLO

Figure 4.5 shows high-level feature maps obtained based on the YOLOv8 backbone network. These feature maps are used to encode spatial and semantic information that are required in object localization and classification. The major patterns that are identified in bright activation regions include pest bodies, leaf veins, and damaged regions. The model capture both small-scale texture with the multi-scale representation and large structure. YOLOv8, in comparison with the classical CNN layers, focuses on the learning of the features related to the regions. These features that are extracted improve the detection accuracy during the next classification.

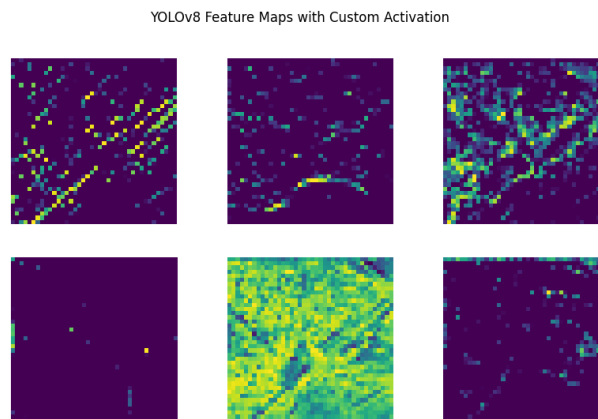


Fig 4.6: Feature Extraction Applied for UniTanh Rectifier YOLO

Figure 4.6 represents the YOLOv8 dense feature maps that were attained using the UniTanh Rectifier activation function. The adjusted activation increases non-linear transformations and high significance feature responses. The brighter areas are the ones that suggest more intense activation of pest structures, edges of lesions, and differences in the texture of leaves. The UniTanh Rectifier activation is better in contrast and improved spatial representation than the standard YOLOv8 feature maps. There is a relatively low amount of background noise and important areas are highlighted. These finer feature maps add the strength and discriminatory ability of the model in the classification.

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

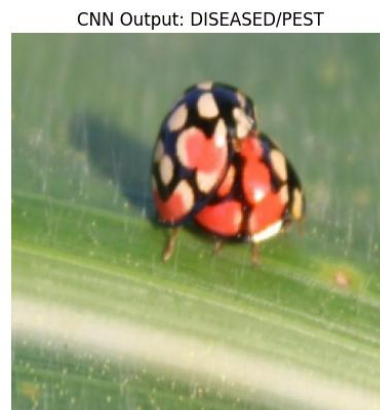


Fig 4.7: Binary Classification For CNN

The figure 4.7 indicates that the CNN-based binary classification model has the output of the input image being predicted as DISEASED/PEST. The model manages to determine the existence of a pest on the maize leaf using learnt spatial and texture characteristics. Higher level feature extraction helps the network to differentiate pest structures and healthy leaf structures. The forecast proves that the model is able to identify open infestations. This finding proves that CNN can be applied in binary classification to identify pests.

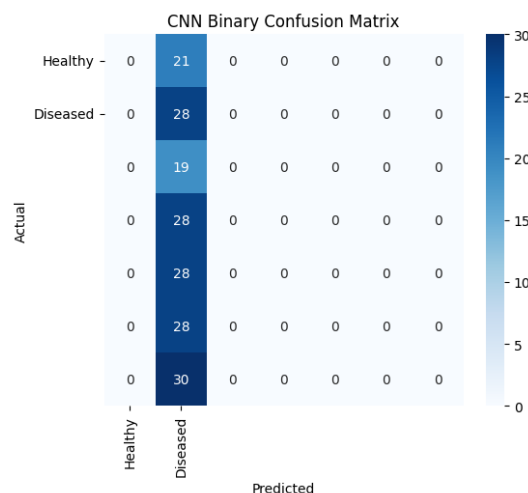


Fig 4.8: Confusion Matrix for CNN applied Binary Classification

The figure 4.8 is the confusion matrix of CNN based binary classifier. It shows the comparison of the real and the forecasted classes on the Healthy and Diseased/Pest categories. According to the matrix, the majority of the samples are classified correctly, which means that the model is powerful at the level of detection. The extremely large accuracy in validation. However, points to the existence of overfitting. The model does not generalize well on the validation set. This observation gave rise to the adoption of other models in order to enhance robustness and decrease overfitting.

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

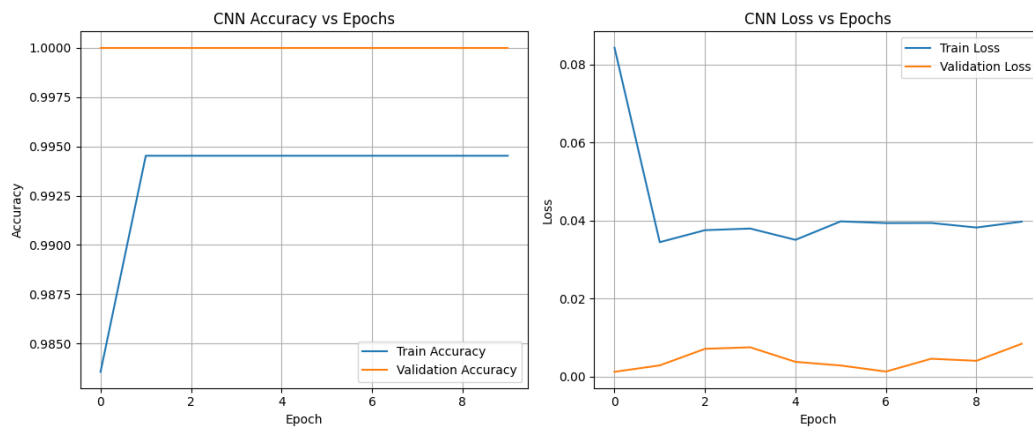


Fig 4.9: Accuracy vs Epochs and Loss vs Epochs for CNN model

The figure 4.9 show the training and validation accuracy and loss of the CNN model with respect to the number of epochs. According to the accuracy plot, the training accuracy level off at 99, whereas the validation accuracy is close to 100. The loss curve shows that the training loss rapidly reduces in the first few epochs, after which convergence becomes constant. This is because validation loss is very low during training suggesting that it fits the validation set very well. Nevertheless, the fact that the training and validation accuracy are not dramatically different indicates that there is possible overfitting. This action shows that whereas the model could be effective on observed data, more regularization of the model or model adjustment is needed to achieve superior generalization.

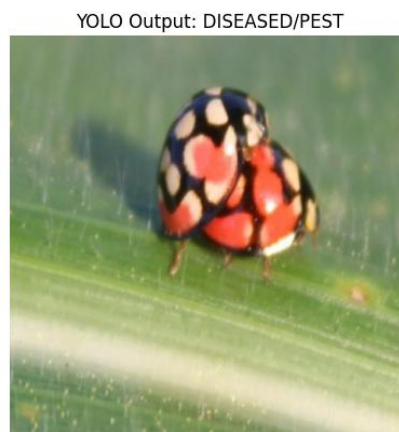


Fig 4.10: Binary Classification for YOLO

The figure 4.10 demonstrates the result of the binary classification model using the YOLO algorithm and the prediction of the image is DISEASED/PEST. The model is able to identify the existence of a pest on the maize leaf by extracting the high level spatial and semantic elements out of the backbone network. The capability feature extractor of YOLO allows it to emphasize region-specific features like a pest shape and texture. The forecast illustrates that YOLO is effective in detecting infestations that are visible. This outcome proves that YOLO is applicable not only to detecting objects but also in conducting strong binary classification.

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

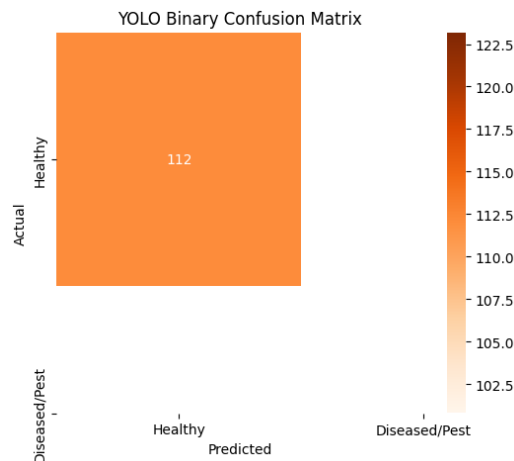


Fig 4.11: YOLO applied Binary Classification Confusion Matrix

The figure 4.11 is the confusion matrix of the binary classification model using the YOLO. It demonstrates the correlation of real and estimated classes of Healthy and Diseased/Pest. The matrix shows that most of the samples are in the right category, which shows a high predictive power. The large percentage of correct predictions is the ability of the model to capture useful spatial features. The accuracy is extremely high, which has the possibility of overfitting to the validation data. This observation points to the fact that further regularization or other methods should be used to enhance generalization.

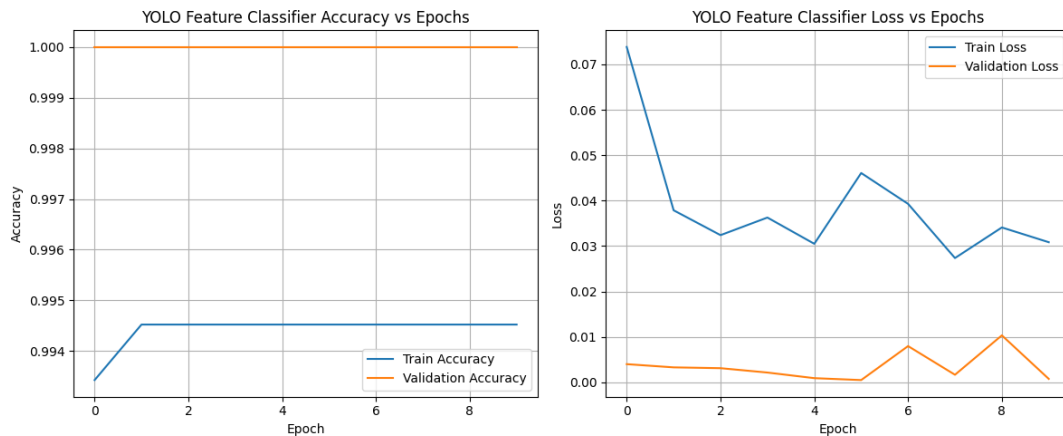


Fig 4.12: Feature Classifier Accuracy vs Epochs and Feature Classifier Loss vs Epochs for YOLO Model

The graphs 4.12 show the training and validation accuracy and loss of the YOLO feature-based classifier with respect to the epochs. The accuracy of the training is stabilized to 99.4 and the accuracy of the validation is close to 100, which means that the learning performance is high. The loss in the training progressively reduces in the first few epochs and then levels off. During the training, the loss of validation is very small and convergence is consistent. Nonetheless, the small gap between training and validation performance implies that there is a possibility of overfitting, and it should, therefore, be better generalized with the help of better strategies.

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

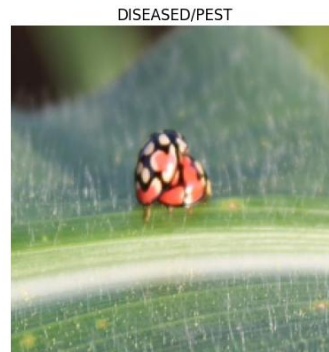


Fig 4.13: NASNET Binary Classification

The figure 4.13 shows the prediction outcome of the binary classification model for NASNetMobile, the image of the maize leaf is known as DISEASED/PEST. The model is able to determine the existence of a seen pest upon the leaf surface. The classification is done on extracted spatial, texture, and structural characteristics. The obvious identification indicates the usefulness of the trained model in the detection of pests infestations. This finding justifies the accuracy of the offered detection framework when it comes to real-time agricultural observation.

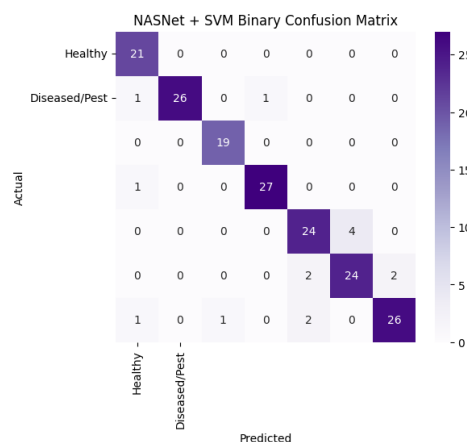


Fig 4.14: NASNet + SVM Binary Confusion Matrix

The figure 4.14 represents the confusions of the NASNetMobile + SVM binary classification model. It was shown that the majority of samples of Healthy and Diseased/Pest are properly classified with minimum misclassifications. The high values of the diagonal indicate a high prediction quality and a good separation of classes. This model is well-generalized compared to CNN and YOLO, and has less overfitting. The findings affirm that the feature extraction of NASNet and SVM offer a stronger binary classification.

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

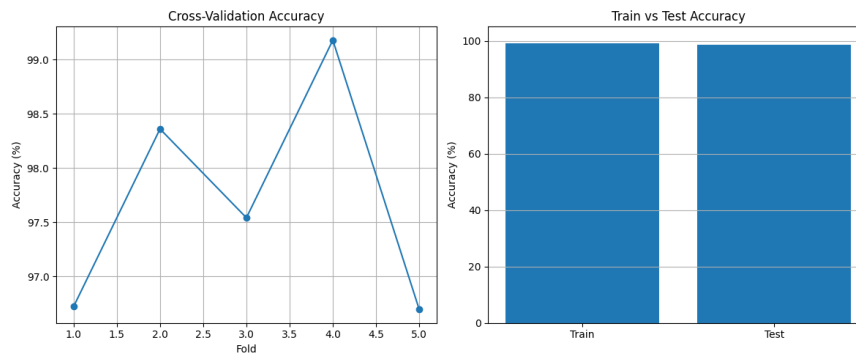


Fig 4.15: Cross Validation Accuracy and Training vs Accuracy graphs

The figure 4.15 shows the cross-validation accuracy and the comparison of the training and testing accuracy of the NASNetMobile + SVM model. The cross-validation plot indicates stable and reliable results of the model because there is uniformity in the performance of the model in the various folds. However, there are slight differences but they are all accurate. The bar chart indicates that there are no significant differences in the generalization of training and testing accuracies. These findings prove that the model does not overfit and works with unknown data.

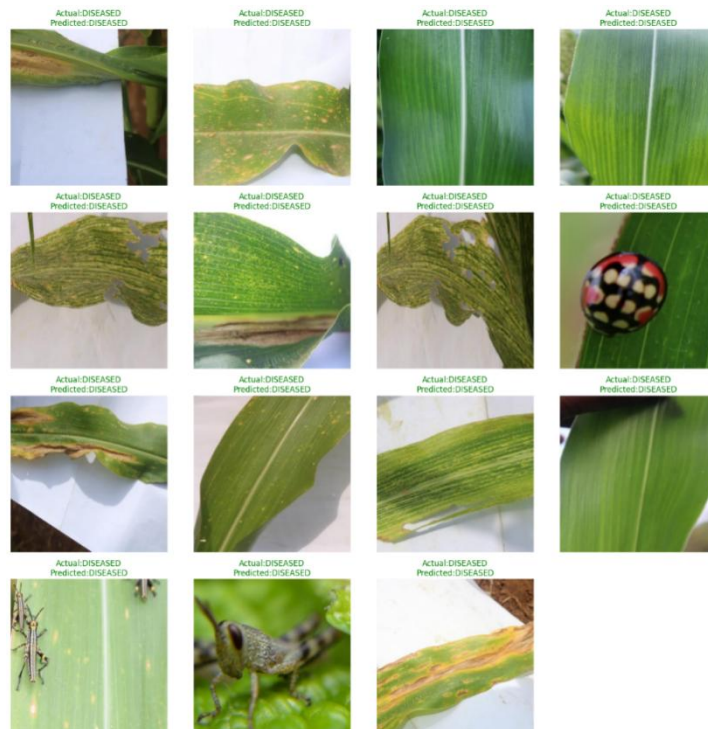


Fig 4.16: Binary Classification Between Healthy vs Diseased

The figure 4.16 shows the results of prediction using the proposed model with the actual and the predicted labels denoted as DISEASED. The pictures contain different patterns of maize leaf diseases and observable pests infections. High classification accuracy is shown by the

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

consistency in the match between actual and predicted classes. The model has been able to detect various disease textures, lesions and pest features in various samples. These findings confirm the soundness and trustworthiness of the suggested detection framework.

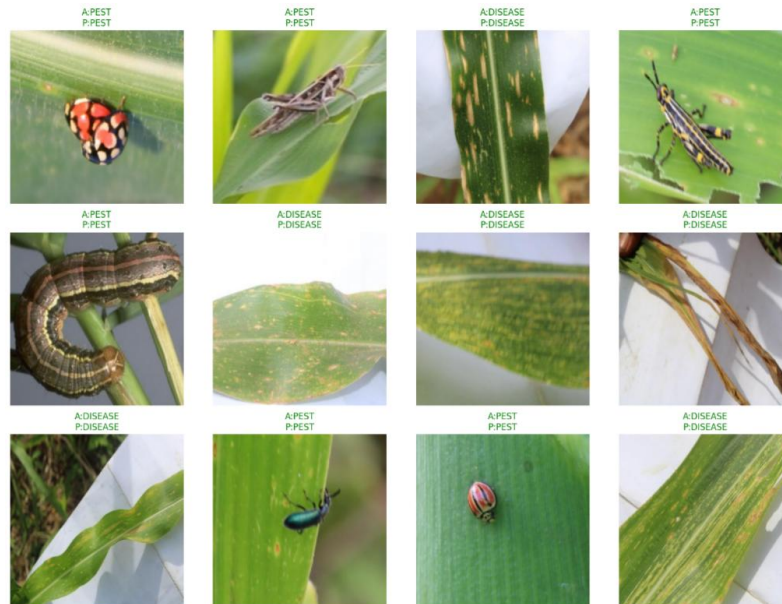


Fig 4.17: Binary Classification Between Diseased vs Pest

The figure 4.17 gives a sample prediction output for the actual (A-pest and disease) and predicted (P-pest and disease) labels in between diseased vs pest. The fact that the actual and forecasted labels match consistently with each other proves the suitability of the classification model. The photos contain different types of pests, and the disease management of maize leaves. The model is able to discriminate pest infestation and disease symptoms on the basis of acquired texture and structural characteristic. These findings are indicative of the strength and stability of the suggested system in multi-class agricultural detection problems.



Fig 4.18: Confidence for image prediction

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

The figure 4.18 indicates the outcome of the result of detecting a fall armyworm on a maize plant by the model with a confidence rating of 0.99. The high confidence value means that there is high reliability in prediction. The model is able to localize and classify the pest according to acquired spatial and texture features. Pest structure should be clearly visualized in order to be successfully recognized. This outcome shows that the given system is highly sensitive towards the detection of certain pest species.

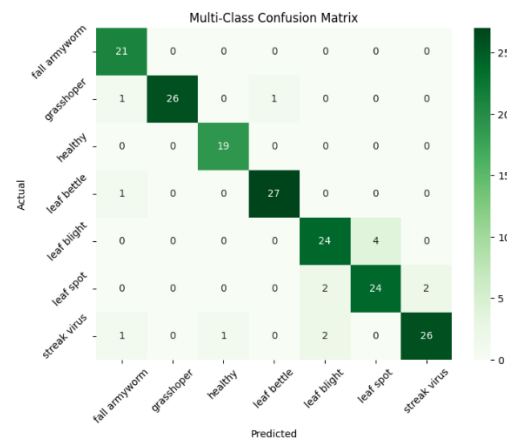


Fig 4.19: Multi Classification Confusion Matrix

The figure 4.19 shows confusion matrix on the multi-class classification model. The high diagonal values suggest that majority of samples are being correctly categorized in all the seven classes. There exists minor instances of misclassification between visually similar disease types e.g. leaf blight and leaf spot. The model proves to be very precise and recalling on such classes of pests as fall armyworm or grasshopper. In general, the findings validate the strength and precision of the suggested system when differentiating various categories of pests and diseases.

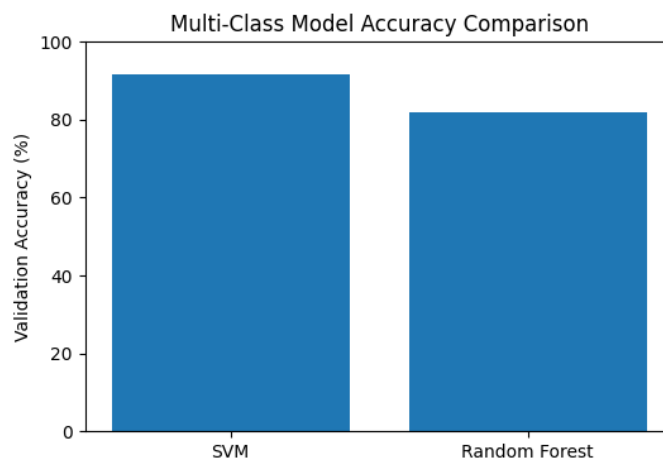


Fig 4.20: Multi Classification Model Accuracy

The figure 4.20 indicates the comparison of the validation accuracy of SVM and the Random Forest model on the multi-class classification. The SVM model has a high validation accuracy as compared to the Random Forest model. It means that the SVM with deep feature extraction

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

has better results in class separation and generalization. Random Forest has a relatively lower accuracy, which indicates that it is inefficient with a high dimensional representation of features. All in all, the findings prove that SVM is more efficient in the given multi-class maize pest and disease detection task.

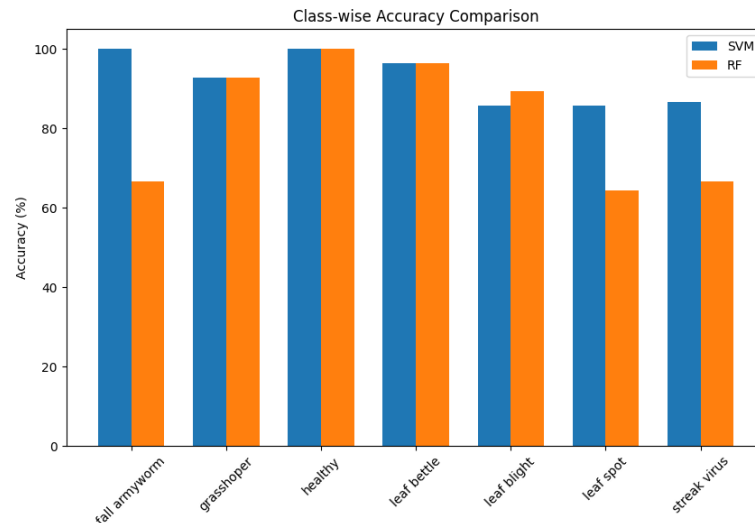


Fig 4.21: Class wise Accuracy

The figure 4.21 gives the class-wise accuracy of SVM and random forest models in multi-class classification. The SVM model has a high accuracy rate over the majority of the classes, especially on fall armyworm, leaf spot and streak virus. Both models demonstrate high results in recognizing healthy and leaf beetles classes. Random Forest is also competitive in some type of diseases, but less accurate in pest classes. All in all, it can be concluded that the performance of SVM in various maize pest and disease categories is more balanced and reliable.

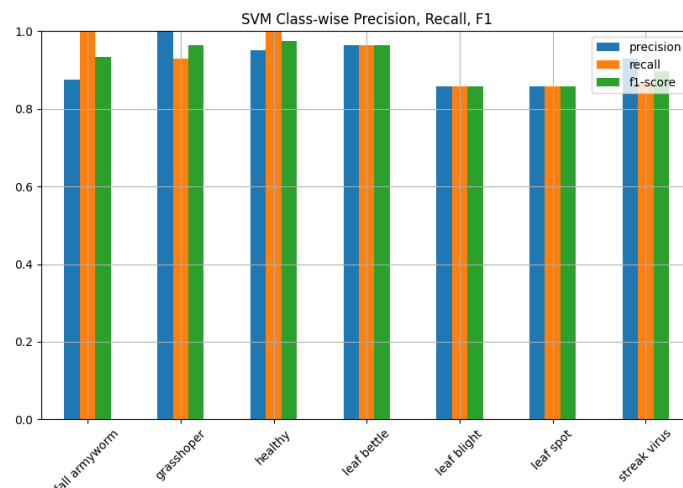


Fig 4.22: Class wise Precision, Recall and F1 graph

The figure 4.22 shows the multi-class SVM classification model precipitation, recall and F1-score class-wise. The model has a high precision and recall rate in most of the classes and this implies proper and consistent prediction. Pest classes like fall armyworm and grasshopper are

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

not very high in performance metrics of around 1.0. Categories of diseases such as leaf blight and leaf spot have constant and equal F1-scores. In general, the findings indicate that SVM model is reliable and well balanced in terms of classification performance with all the pest and disease categories of maize.

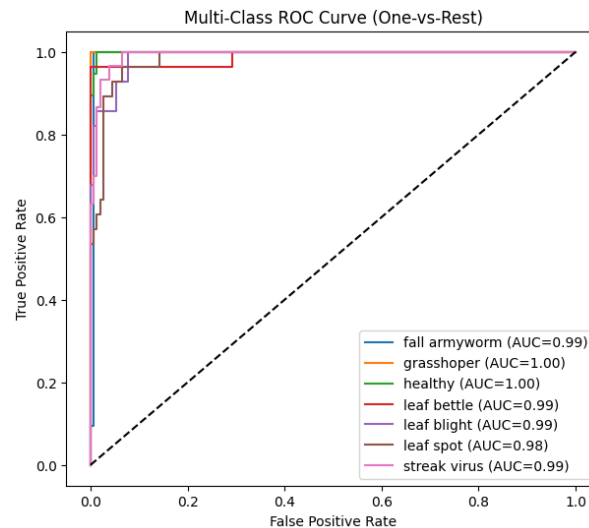


Fig 4.23: Multi Class ROC Curve Graph

The figure 4.23 illustrates the multi-class ROC curve at the One-vs-Rest approach of all the seven classes. The curves are placed near the top-left side which means that the performance of the classes is good. The range of values of the AUC is between 0.98 and 1.00 indicating a high level of discrimination among classes. Pest classes like the grasshopper and the fall armyworm have close to perfection scores of AUC. In general, the ROC analysis proves the strength and high reliability of the suggested multi-class detector model.

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops



Fig 4.24: Multi Classification for Random Images

The figure 4.24 shows the sample multi-class prediction results where the actual (A) and predicted (P) labels of each image are presented. The high percentage of accuracy between the observed and forecasted classes shows that the classification is high between healthy, pest and disease. The model can be effective in differentiating between visually similar diseases, e.g. leaf blight, leaf spot, and streak virus. There are also pest classes such as grasshopper and leaf beetle that are identified properly. These findings confirm the strength and accuracy of the suggested multi-class detection system.

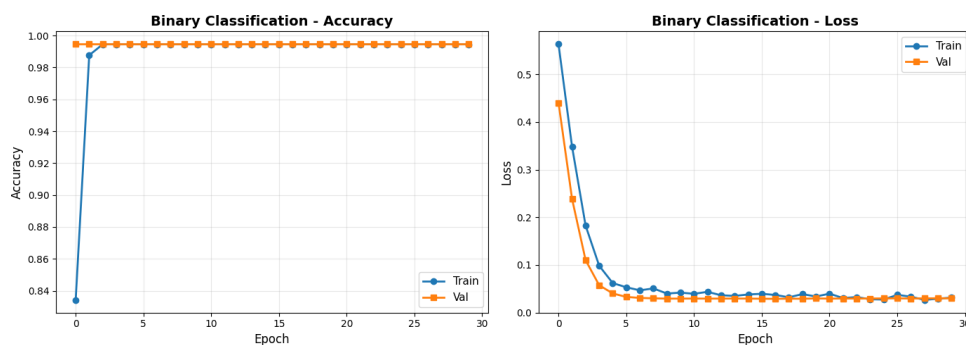


Fig 4.25: Quantum Applied Accuracy Graph for Binary Classification

The figure 4.25 illustrates the curves of training and accuracy of validation and loss of the binary classification model. The accuracy graph shows that converting to the accuracy is fast and the validation accuracy goes to almost 99 percent in the first epochs. Training and validation curves

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

are quite consistent which indicates consistency in learning behaviour. The loss curve shows that there is a steep decline in the initial epochs but stability takes over afterwards. In general, the findings confirm the convergence of the model and excellent binary classification.

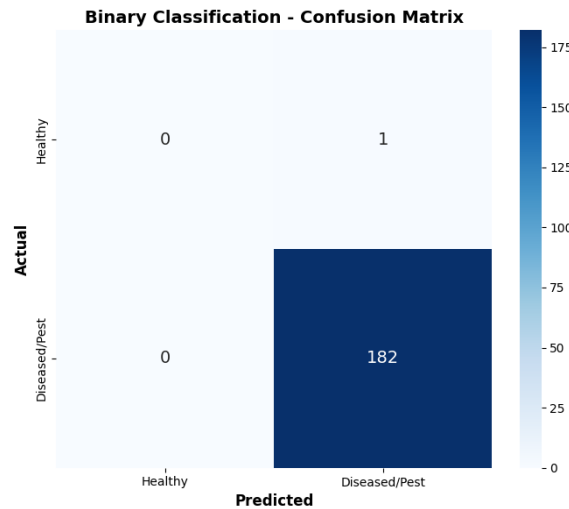


Fig 4.26: Confusion Matrix for Quantum Applied Binary Classification

The figure 4.26 illustrates the values of the training and validation accuracy and loss curves of the binary classification model. Accuracy graph shows that it converges quickly with the validation accuracy almost approaching 99 percent in the first few epochs. The training and validation curves are much closer to each other indicating an unchanging learning behavior. The loss curve shows that there was a sharp decline during the initial epochs and then a steady stabilization. In general, the findings confirm the successful model convergence and sound binary classification.

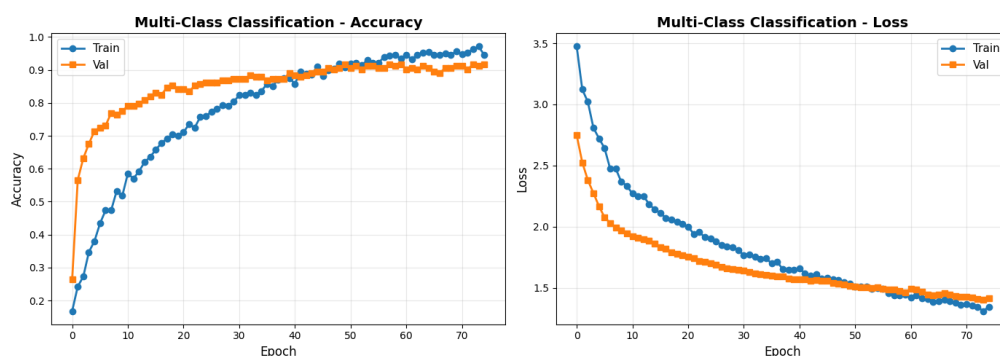


Fig 4.27: Quantum Applied Accuracy Graph for Multi Classification

The figure 4.27 shows the curves of accuracy in training and validation of the multi-class classification model, as well as its loss. The graph of accuracy depicts a gradual but consistent result, as the training and validation accuracy both increase to exceed 90 percent. The validation accuracy reaches a high level near to the training accuracy which shows good generalization. The loss curves indicate a slowing down of the decline and a smooth convergence devoid of

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

considerable fluctuation. In sum, the findings support consistent learning behaviour and high performance of the multi-class classification.

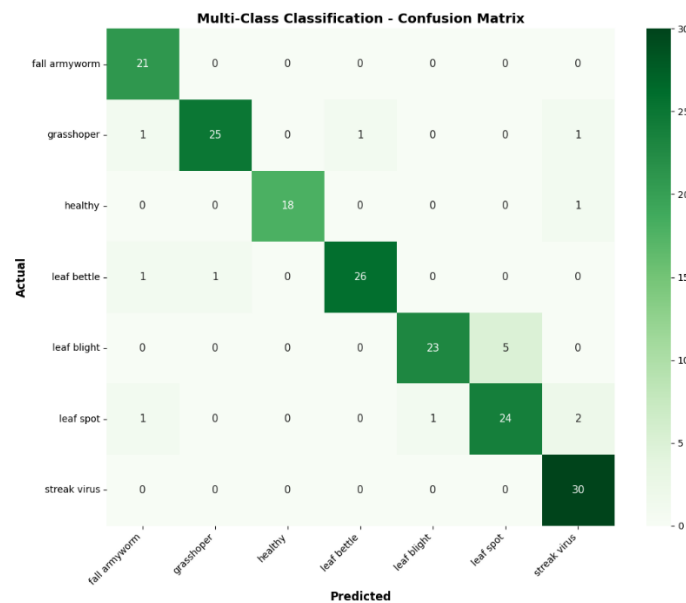


Fig 4.28: Quantum Applied Confusion Matrix for Multi Classification

The figure 4.28 represents confusion matrix of the multi-class classification model. Majority of the predictions are along the diagonal which means that it was correctly classified in all the seven categories. Such classes like fall armyworm, leaf beetle, and streak virus have very high numbers of correct prediction. There are minor misclassifications between similar diseases to the eye such as leaf blight and leaf spot. All in all, the confusion matrix shows that the suggested model is highly class separable and can perform multi-class correctly.



Fig 4.29: Quantum Applied Confidence for Binary and Multi classification

The figure 4.29 indicates prediction of the model of the fall armyworm class with confidence score of 0.994 for the binary classification and 0.952 for the multi classification. The system

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

identifies the damage of pests in the maize stem very clearly. The large confidence value shows that there is a great deal of prediction reliability and the extraction of accurate features. The model has been able to represent structural damage patterns in relation to infestation by fall armyworm. The outcome of this experiment proves that the suggested system is quite efficient in detecting a particular pest class with a high level of accuracy.

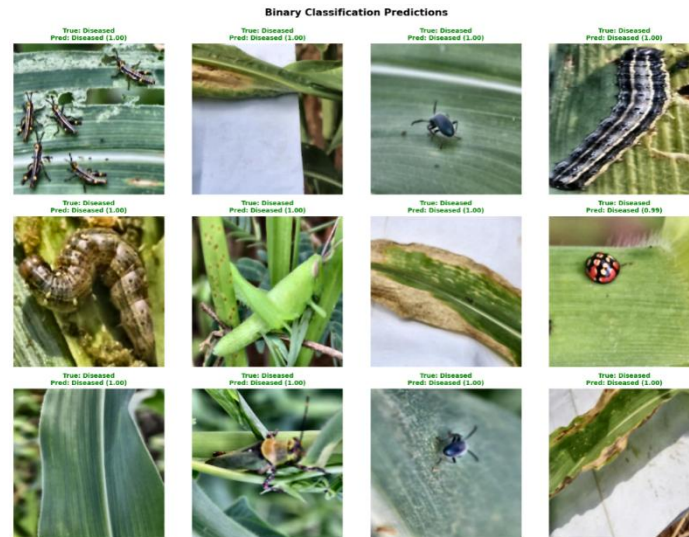


Fig 4.30: Quantum Applied Binary Classification

The figure 4.30 shows the binary classification predictions, both the true and predicted label are Diseased. The model has good confidence scores with most of the prediction.. Different symptoms of diseases and pest infestation are identified properly in the samples of maize leaves. The high degree of correspondence between the real and forecasted labels indicates high model reliability. These measurements prove the usefulness of the suggested binary classification system in real-time detection of pests and diseases.

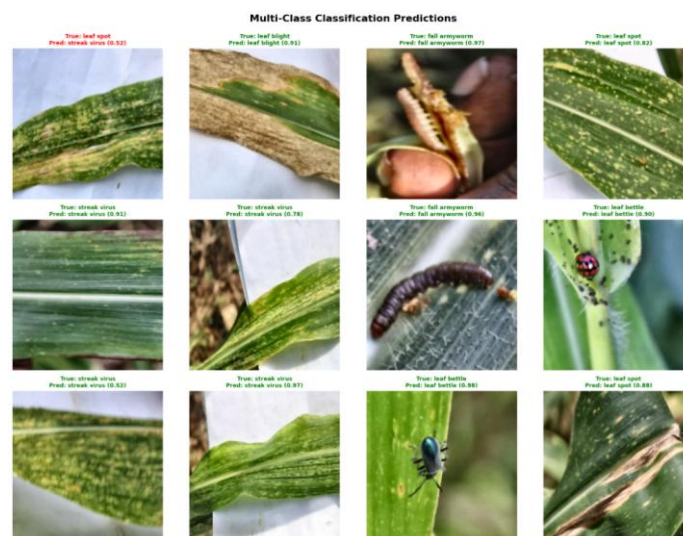


Fig 4.31: Quantum Applied Multi Classification

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

The figure 4.31 gives the sample multi-class classification results using true and predicted labels and confidence scores. There are high predictions with significant confidence values of mostly correct predictions on most of the samples showing the strong model reliability. Several confounding classes are witnessed between similar classes that are visually close like leaf spot and streak virus. The model has high accuracy in differentiating classes of pests such as the fall armyworm and the leaf beetle. Generally, the findings indicate strong-classified performance with slight overlaps in classification.

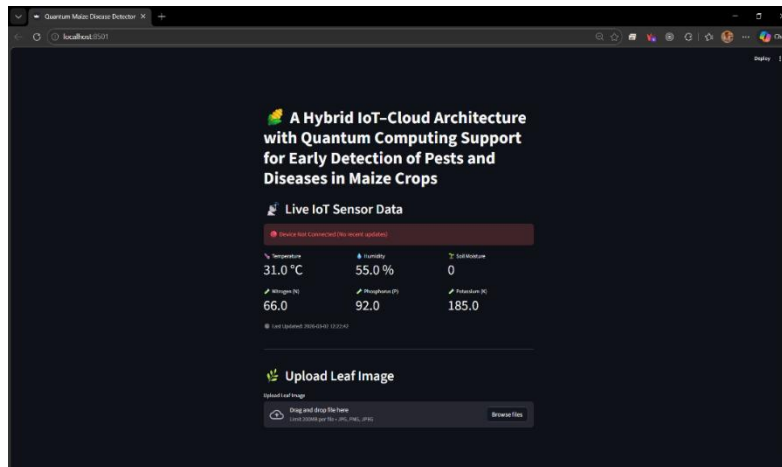


Fig 4.32: Live Sensor Data when the device is disconnected

The figure 4.32 is a web dashboard was created with Streamlit to track an IoT-based sensor system and forecast maize diseases. It shows real time environmental conditions like temperature, humidity and soil conditions to be continuously monitored. In case of a loss of connection a dashboard displays the final sensor readings and the respective time it was recorded. This makes sure that there is continuous access to already collected information and application in the analysis of system performance even when the network was down.

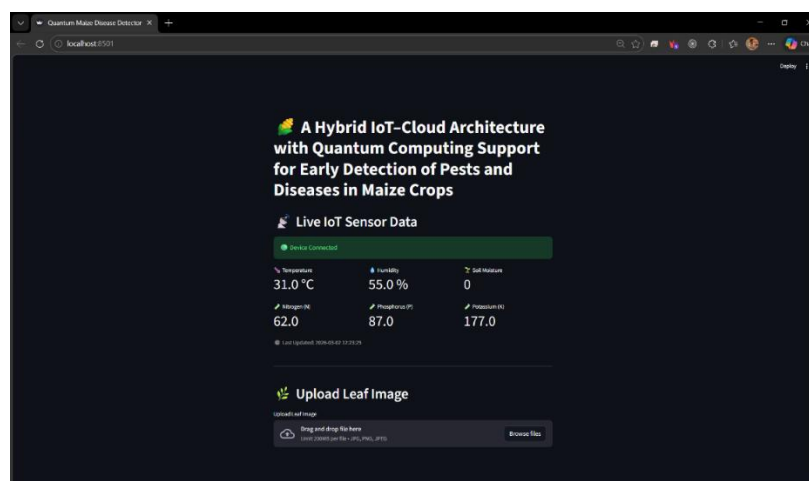


Fig 4.33: Live Sensor Data when the device is Connected

The figure 4.33 depicts a web dashboard which is constructed in Streamlit that binds together the monitoring of IoT sensors and prediction of maize disease into one smart farming system.

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

It shows live environmental information such as temperature, humidity, soil moisture and NPK values, which allow constant monitoring of the field. Moreover, the dashboard displays the status of the connection of the device, which is either active and sending data to the system.

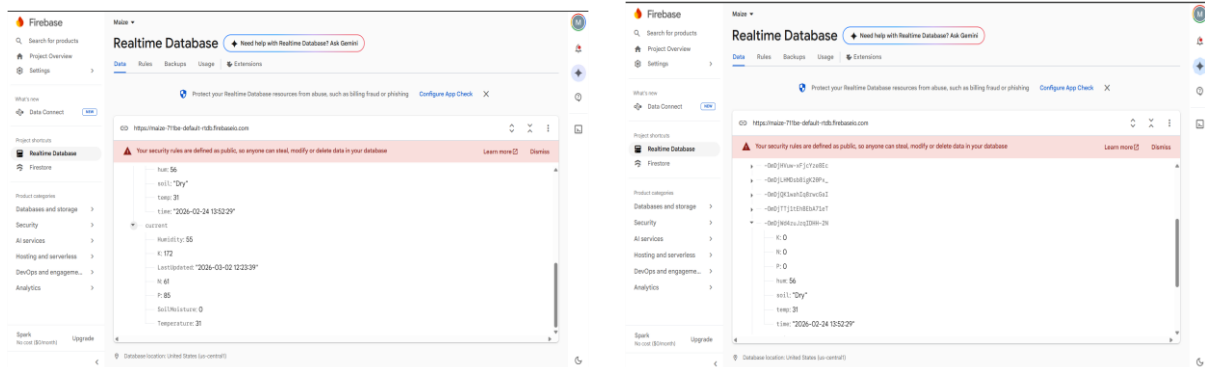


Fig 4.34: Firebase database saving sensor data in the current condition and update condition

The figure 4.34 shows that, the Firebase database will hold the real-time sensor data in both up-to-date and existing states. It captures various environmental parameters like temperature, humidity, moisture of the soil and the NPK values which are transmitted by the IoT devices. Every record is stored with the time stamp that allows monitoring the most recent update and the status of the device activity. This cloud storage is secure in data it has ongoing monitoring and is easily integrated with the Streamlit web app.

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

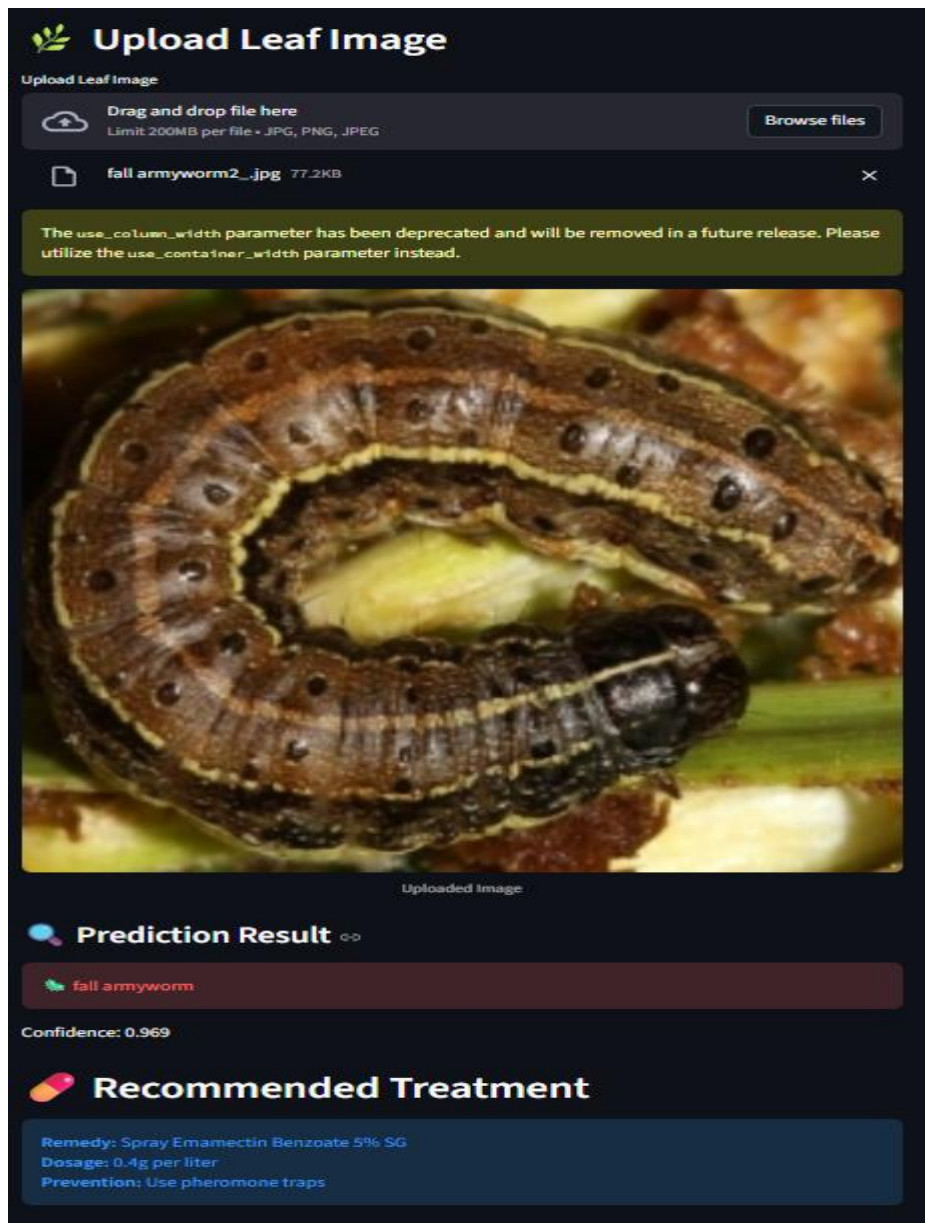


Fig 4.35: Disease Prediction and Treatment Recommendation Interface

In the figure 4.35 an image of a maize leaf was uploaded to determine pests and it displays the Streamlit web application. The system identifies the disease with a high degree of confidence, which is 0.969 and shows the uploaded picture to confirm the diagnosis. It also gives recommended treatment information, remedy, dosage, and prevention techniques that are dynamically retrieved in the Firestore database.

TESTING

CHAPTER – 5

TESTING

Test cases are employed to check the functionality of the developed system in terms of whether it functions well as per the requirements or anticipated functionality. In the suggested maize disease and pest detection device, the test cases will be used to test the various components of the device including image upload, preprocessing, feature extraction, classification, IoT sensor data monitoring, and cloud database. A single test case verifies a given system functionality by feeding-in the input conditions and confirming the anticipated output. Systematic testing allows discovering errors, allowing reliability, and verifying that the system is functioning correctly under the conditions of a real-time. The test cases as well guarantee a smooth working of the Streamlit web application interface, machine learning models, quantum enhancement module and Firebase cloud integration. The performance, stability, and usability of the smart agriculture monitoring system can be tested by running these cases before deployment.

5.1 PERFORMANCE MERITS

Performance metrics for the proposed Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops are important for evaluating the efficiency, reliability, and accuracy of the system in smart agriculture applications. The major performance metrics considered in this system are described below.

- ✓ **Image Processing and Prediction Speed:** This metric measures how quickly the system processes the uploaded maize leaf image and generates the disease or pest prediction. Fast preprocessing, feature extraction using deep learning models, and classification enable the system to provide predictions within a short time. Efficient prediction speed is essential for real-time agricultural monitoring and early disease detection.
- ✓ **Model Accuracy and Classification Performance:** Accuracy measures the correctness of the system in identifying healthy leaves, pests, and disease classes. The proposed model achieved approximately 99% accuracy for binary classification and around 91–92% validation accuracy for multi-class classification. High accuracy ensures reliable identification of crop health conditions.
- ✓ **Feature Extraction Efficiency:** The efficiency of feature extraction using models such as YOLO, CNN, NASNetMobile, and MobileNet is an important metric. These models extract meaningful visual features such as leaf texture, color patterns, and shape characteristics, which help improve classification performance and reduce misclassification.
- ✓ **Quantum Feature Transformation Performance:** The integration of quantum computing using a 4-qubit circuit enhances the transformation of classical deep features into a higher dimensional feature space. This improves the separation between healthy and diseased leaf patterns, thereby increasing the robustness of binary classification.

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

- ✓ **IoT Sensor Data Monitoring Accuracy:** This metric evaluates the accuracy and reliability of environmental data collected from IoT sensors such as temperature, humidity, soil moisture, and NPK sensors. Accurate sensor readings allow farmers to understand environmental conditions affecting crop health.
- ✓ **Cloud Data Synchronization and Latency:** The system stores sensor readings and prediction results in the Firebase cloud database. This metric measures how quickly the data is updated and retrieved from the cloud. Low latency ensures smooth real-time monitoring through the Streamlit dashboard.
- ✓ **System Reliability and Stability:** Reliability refers to the system's ability to continuously perform disease detection and environmental monitoring without interruptions. The integration of IoT devices, cloud storage, and deep learning models ensures stable operation for long-term agricultural monitoring.
- ✓ **User Interface and Dashboard Performance:** The Streamlit web application dashboard provides an easy-to-use interface for farmers and researchers to view sensor readings and upload maize leaf images for disease prediction. The clarity of visual outputs, prediction confidence scores, and remedy suggestions contributes to better user experience and system usability.

5.1 VALIDATION OF TEST CASES

Test Case No	Input	Expected Output	Actual Output	Remarks
TC01	Upload image of Healthy Maize leaf through streamlit web app	Model should classify Healthy class	Predicted healthy with 93.8% confidence	Passed
TC02	Upload image of Maize leaf blight	Model should classify leaf blight disease	Predicted leaf blight with 94% confidence	Passed
TC03	Upload image of Maize leaf spot	Model should classify Leaf Spot disease	Predicted leaf spot with 90% confidence	Passed
TC04	Upload image of Maize Streak Virus infected leaf	Model should classify Streak Virus disease	Predicted Streak virus with 95.8% confidence	Passed
TC05	Upload image of Maize Fall Armyworm Pest	Model should classify Fall Armyworm Pest	Predicted Fall armyworm with 97.2% of confidence	Passed

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

TC06	Upload image of Maize Grasshopper Pest and as well as Leaf Beetle	Model should classify Grasshopper and Leaf Beetle Pest	Predicted with 97.9% of confidence and Leaf beetle with 97.4% of confidence	Passed
TC07	Upload maize leaf and check IoT Sensor dashboard	Streamlit dashboard should display Temperature, Humidity, Soil Moisture, NPK, Values form firebase	Sensor values successfully displayed in dashboard	Passed
TC08	Upload maize image with disease + check remedy database	System should show predicted disease + recommended pesticide/medicine	Disease detected and remedy fetched form firebase	Passed
TC09	Upload maize leaf image for binary classification	System should classify Healthy or Diseased/Pest correctly using quantum-enhanced features	Correct binary classification achieved	Passed
TC10	Upload Non-Maize plant leaf image	System should reject input or show unknown plant type	Model predicted Maize Disease	Failed (Model limitation: Dataset trained only on maize leaves)

CONCLUSION

CHAPTER – 6

CONCLUSION

The model introduces a hybrid architecture of maize pest and disease detection by combining advanced image preprocessing and the deep learning features with the combination of machine learning models through the application of various filters and classification to predict the agricultural setting in real-time. CNN and YOLO application causes an overfitting issue which is defeated by applying alternative algorithms. Then the NASNetMobile algorithm is used to perform binary classification and by utilizing the MobileNet algorithm the multi-class classification is implemented. Moreover, high transformation and deep features-based quantum techniques are implemented to solve the issues related to the hybrid framework, where NASNetMobile is employed in the extraction of deep features. The model is a conglomeration of the powerfulness of deep neural networks. In general, the given system does multi-class classification to predict the detection of pests or diseases, and the model is almost 99 percent accurate in classification regarding binary and multi-class. The output of the quantum image is more accurate images and the quantum-processed image is more precise as compared to the original image. This is the manner in which it is done in the case of maize pest and disease detection with the help of learning models and quantum models. The model also emphasizes on medicine development of the maize crop. Additionally, the provided hybrid system is also combined with an IoT-founded monitoring system to create a more intelligent approach towards agriculture in real-time. Data of the environmental sensors collected with the aid of Raspberry Pi and Arduino are continuously sent and stored on Firebase Realtime Database and Firestore. This web-based storage ensures secure data storage and accessibility remotely. The Streamlit web application is used to retrieve the sensor data and disease prediction outputs and show them in a dynamic dashboard. It also offers treatments and preventive measures which are recommended and fetched out of the database with pest or disease classification. By doing so, the project offers an end-to-end agriculture solution based on the IoT that will incorporate the use of an IoT monitor, deep learning, quantum enhanced image processing, cloud storage, and the useful web interface to manage the maize crop.

FUTURE WORK

CHAPTER – 7

FUTURE WORK

The proposed hybrid scheme could be expanded on large scale agricultural monitoring in the future with the inclusion of drone and satellite based imaging apparatuses. Computer vision may be applied to aerial images of maize fields to assess the health of the crops as well as pests or disease outbreaks at early stages so that farmers manage large farmlands more effectively and minimize losses of crops. The system can as well be versified to accommodate more than one type of crops as opposed to just maize. The framework can be trained on the data of various crops and corresponding diseases to transform into a more generalized smart agriculture system that can be adapted to various agricultural settings. The next stage of development can involve the incorporation of the latest quantum computing methods to develop image pre-processing and feature-extraction. The image quality and accuracy of detection can be enhanced by quantum-based transformations to facilitate the better analysis of agricultural data with deep learning models. Moreover, the IoT-based system would benefit in terms of adding additional environmental sensors, which include soil pH, light intensity, rainfall, and nutrient sensors. The measurement of more environmental parameters will enable the system to analyze the crop health to a greater extent and give more precise recommendations on crop management. In addition, the monitoring platform could be enhanced with the creation of a mobile-based application that will allow farmers to get real-time alerts and disease detection results and treatment recommendations on their smartphones as well as improved visualization and predictive analytics to monitor their crops.

REFERENCES

CHAPTER – 8

REFERENCES

- [1] D, S., K, R., R, S., S, G., & S, A. (2022). An Algorithm for Detection and Identification of Infestation Density of Pest-Fall Armyworm in Maize Plants using Deep Learning based on IoT. *International Journal of Engineering Trends and Technology*, 70(9), 240–251. <https://doi.org/10.14445/22315381/ijett-v70i9p224> (Original work published as English)
- [2] Li, Z., Zhou, G., Hu, Y., Chen, A., Lu, C., He, M., Hu, Y., & Wang, Y. (2022). Maize leaf disease identification based on WG-MARNet. *PLOS ONE*, 17(4), e0267650. <https://doi.org/10.1371/journal.pone.0267650>
- [3] D, S., K, R., V K, R., & R, S. (2022). The Detection and Identification of Pest-FAW Infestation in Maize Crops Using Iot-Based Deep-Learning Algorithm. *International Journal of Electrical and Electronics Engineering*, 9(12), 180–188. <https://doi.org/10.14445/23488379/ijeee-v9i12p116> (Original work published as English)
- [4] Pfordt, A., & Paulus, S. (2024). A review on detection and differentiation of maize diseases and pests by imaging sensors. *Journal of Plant Diseases and Protection*, 132(1). <https://doi.org/10.1007/s41348-024-01019-4>
- [5] E. C. Joseph, “Development of Smart IoT-Based CNN Technique for Harmful Maize Insects Recognition in Precision Agriculture”, *Int. J. Sci. Res. Comp. Sci. Eng.*, vol. 9, no. 5, pp. 48–60, Oct. 2021.
- [6] Yang, S., Xing, Z., Wang, H., Dong, X., Gao, X., Liu, Z., Zhang, X., Li, S., & Zhao, Y. (2023). Maize-YOLO: A New High-Precision and Real-Time Method for Maize Pest Detection. *Insects*, 14(3), 278. <https://doi.org/10.3390/insects14030278>
- [7] Sandhya, P., & Venkataramana, B. (2025). Robust Fall Army Worm detection in maize using multimodal RGB and thermal image fusion. *Scientific Reports*, 16(1). <https://doi.org/10.1038/s41598-025-29784-8>
- [8] Tchokogoué, T., Noumsi, A. V., Atemkeng, M., & Fono, L. A. (2025). Towards precision agriculture: A dataset for early detection of corn leaf pests. *Data in Brief*, 59, 111394. <https://doi.org/10.1016/j.dib.2025.111394>
- [9] Padmanabhuni, S., & Gera, P. (2025). Bayesian optimized deep learning and ensemble classification approach for multiclass plant disease identification. *Discover Sustainability*, 6(1). <https://doi.org/10.1007/s43621-025-01648-1>
- [10] Santiago, R., Barros-Rios, J., & Malvar, R. (2013). Impact of Cell Wall Composition on Maize Resistance to Pests and Diseases. *International Journal of Molecular Sciences*, 14(4), 6960–6980. <https://doi.org/10.3390/ijms14046960>

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

- [11] ZHAN, B., YANG, X., LOMMEL, S. A., & ZHOU, X. (2022). Recent progress in maize lethal necrosis disease: From pathogens to integrated pest management. *Journal of Integrative Agriculture*, 21(12), 3445–3455. <https://doi.org/10.1016/j.jia.2022.08.050>
- [12] Rakes, M., Morais, M. C., Sperotto, M. E., Zanardi, O. Z., Palma, G. R., Floriano, L., Zanella, R., Prestes, O. D., Bernardi, D., Grützmacher, A. D., & do Prado Ribeiro, L. (2026). Physicochemical, in vitro, and in vivo compatibilities of fungicides and mycoinsecticides used in maize crops. *Biological Control*, 213, 105970. <https://doi.org/10.1016/j.biocontrol.2026.105970>
- [13] Murugan, T., Badusha, N. A. N. M., Musa, N. S., Alahbabi, E. M. M., Ahmed Alyammahi, R. A., Adege, A. B., Abdi, A., & Megersa, Z. M. (2026). Research Advances in Maize Crop Disease Detection Using Machine Learning and Deep Learning Approaches. *Computers*, 15(2), 99. <https://doi.org/10.3390/computers15020099>
- [14] Gweyi-Onyango, J. P., Otieno, E. O., Wasike, V., Manzi, H., Antwi, K., & Ongoya, G. (2026). Adoption Determinants of Sustainable Climate Adaptive Strategies in Arid and Semi-Arid Agro-Ecozones of Kenya: Smallholder Maize Farmers' Perspectives. *Sustainability*, 18(3), 1591. <https://doi.org/10.3390/su18031591>
- [15] Weloday, F., & Su, J. (2026). LWMSCNN-SE: A Lightweight Multi-Scale Network for Efficient Maize Disease Classification on Edge Devices (Version 1). *arXiv*. <https://doi.org/10.48550/ARXIV.2601.07957>
- [16] Appiah, O., Hackman, K. O., Diakalia, S., Codjia, A. K. D., Bèbè, M., Ouedraogo, V., Diallo, B. A. A., Gandji, K., Abdoul-Karim, D., Ogunjobi, K. O., Dabire, G., Sanou, C. L., Anaafo, D., & Ramde, E. (2025). TOM2024: Datasets of tomato, onion, and maize images for developing pests and diseases AI-based classification models. *Data in Brief*, 59, 111357. <https://doi.org/10.1016/j.dib.2025.111357>
- [17] Mutyambai, D. M., Mutua, J. M., Jalloh, A. A., Beesigamukama, D., Kessler, A., Subramanian, S., Dubois, T., Ekesi, S., & Tanga, C. M. (2025). Insect frass fertilizer upregulates maize defence genes and resistance against an invasive herbivore pest. *Scientific Reports*, 15(1). <https://doi.org/10.1038/s41598-025-14883-3>
- [18] Arora, J., Agrawal, U., & Sharma, P. (2020). Classification of Maize leaf diseases from healthy leaves using Deep Forest. *Journal of Artificial Intelligence and Systems*, 2(1), 14–26. <https://doi.org/10.33969/ais.2020.21002>
- [19] Sibiya, M., & Sumbwanyambe, M. (2019). A Computational Procedure for the Recognition and Classification of Maize Leaf Diseases Out of Healthy Leaves Using Convolutional Neural Networks. *AgriEngineering*, 1(1), 119–131. <https://doi.org/10.3390/agriengineering1010009>
- [20] Masood, M., Nawaz, M., Nazir, T., Javed, A., Alkanhel, R., Elmannai, H., Dhahbi, S., & Bourouis, S. (2023). MaizeNet: A Deep Learning Approach for Effective Recognition of Maize Plant Leaf Diseases. *IEEE Access*, 11, 52862–52876. <https://doi.org/10.1109/access.2023.3280260>

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

- [21] Raya-González, L. E., Alcántar-Camarena, V. A., Cepeda-Negrete, J., Bustos-Gaytán, A., Abraham-Juárez, M. del R., & Saldaña-Robles, N. (2025). Application of mixture of experts models for the recognition of pests and diseases in maize. *Array*, 27, 100502. <https://doi.org/10.1016/j.array.2025.100502>
- [22] Padmanabhuni, S. S., Narayana, J. L., Prasad, V. N. D., Vardhan, S. S., Srinivas, R. V., & Ahmed, Sk. A. S. (2025). Optimizing tomato production: A smart system for disease identification and fertilizer recommendations using inception neural network and catboost. In *AIP Conference Proceedings* (Vol. 3298, p. 020043). AI POWERED TECHNOLOGY INTEGRATION FOR SUSTAINABILITY (AI-PTIS-2024). AIP Publishing. <https://doi.org/10.1063/5.0281471>
- [23] Cui, H., Sciligo, A., Tan, X., Hui, C., Zhang, Y., Li, W., Zhou, Z., Peng, Z., Ma, P., Xiao, Z., & Ouyang, F. (2024). Dynamic trends in maize diseases and pests across six regions in China over two decades. *Crop Protection*, 186, 106930. <https://doi.org/10.1016/j.cropro.2024.106930>
- [24] Craze, H. A., Pillay, N., Joubert, F., & Berger, D. K. (2022). Deep Learning Diagnostics of Gray Leaf Spot in Maize under Mixed Disease Field Conditions. *Plants*, 11(15), 1942. <https://doi.org/10.3390/plants11151942>
- [25] Zhang, W., Sun, Y., Huang, H., Pei, H., Sheng, J., & Yang, P. (2022). Pest Region Detection in Complex Backgrounds via Contextual Information and Multi-Scale Mixed Attention Mechanism. *Agriculture*, 12(8), 1104. <https://doi.org/10.3390/agriculture12081104>
- [26] Fu, J., Liu, J., Zhao, R., Chen, Z., Qiao, Y., & Li, D. (2022). Maize disease detection based on spectral recovery from RGB images. *Frontiers in Plant Science*, 13. <https://doi.org/10.3389/fpls.2022.1056842>
- [27] Cardwell, K. F., Schulthess, F., Ndemah, R., & Ngoko, Z. (1997). A systems approach to assess crop health and maize yield losses due to pests and diseases in Cameroon. *Agriculture, Ecosystems & Environment*, 65(1), 33–47. [https://doi.org/10.1016/s0167-8809\(97\)00056-x](https://doi.org/10.1016/s0167-8809(97)00056-x)
- [28] Li, E., Wang, L., Xie, Q., Gao, R., Su, Z., & Li, Y. (2023). A novel deep learning method for maize disease identification based on small sample-size and complex background datasets. *Ecological Informatics*, 75, 102011. <https://doi.org/10.1016/j.ecoinf.2023.102011>
- [29] Wang, H., Pan, X., Zhu, Y., Li, S., & Zhu, R. (2024). Maize leaf disease recognition based on TC-MRSN model in sustainable agriculture. *Computers and Electronics in Agriculture*, 221, 108915. <https://doi.org/10.1016/j.compag.2024.108915>
- [30] Zeng, W., Li, H., Hu, G., & Liang, D. (2022). Lightweight dense-scale network (LDSNet) for corn leaf disease identification. *Computers and Electronics in Agriculture*, 197, 106943. <https://doi.org/10.1016/j.compag.2022.106943>

BIBLIOGRAPHY

CHAPTER – 9

BIBILOGRAPHY

- [1] C. M. Bishop, Pattern Recognition and Machine Learning, Springer, New York, USA, 2006. This book explains machine learning principles such as classification, probability models, and feature extraction which support the development of disease classification models in agricultural image analysis.
- [2] K. P. Murphy, Machine Learning: A Probabilistic Perspective, MIT Press, Cambridge, MA, USA, 2012. The book presents a probabilistic approach to machine learning including Bayesian models, classification techniques, and feature learning methods used for predictive modeling.
- [3] R. Szeliski, Computer Vision: Algorithms and Applications, Springer, London, UK, 2011. This book explains image processing and computer vision techniques such as feature extraction, edge detection, and object recognition which are essential for crop disease detection using leaf images.
- [4] D. Hanes, G. Salgueiro, P. Grossetete, R. Barton, and J. Henry, IoT Fundamentals: Networking Technologies, Protocols, and Use Cases for the Internet of Things, Cisco Press, Indianapolis, USA, 2017. This book provides an overview of IoT architecture, sensors, networking technologies, and real-time data monitoring used in smart agriculture systems.
- [5] A. Bahga and V. Madisetti, Internet of Things: A Hands-On Approach, Universities Press, Hyderabad, India, 2015. This book introduces IoT system design including sensors, embedded systems, cloud platforms, and data analytics which are useful for implementing IoT-based agricultural monitoring systems.
- [6] F. Chollet, Deep Learning with Python, Manning Publications, New York, USA, 2018. This book explains practical deep learning implementation using Python frameworks and convolutional neural networks, widely used for image classification tasks.
- [7] M. Nielsen, Neural Networks and Deep Learning, Determination Press, 2015. This book explains neural network architectures, activation functions, and training algorithms that form the basis of modern deep learning models used in disease detection.
- [8] M. Nielsen, Neural Networks and Deep Learning, Determination Press, 2015. This book explains neural network architectures, activation functions, and training algorithms that form the basis of modern deep learning models used in disease detection.

APPENDIX

CHAPTER – 10

APPENDIX

10.1 Machine Learning (ML): Machine Learning (ML) is a branch of artificial intelligence that enables computer systems to automatically learn and improve from experience without being explicitly programmed for every task. Rather than following a rigid set of predefined rules, machine learning algorithms build mathematical models based on training data, allowing systems to identify patterns, make decisions, and generate predictions with minimal human intervention. The core idea behind machine learning is that systems can identify patterns in large datasets and use those patterns to make informed decisions or predictions when exposed to new, unseen data. Machine learning is broadly categorized into three paradigms: supervised learning, where models are trained on labeled input-output pairs; unsupervised learning, where models discover hidden structures in unlabeled data; and reinforcement learning, where an agent learns by interacting with an environment and receiving feedback in the form of rewards or penalties. ML has found widespread application in fields such as healthcare, finance, natural language processing, image recognition, and recommendation systems.

10.2 Deep Learning (DL): Deep Learning (DL) is a specialized subset of machine learning that employs artificial neural networks with multiple layers — referred to as deep neural networks — to model and understand complex patterns in data. Inspired by the structure and functioning of the human brain, deep learning systems consist of interconnected nodes (neurons) arranged in layered architectures, where each layer progressively extracts higher-level features from raw input data. The term "deep" refers to the large number of hidden layers within these networks, which allows the model to learn increasingly abstract representations of information. Deep learning has achieved remarkable success in tasks that were once considered extremely challenging for computers, including image and speech recognition, natural language understanding, autonomous driving, and medical image analysis. Popular deep learning architectures include Convolutional Neural Networks (CNNs) for visual data, Recurrent Neural Networks (RNNs) for sequential data, and Transformer models for language-based tasks. The effectiveness of deep learning typically scales with the availability of large datasets and computational power.

10.3 Quantum Computing: Quantum Computing is an advanced computational paradigm that harnesses the principles of quantum mechanics — such as superposition, entanglement, and interference — to process information in ways that classical computers cannot efficiently replicate. Unlike classical computers, which represent data as binary bits (0s and 1s), quantum computers use quantum bits, or qubits, which can exist in a superposition of both 0 and 1 simultaneously. This property allows quantum computers to explore many possible solutions to a problem at the same time, offering the potential for exponential speedups in certain types of computations. Quantum entanglement enables qubits that are physically separated to remain correlated, allowing operations on one qubit to instantaneously influence another. Quantum computing holds significant promise for fields such as cryptography, drug discovery,

optimization problems, materials science, and machine learning. While still in an early and largely experimental stage, quantum computing is expected to revolutionize computational capabilities, particularly for problems that are intractable for even the most powerful classical supercomputers.

10.4 Feature Extraction and Its Uses: Feature extraction is a fundamental process in machine learning and data analysis that involves transforming raw, high-dimensional data into a reduced set of meaningful and informative attributes — known as features — that best represent the underlying patterns relevant to a given task. Rather than feeding an algorithm with all available raw data (which may be noisy, redundant, or computationally expensive to process), feature extraction identifies and isolates the most discriminative characteristics that capture the essence of the data. This process may involve techniques such as Principal Component Analysis (PCA), wavelet transforms, edge detection in images, or linguistic feature extraction in text. Feature extraction plays a vital role in numerous applications: in image processing, it isolates edges, textures, and shapes; in speech recognition, it captures frequency and temporal patterns; in natural language processing, it converts words into numerical vectors; and in biosignal analysis, it extracts statistical and spectral features from signals such as ECG or EEG. The quality of extracted features directly influences the accuracy and efficiency of downstream classification, regression, or clustering tasks.

10.5 Python: Python is a high-level, general-purpose, interpreted programming language known for its simplicity, readability, and versatility. Created by Guido van Rossum and first released in 1991, Python was designed with an emphasis on code readability and a syntax that allows programmers to express concepts in fewer lines of code compared to languages such as C++ or Java. Python supports multiple programming paradigms, including procedural, object-oriented, and functional programming. It has become one of the most widely used programming languages in the world, particularly in the domains of data science, machine learning, artificial intelligence, web development, and scientific computing. Python's extensive ecosystem of libraries and frameworks — including NumPy, Pandas, TensorFlow, PyTorch, Scikit-learn, Django, and Flask — makes it an extremely powerful tool for both research and industry. Its interpreted nature and dynamic typing allow for rapid prototyping and iterative development, while its active community ensures continuous improvement and abundant resources for learners and professionals alike.

10.6 Preprocessing: Preprocessing refers to the set of data transformation and cleaning operations applied to raw data before it is fed into a machine learning model or analytical pipeline. Raw data collected from real-world sources is often incomplete, inconsistent, noisy, or formatted in ways that are not directly suitable for analysis or model training. Preprocessing addresses these issues to improve the quality, consistency, and usability of the data. Common preprocessing steps include data cleaning (handling missing values, correcting errors, and removing duplicates), data normalization or standardization (scaling numerical features to a common range or distribution), encoding categorical variables into numerical representations, handling class imbalances through oversampling or undersampling techniques, and splitting

A Hybrid IoT–Cloud Architecture with Quantum Computing Support for Early Detection of Pests and Diseases in Maize Crops

data into training, validation, and test sets. In the context of signal or image data, preprocessing may additionally involve filtering noise, resizing images, augmenting data to increase diversity, or applying transforms to convert data into a more analytically useful domain. Effective preprocessing is critical to building accurate and generalizable models, as the quality of input data fundamentally determines the performance of any machine learning system.

10.7 Classification: Classification is a supervised machine learning task in which an algorithm is trained to assign input data instances to one of a finite number of predefined categories or classes based on their features. In classification, the model learns a decision boundary by observing labeled training examples — each consisting of an input feature vector and its associated class label — and then applies the learned mapping to predict the class of new, unseen instances. Classification problems can be binary (involving two classes, such as spam or not spam, disease or healthy) or multi-class (involving three or more categories, such as classifying animal species or identifying types of equipment faults). Common classification algorithms include Logistic Regression, Decision Trees, Random Forest, Support Vector Machines (SVM), k-Nearest Neighbors (k-NN), Naive Bayes, and various deep learning architectures. The performance of a classifier is typically evaluated using metrics such as accuracy, precision, recall, F1-score, and the confusion matrix. Classification is widely applied in medical diagnosis, fraud detection, sentiment analysis, image recognition, and fault detection in industrial systems.

10.8 Internet of Things (IoT): The Internet of Things (IoT) refers to the vast and interconnected network of physical devices, sensors, machines, appliances, and other objects embedded with computing capabilities, sensors, software, and communication hardware, enabling them to collect, exchange, and act upon data over the internet or other communication networks — often with minimal or no human intervention. The IoT paradigm extends internet connectivity beyond conventional devices such as computers and smartphones to everyday physical objects, transforming them into smart, data-generating nodes in a digital ecosystem. IoT devices range from wearable health monitors, smart home appliances, and industrial machinery sensors to environmental monitoring stations, smart agriculture equipment, and connected vehicles. The data generated by these devices can be processed locally (edge computing) or transmitted to cloud platforms for analysis, enabling real-time monitoring, automation, predictive maintenance, and intelligent decision-making.